Welcome to this course, where we will move beyond basic prompting and explore how Claude Code can work with external tools, structured workflows, and reusable capabilities. This course is designed to help you understand how Claude Code becomes significantly more powerful when connected to the right tools in the right way.

Prerequisites

Before you begin, there are a few prerequisites. You should already be familiar with the following:

- The basics of Claude Code, including simple prompting, the Scratchpad workflow, and the role of Claude.md.
- Using the terminal and working inside VS Code.
- Handling basic git commands.
- You are also expected to have a code editor and Node.js installed on your machine.

Course outline

This course is about building structured, useful, and repeatable workflows with Claude Code. We will cover several key areas in a logical progression:

1. We will start with MCP (Model Context Protocol) fundamentals and the server setup process.
2. Next, we will move into real-world workflows using the file system MCP, GitHub MCP, and Playwright MCP.
3. Along the way, we will cover ways to control and shape these workflows using files like AGENTS.md.
4. Finally, we will learn to package repeatable workflows using Claude Code skills.

Learning outcomes

By the end of this course, you will understand not just what MCP is, but how to use it in practical, real-world scenarios. You will learn how to:

- Connect Claude Code to external servers.
- Validate and guide Claude’s behavior.
- Work through a complete issue-to-pull-request workflow in GitHub.
- Interact with live websites using the Playwright MCP.
- Build reusable skills on top of these capabilities.

About Zenva Academy

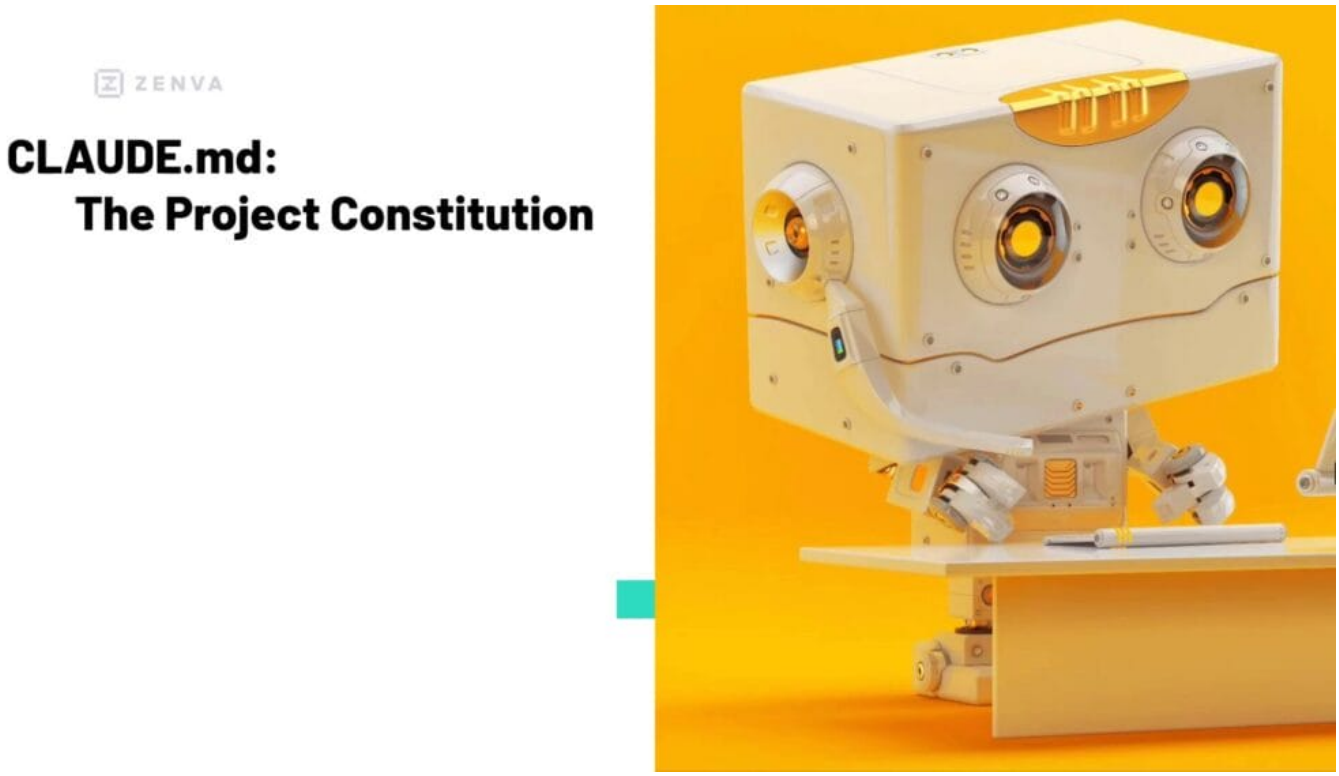
Zenva serves over a million learners with a comprehensive catalog spanning foundational courses and specialized tracks in game development, web development, and AI tools. You can learn through step-by-step video instruction or concise lesson summaries directly on the platform. Now, let’s get started.

Before jumping into MCP servers, agents, and skills, it is worth pausing to recalibrate how Claude Code actually thinks. This foundation determines whether MCP feels chaotic or whether it feels powerful. Claude Code is not just a chatbot that answers questions: it behaves like a structured development system that runs on rules, short-term working memory, planning, and constrained execution. Once this mental model is clear, everything added later, especially external tool access, will make much more sense.

Defining the rules with claude.md

The easiest way to understand the claude.md file is to treat it like the constitution of your project. It describes the rules of the environment in which Claude Code operates, and it sets the boundaries around what Claude is and is not allowed to do. A well-written claude.md typically covers:

- What must not change.
- The architectural constraints that must be respected.
- The coding style that is expected.
- The security boundaries that must be maintained.
- The performance expectations for the project.
- The file structure policies that should stay stable.



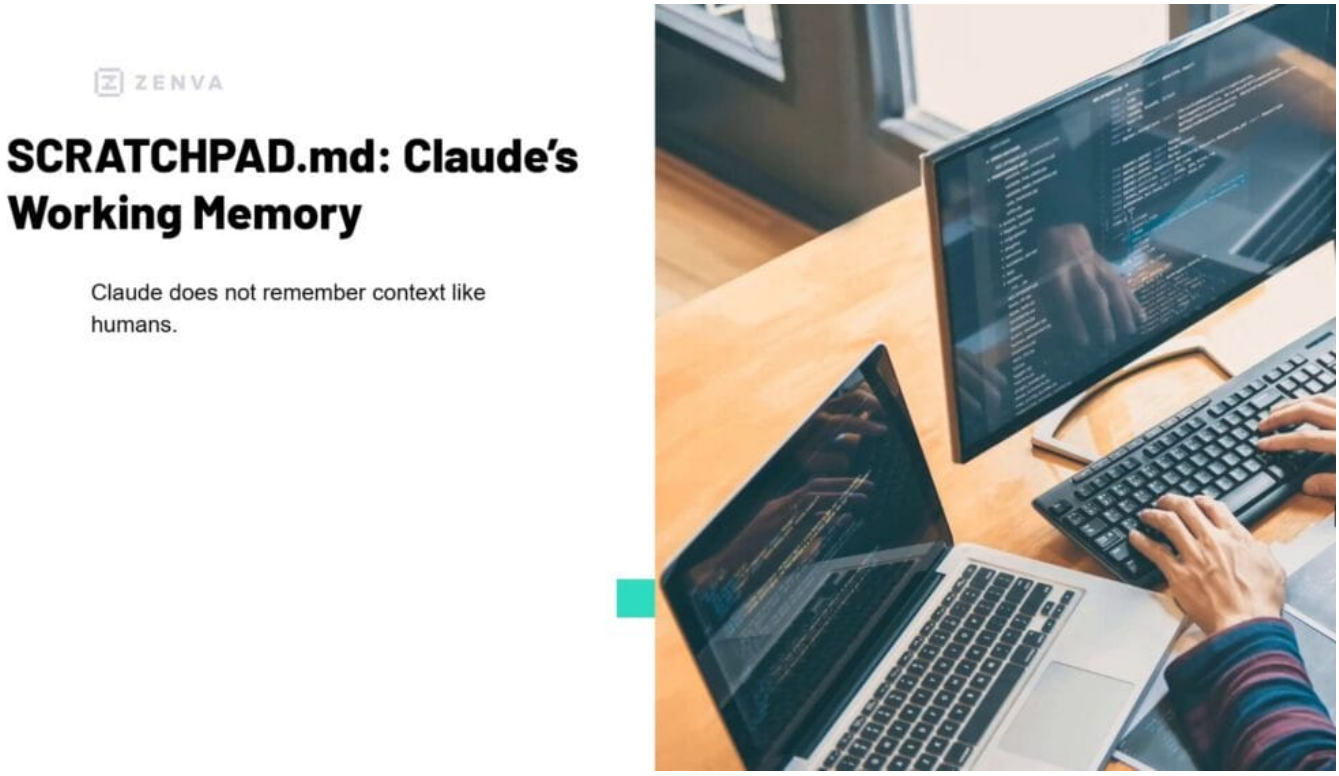
Without a claude.md file, Claude Code will often try to be helpful in ways you did not explicitly ask for. It might clean up code, reorganize the structure, or perform refactors that seem reasonable to the model but fall outside your intended scope. Writing the rules down explicitly makes Claude Code respect the existing structure. This matters because the model is optimized to infer goals, predict future needs, and sometimes expand scope in order to deliver a more complete solution. A clear claude.md prevents silent refactors, architectural drift, and other unintended changes by defining the playing field from the start.

Working memory with scratchpad.md

The next piece of the mental model is the scratchpad file, usually called scratchpad.md, which

functions like working memory. Claude Code does not remember context the way humans do. Instead, it builds a temporary reasoning state while it works on a task. The scratchpad gives that reasoning a dedicated place to live, covering things like:

- Planning tasks and outlining steps.
- Tracking checklists and progress.
- Coordinating complex operations.
- Staging ideas before implementation.

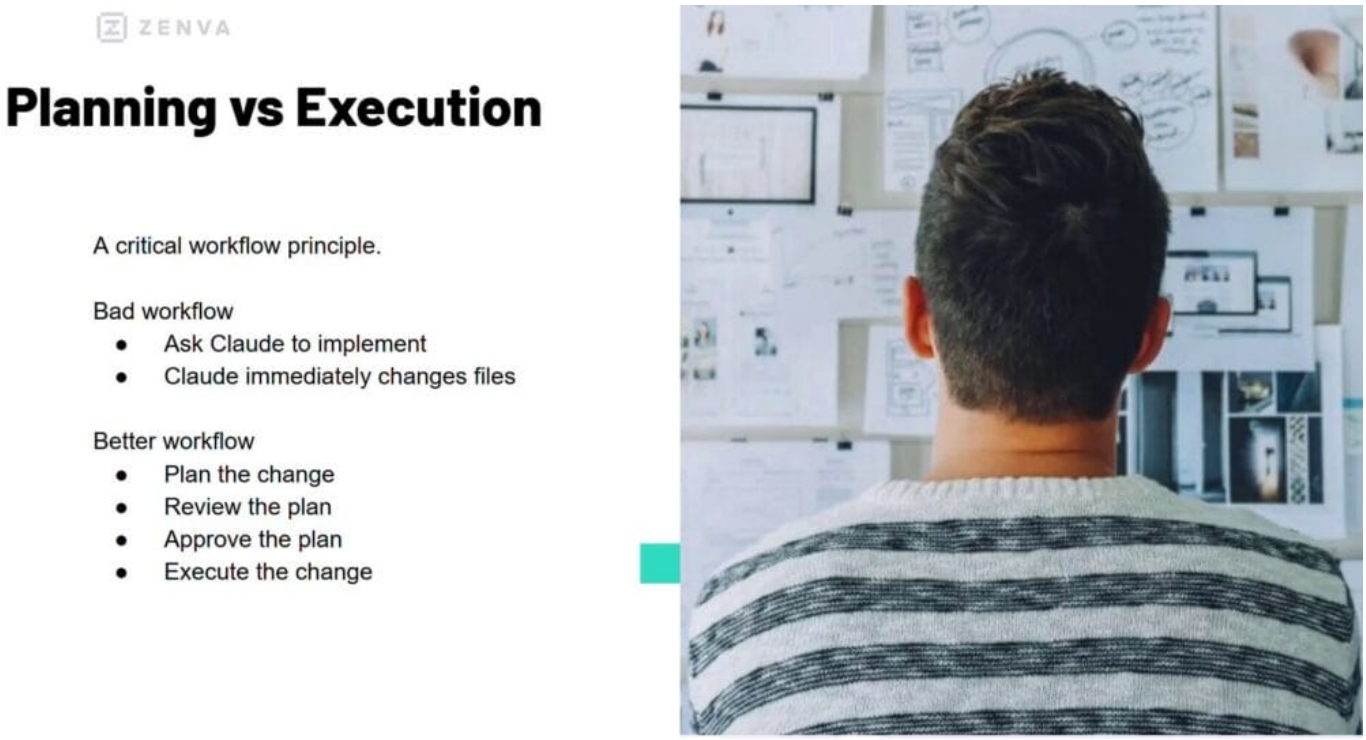


When Claude Code operates without a scratchpad, the reasoning happens implicitly. That is where risks such as context drift, half-finished approaches, abandoned plans, and scope expansion start to appear. When a scratchpad is used properly, planning becomes visible and execution becomes controlled, creating a clean separation between thinking and doing. That separation is fundamental for any effective agentic workflow.

Planning before execution

A common mistake when people start using agentic tools is collapsing planning and execution into a single step. Someone asks to “implement this feature,” and the model immediately begins changing files. That is execution without constraint, and it routinely leads to errors. A much safer workflow splits the work into four distinct phases:

1. Plan the required changes.
2. Review the proposed plan.
3. Approve the plan.
4. Execute the plan.



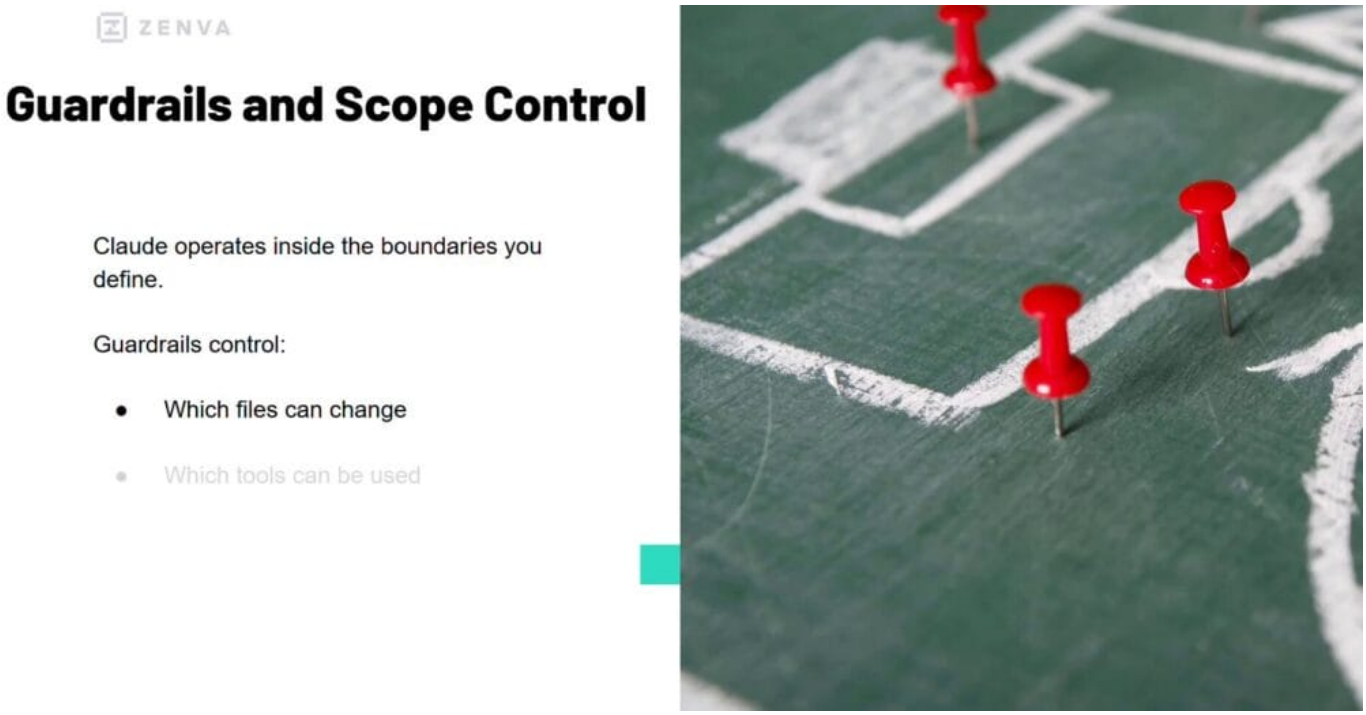
During the planning phase, Claude Code should identify which files will be affected, clarify what must not change, estimate the scope of the work, and flag any risks. During the execution phase, it should stick to minimal, localized changes that follow the constraints you have already defined. Operating this way keeps you in control, because without it Claude can become proactive in ways you never authorized.

Guardrails and scope constraints

Claude Code operates inside the boundaries you set, and the language you use to describe a task directly shapes its scope. If you say “refactor this logic,” you have granted broad scope and should expect broad changes. If instead you say “make a small, localized improvement without changing the structure,” you have given a constrained scope, and the model is far more likely to behave predictably.

Guardrails give you that precise control. For any meaningful task you should define:

- Which files can be changed.
- Which tools can be used.
- What actions require explicit approval.
- How large the scope of the work is.

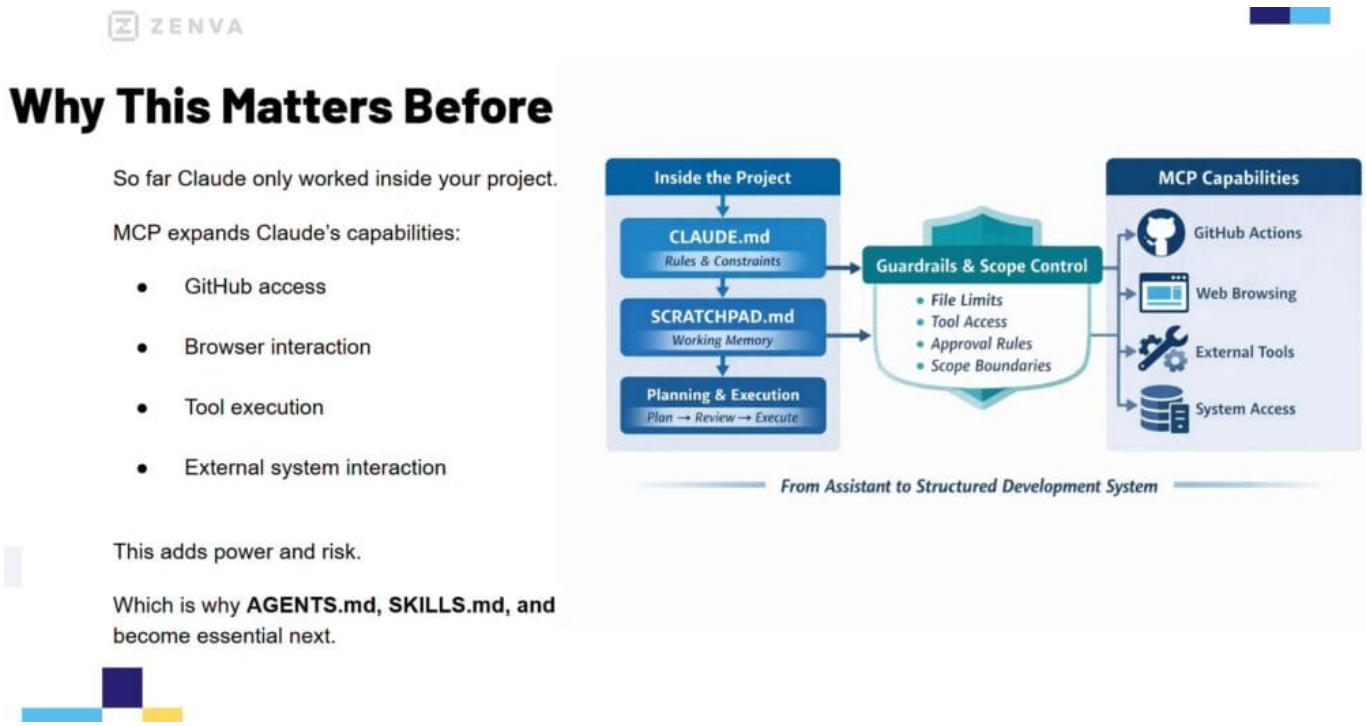


These guardrails are not optional. They are the difference between a helpful assistant and an unpredictable agent, and defining them consistently is what prevents random, unexpected behavior from creeping into your project.

Extending the model with MCP

So far, Claude Code has mostly been operating inside your project, reading code, editing files, and planning changes. The Model Context Protocol (MCP) changes that by giving Claude external capabilities. With MCP, Claude Code can:

- Communicate with services such as GitHub.
- Open and drive a web browser.
- Interact with the file system through structured interfaces.
- Call external tools with clearly defined parameters.



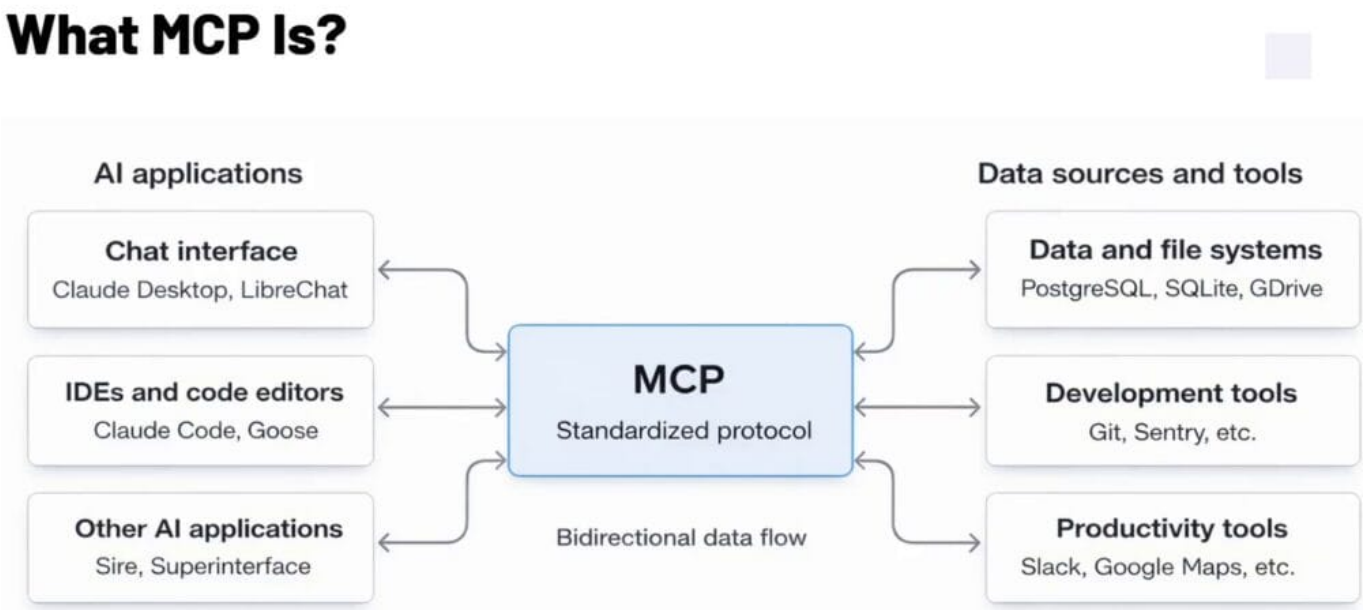
This dramatically increases Claude Code's power, but it also increases risk, because tool access can change real-world external state. That is why concepts like Agents.md, Skills.md, tool definitions, and delegation control become essential once MCP is introduced. MCP is not a replacement for claude.md and scratchpad.md; it is an extension of them. You still need rules, memory, planning, and guardrails. With MCP, you are simply adding capabilities that reach beyond the local codebase.

In the next lesson, we will look at exactly what MCP is and how it enables Claude Code to move from a project assistant into a structured development system that can act in the wider world.

So far, Claude Code has mostly operated inside your local project. It can read files, edit code, follow the rules you set in CLAUDE.md, and use a scratchpad to plan before it executes. That is already useful, but its reach stops at the boundary of your local workspace. The real shift happens when you want Claude to go beyond that boundary and interact with external systems, such as opening a GitHub pull request, extracting data from websites, running structured tools in your workspace, or calling APIs to fetch data. To do that reliably and safely, Claude needs a standardized connection layer, and that layer is MCP.

What is the Model Context Protocol (MCP)?

MCP stands for Model Context Protocol. The simplest way to understand it is as a universal connector for language models. Think about why USB-C is so successful: it is a global, standardized protocol for charging and data transfer, so you can plug it into almost anything and it just works. MCP aims to play a similar role for large language models. It is a standardized protocol that gives models a consistent way to connect to other services and resources, whether those resources are data, tools, or applications on your computer or on the internet.



Why MCP is necessary

This matters because language models generate text, but tools require structure. GitHub does not understand a vague instruction like “please make a pull request.” A browser cannot act on “go to something smart.” APIs do not accept intentions, they accept parameters. The real problem is not whether Claude can reason about what to do, but how Claude can reliably turn an intention into a valid tool call without inventing parameters, breaking formats, or hallucinating results.

MCP solves this by providing a protocol that defines:

- How tools and resources are described.
- How tools are called.
- How results are returned.

In other words, MCP creates a structured contract between Claude and external capabilities.

MCP system architecture

MCP systems are built around three main roles:

1. **MCP host:** the application using MCP, for instance, Claude Code.
2. **MCP client:** the component that maintains the protocol connection and translates model intentions into formal MCP calls.
3. **MCP server:** the component that exposes tools and resources and executes the requested actions safely.

From a practical perspective, the key point is that this protocol brings the utility of a natural language interface to almost anything that has an MCP server built for it. If there is an MCP server for GitHub, your natural language instruction can become a real GitHub action. If there is an MCP server for Playwright, your natural language instruction can become real browser actions.

MCP is also open source and client agnostic, which means anyone can build an MCP server and any application that already leverages LLMs can hook into MCP. The ecosystem is currently very developer focused, but the direction is clear: user interfaces are changing rapidly, integrations are getting easier, and what feels technical now will likely become a normal product capability soon. In the future, when you create an application, you will probably also create an MCP server for it so that users can automate tasks with natural language.

What MCP servers typically provide

When you look at what MCP servers typically provide, you can think in three core categories:

- **Resources:** data exposed by the server, such as files, repository metadata, web content, or structured datasets. Anything the model can retrieve and use as context.
- **Prompt templates and workflows:** reusable prompts and standard operating procedures that improve the consistency of the model’s outputs.
- **Tools:** the actual actions the server can perform, such as creating a branch, opening a pull request, navigating to a URL, extracting headings, or clicking an element.

These three categories alone already unlock most practical use cases.

Advanced concepts: sampling and routes

MCP also includes more advanced concepts that matter for agentic systems. Two worth knowing are sampling, where a server can request completions from an LLM, and routes, where the client informs the server about the boundaries within which it is allowed to operate. Those boundaries are not a minor detail. They are the difference between controlled automation and uncontrolled automation, because they define where the MCP server is permitted to act.

Power comes with responsibility

With MCP, agents can now modify repositories, interact with services, and affect external state. That means safety boundaries become critical. The more independently an agent can act, the larger the potential threat surface becomes. Giving a model the ability to take actions, especially actions that affect external state, means you must design for safety. MCP is not something you add in isolation: it expands capability, and that capability must be matched by appropriate guardrails.

Now that we understand what the Model Context Protocol (MCP) is, we need to look at something even more important: how Claude actually calls tools. This is the point where reliability either happens or breaks down completely. Tools do not work because the model is smart, they work because of contracts. A tool contract is what makes tool usage predictable, and in this lesson we will explore what those contracts look like, why they prevent hallucinations, the role of schemas, and how all of this connects to agentic systems.

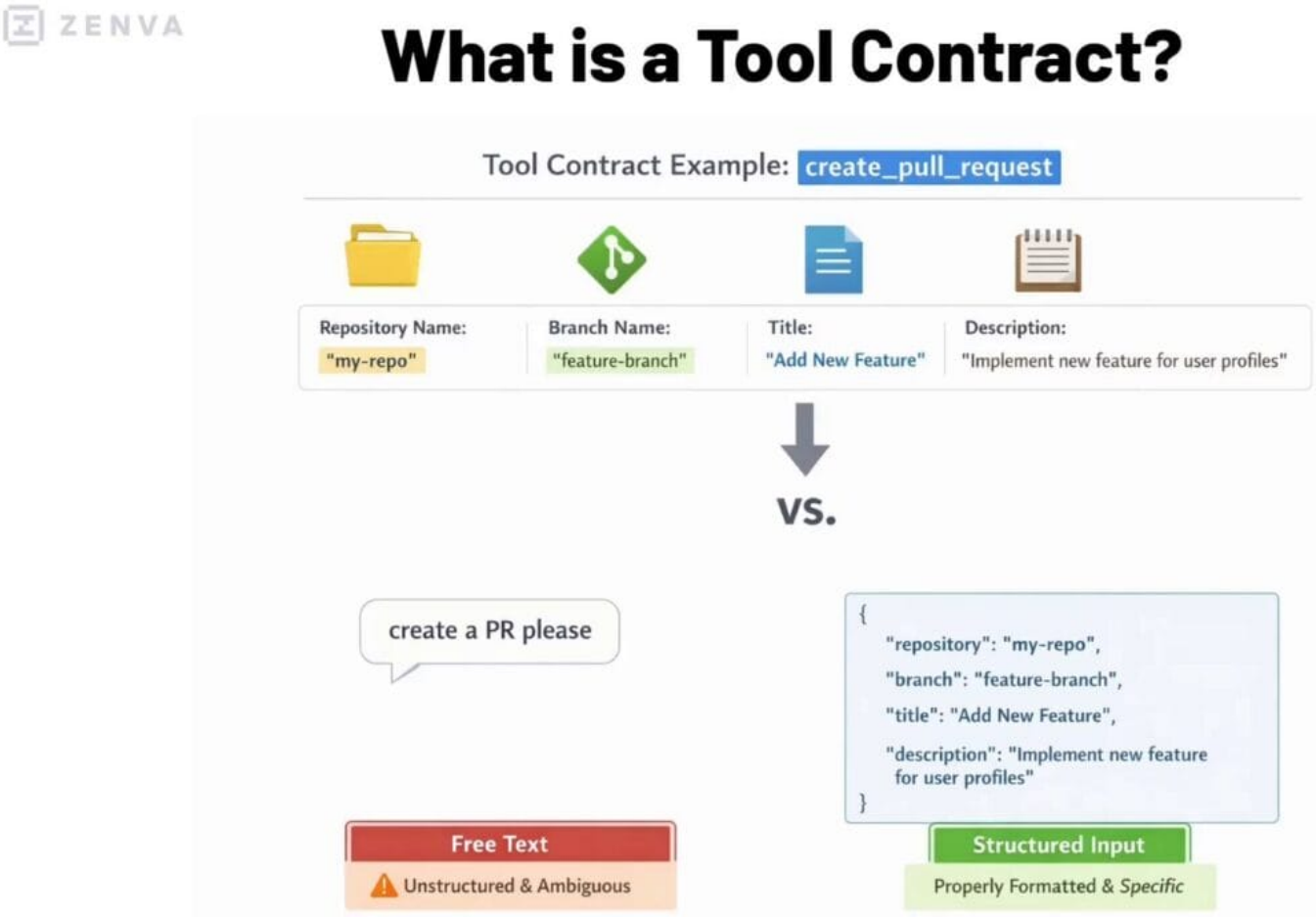
What is a tool contract?

A useful way to think about a tool contract is as a structured definition. It essentially says: this tool exists, here is what it does, and here is how you call it. Rather than letting the model invent its own way to invoke a tool, the contract spells out the rules in advance.

Consider a tool named create_pull_request. Its contract might require four inputs:

1. A repository name
2. A branch name
3. A title
4. A description

Each of those inputs must follow a specific structure. That means Claude cannot simply say “create a pull request, please” and hope it works. It has to provide properly formatted arguments that match the tool’s expectations. This is the difference between free text instructions and structured execution.



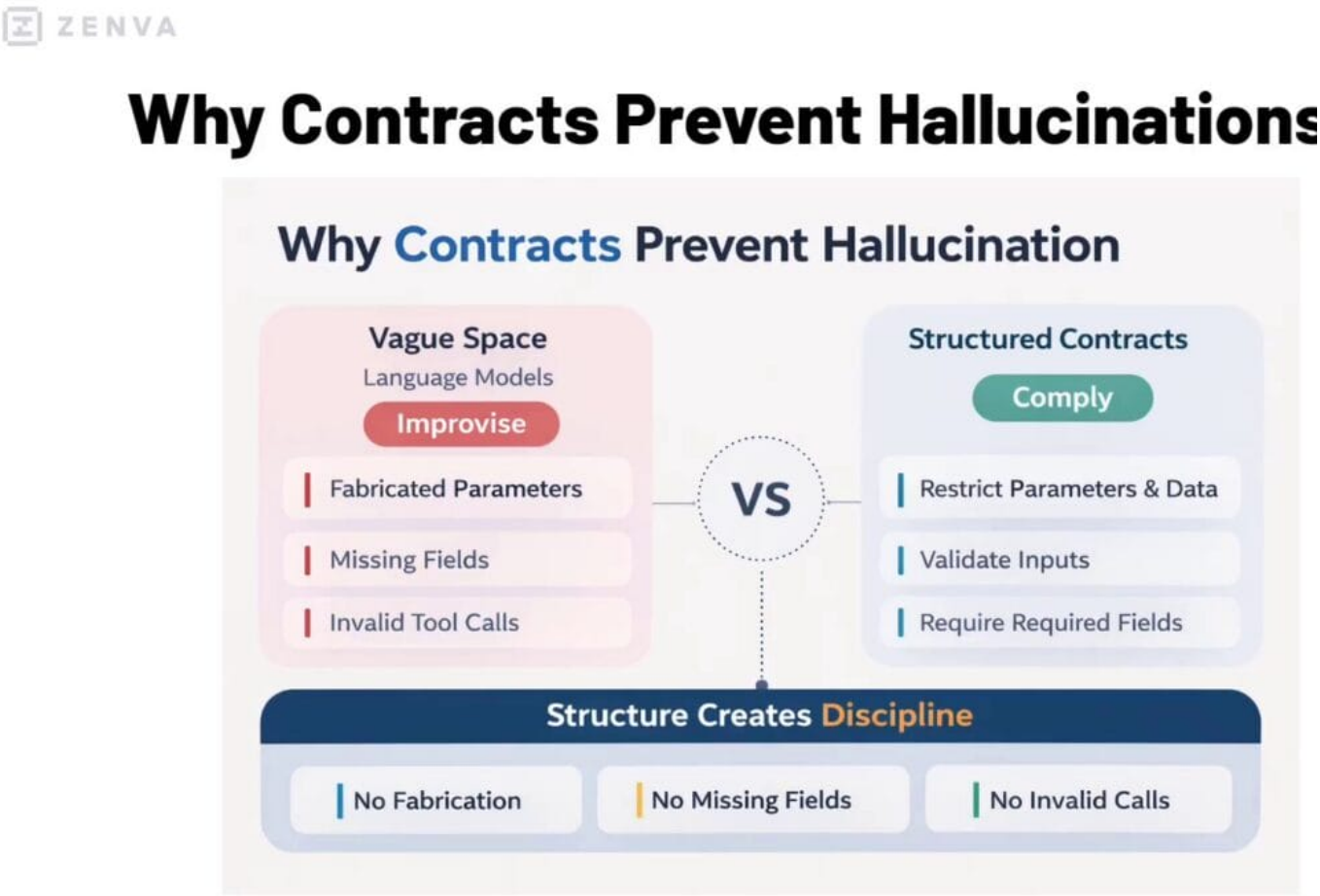
Why contracts prevent hallucinations

Language models are probabilistic by nature. If you give them vague space, they improvise. If you give them structure, they comply. Tool contracts give the model that structure and turn an open ended interaction into a disciplined one.

When you follow a tool contract, you constrain the model in several ways:

- **Restricted parameter names:** only the names defined in the contract are accepted.
- **Restricted data types:** if a parameter expects a number, the model cannot pass a string.
- **Allowed formats:** the contract can specify formats, for example a particular date layout.
- **Required fields:** the model must supply every mandatory input before the call is considered valid.

Claude must satisfy all of these conditions before a tool call is accepted. The result is a dramatic reduction in fabricated parameters, missing fields, and malformed tool calls. Structure creates discipline, and discipline is what makes the system predictable.

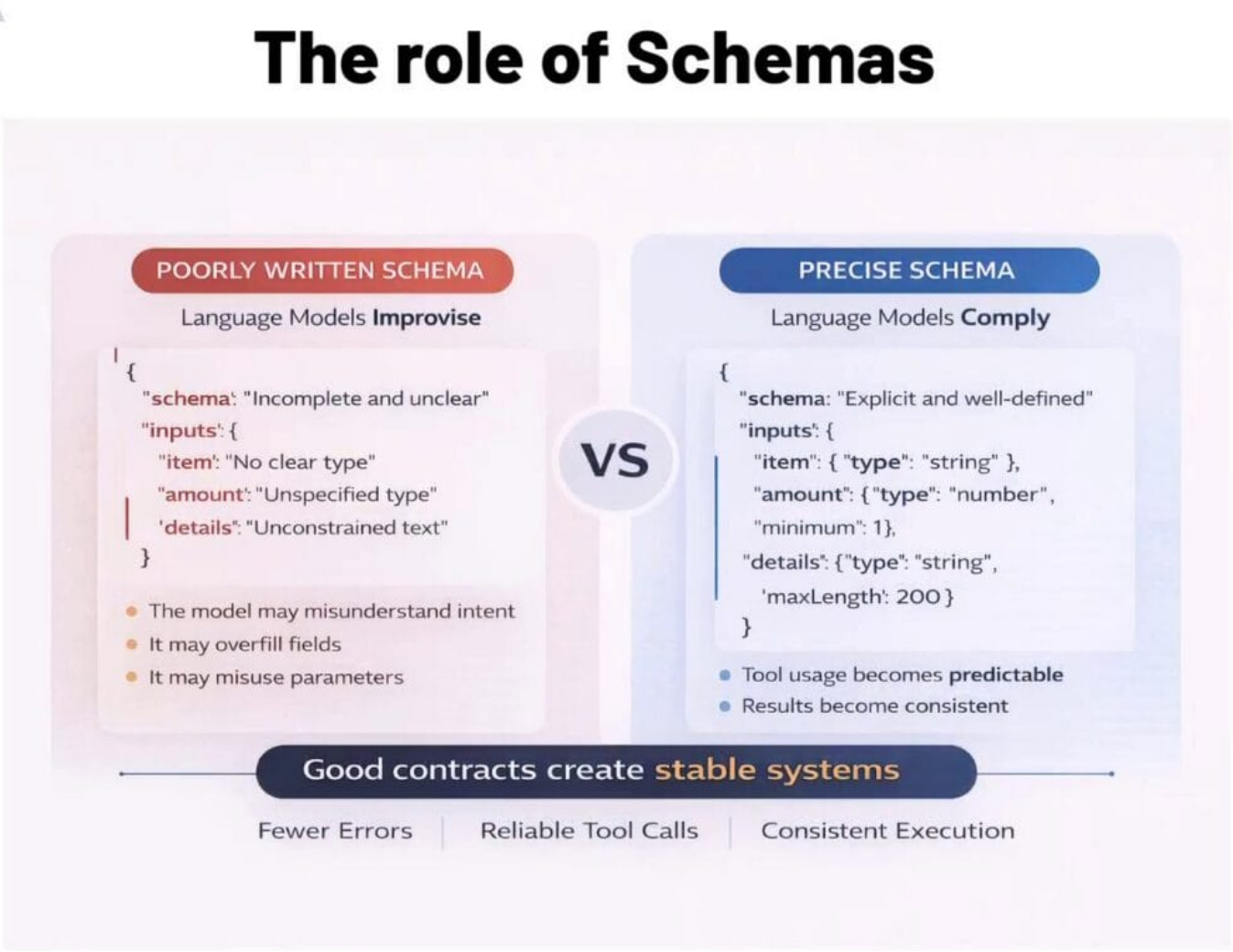


The role of a schema

Under the hood, tool contracts are typically expressed using something called a schema, often a JSON Schema. You do not need to become an expert in writing schemas, but you do need to understand their purpose. A schema is a machine readable description of the allowed inputs, the required fields, the data types, and any other constraints a tool expects.

Schemas are not written for humans, they are written for machines. They force clarity. When Claude decides to use a tool, it reads the schema first and then constructs a request that complies with it. If the schema is poorly written, the model may misunderstand intent, overfill parameters, or misuse

the tool. If the schema is precise, tool usage becomes predictable and the results become consistent. In short, good contracts create stable systems.



How this connects to agentic systems

Tool contracts become even more important once an AI agent has access to many tools at once. Later in the course we will introduce the GitHub MCP, the Playwright MCP, and a File System MCP. With multiple tools available, Claude has to make three decisions every time it acts:

- Which tool to use
- When to use it
- What arguments to provide

Each of those decisions is influenced by the tool's description, the clarity of its parameters, the guardrails around its use, and the constraints defined in files such as AGENTS.md. Tool contracts are the foundation for all of this. Without them, a multi tool agent would be guessing, but with them, it can reason reliably about what to do next.

In the next lesson, we will set up our environment and connect our first tools through MCP, putting these ideas into practice.

This lesson marks the point where Claude Code stops being limited to what already exists inside your project and starts connecting to external tools in a structured way. Through the Model Context Protocol (MCP), Claude can read from the file system, drive a browser, or later interact with services like GitHub. Once an MCP server is in place, Claude is no longer just generating text: it discovers the tools the server exposes, decides when to use them, and calls them through a defined protocol.

In this session, we will make MCP real on Windows using Claude Code, beginning with the safest and simplest first example: a file system MCP server.

Why start with the file system?

The file system server is a good first MCP server for several reasons:

- It is fully local, so it is easier to reason about and debug.
- Its results are immediately visible: new folders, edited files, and directory listings are all things you can see on disk right away.
- It uses directory access controls, so it only operates on the paths you explicitly allow.

This makes it a safe starting point before moving on to more powerful integrations like the GitHub MCP server in upcoming lessons.

Prerequisites

Before proceeding, make sure you have the following:

1. **Claude Code** installed and available from the command line.
2. **Node.js**, since the file system MCP server is a Node.js package.

You can verify Node.js is installed by running:

```
node -v
```

If Node.js is installed, this command returns the version number. The exact version is not critical as long as it is a reasonably recent one.

The general MCP setup process

Most MCP servers follow the same repeatable pattern. Understanding this skeleton once will make future setups (including GitHub MCP) much easier:

1. Make sure the server’s runtime (such as Node.js) is available on your machine.
2. Register the server with Claude Code from the command line.
3. Reload or inspect Claude’s MCP connections to confirm the registration.
4. Verify that Claude can actually use the new tools.

Creating the project folder and starting Claude

For this setup we will use two terminal windows: one to run Claude, and a second one for configuration commands.

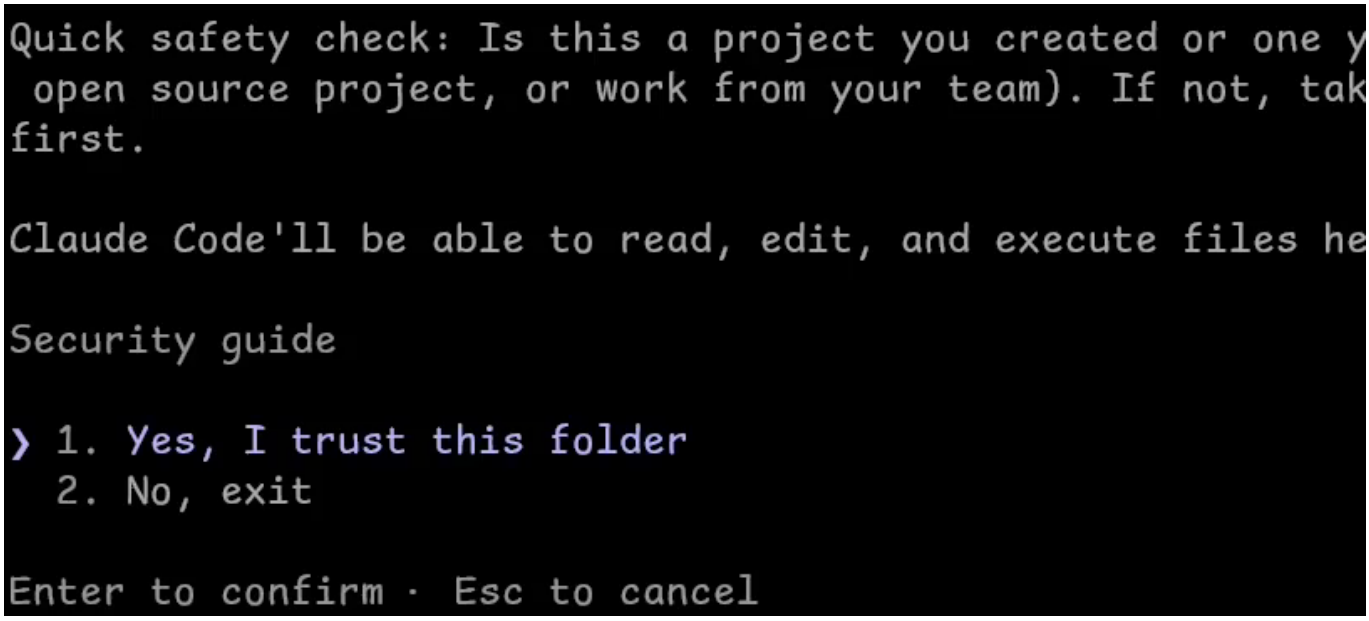
First, create a new project folder on the Desktop (or wherever you prefer) and change into it. In a terminal window, run:


```
mkdir mcp-examples
cd mcp-examples
```

Then start Claude in that directory:

```
claude
```

The first time Claude runs inside a new folder, it performs a quick safety check and asks whether you trust the files in the directory. Confirm that you do, and Claude Code becomes active in that workspace.



Opening a second terminal and checking Node.js

Leave the first terminal running Claude, and open a second terminal in the same mcp-examples directory. This second terminal is where we will issue MCP setup commands.

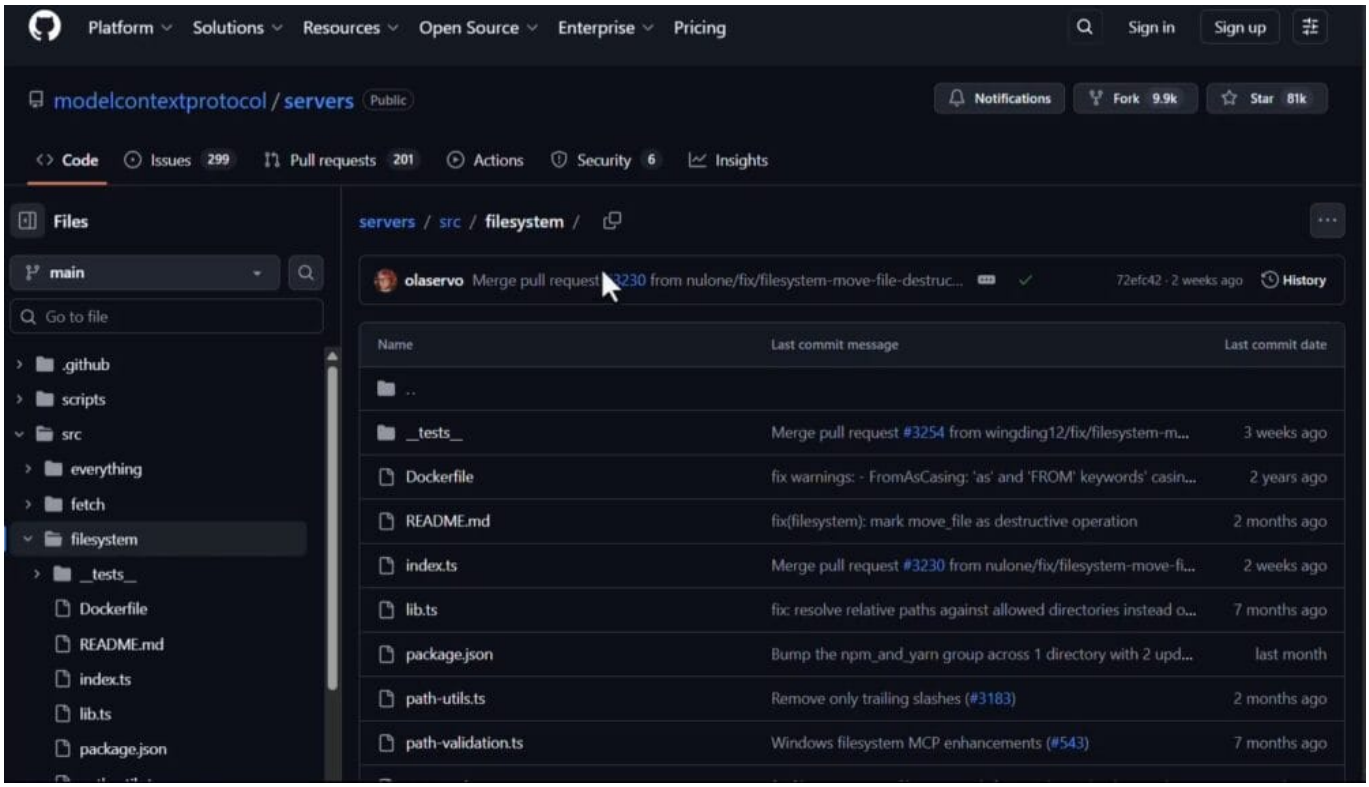
Because the file system MCP server is a Node.js package, confirm that Node.js is available:

```
node -v
```

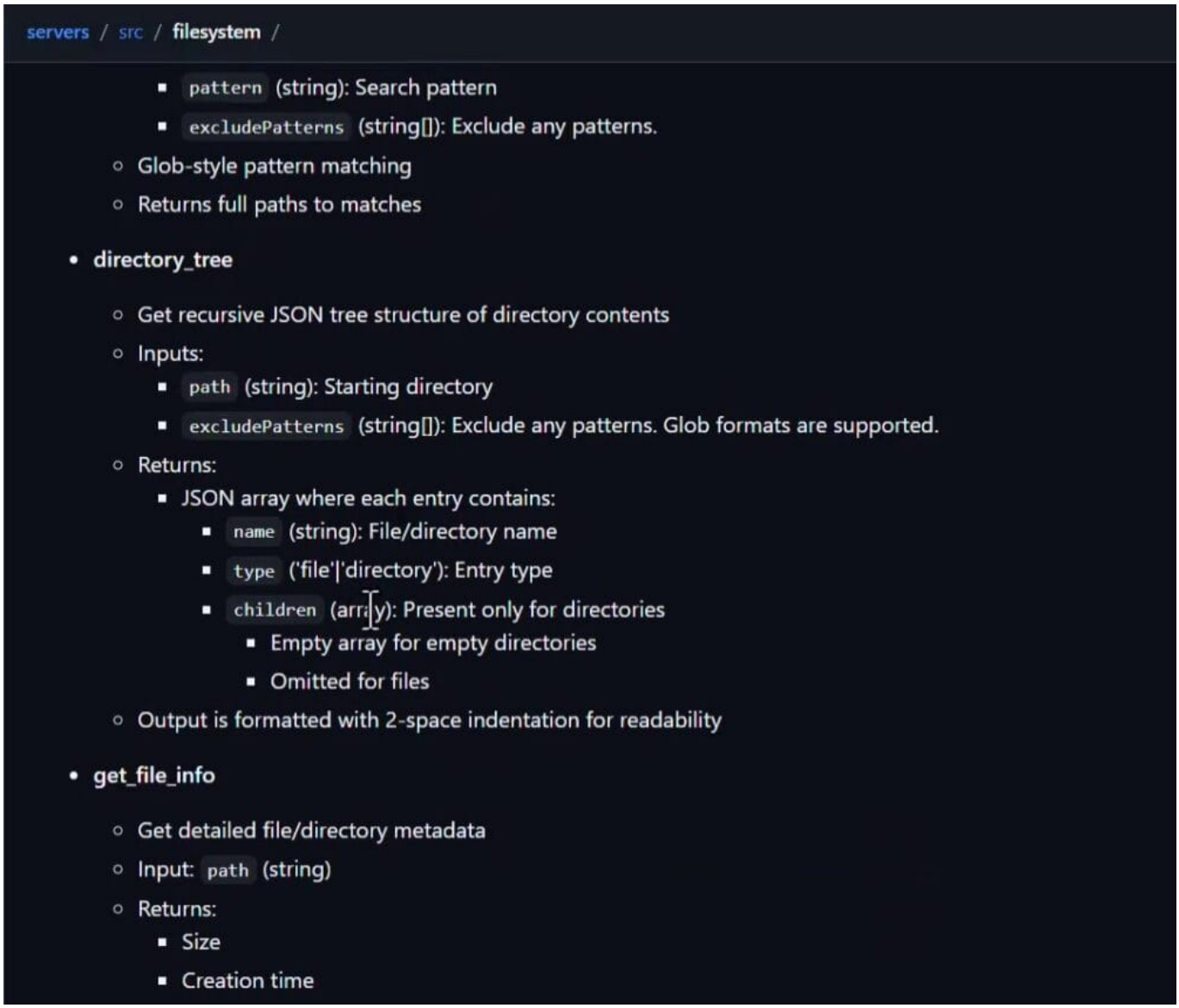
If Node.js is installed, the command prints the version number, and we are ready to move on.

Finding the official filesystem server

The package we will register lives in the official Model Context Protocol reference repository on GitHub, under modelcontextprotocol/servers, in the src/filesystem folder.



The README for that package documents how to set it up and lists the tool-calling functions the server exposes, including reading files, editing files, creating directories, searching for files, walking a directory tree, and getting file info. These are the capabilities that will become available to Claude once the server is registered.



Picking the right terminal on Windows

The command syntax we will use is the one shown in Claude Code's official documentation, which is written for Command Prompt (CMD) on Windows. PowerShell uses slightly different quoting and argument rules, so to follow the official examples exactly, switch to a Command Prompt window inside mcp-examples before running the registration command. On Linux distributions or Git Bash, the default command works without changes.

Registering the file system MCP server

With Command Prompt open in the project directory, register the server with this command:

```
claude mcp add --transport stdio filesystem -- cmd /c npx -y @modelcontextprotocol/server-filesystem .
```

For Mac/Linux, run the following instead:


```
claude mcp add --transport stdio filesystem -- npx -y @modelcontextprotocol/server-filesystem .
```

Each piece of this command has a specific role:

- claude mcp add: the primary command for registering a new MCP server with Claude Code.
- --transport stdio: tells Claude to communicate with the server over standard input and output, which is the typical model for local MCP processes.
- filesystem: the name we are assigning to this server. Claude will use this name when listing or inspecting MCP connections.
- --: separates the Claude Code arguments from the command that will actually launch the server.
- cmd /c: a Command Prompt specific prefix that runs the following command string and then exits. It is needed for Windows CMD only.
- npx -y @modelcontextprotocol/server-filesystem: uses npx to run the file system MCP server package without requiring a global install. The -y flag auto-accepts any prompts.
- .: grants the server access to the current directory (our mcp-examples folder). This is where the directory access control mentioned earlier comes from.

Confirming the registration

After pressing Enter, Claude Code confirms that the server has been added, reporting the server name, the launch command, and the local config file it updated. The configuration is written to a .claude.json file scoped to the current project folder, which means the connection is tied to this specific mcp-examples project.

```
C:\Users\mumin\Desktop\mcp_examples>claude mcp add --transport stdio filesystem -- cmd /c npx -y @modelcon
textprotocol/server-filesystem .
Added stdio MCP server filesystem with command: cmd /c npx -y @modelcontextprotocol/server-filesystem . to
 local config
File modified: C:\Users\mumin\.claude.json [project: C:\Users\mumin\Desktop\mcp_examples]
```

You can open .claude.json later to inspect or edit the MCP entries directly if you need to tweak them.

At this point the file system MCP server is registered with Claude Code, and the repeatable MCP setup pattern has been applied end to end on Windows. In the next lesson we will reload Claude’s MCP connections and confirm that Claude can actually use the new filesystem tools.

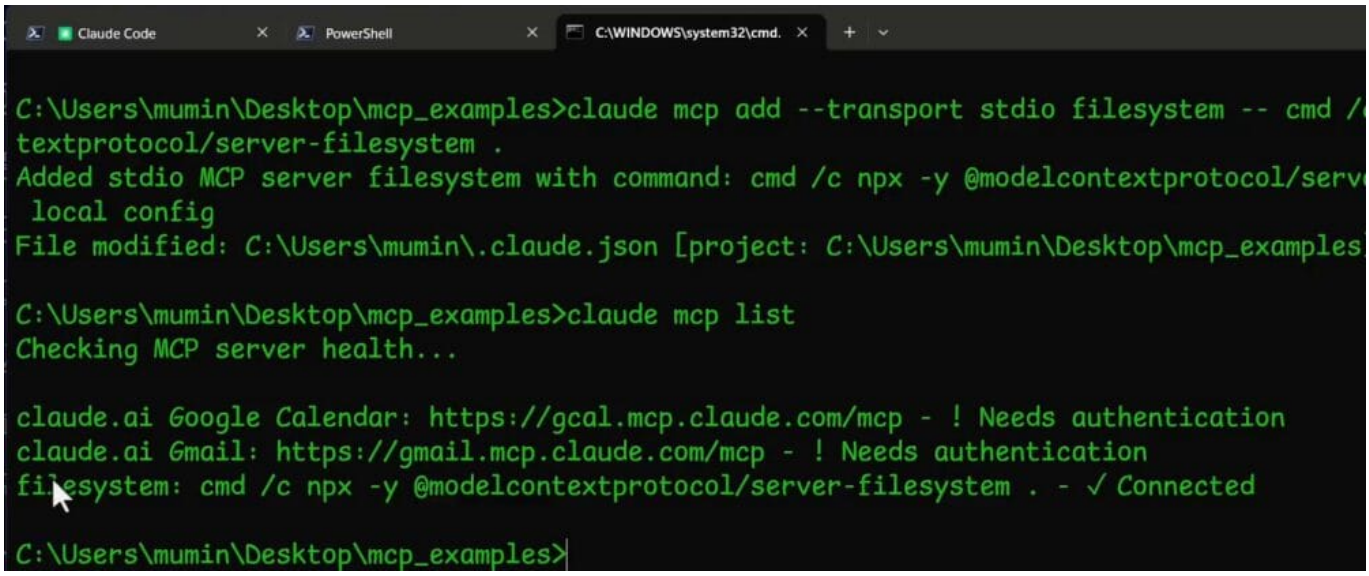
With the Model Context Protocol (MCP) filesystem server registered in the previous lesson, the next step is to verify that Claude Code can see it, inspect the tools it exposes, and use those tools to interact with real files on disk. This lesson walks through checking the server status from the terminal, managing the connection from within a Claude Code session, and running two simple tests against a small set of example files.

Verifying the server from the terminal

Before jumping back into Claude Code, it is worth confirming that the server is registered correctly. Open a terminal/PowerShell in the project folder, for example mcp_examples, and list the registered MCP servers:

```
claude mcp list
```

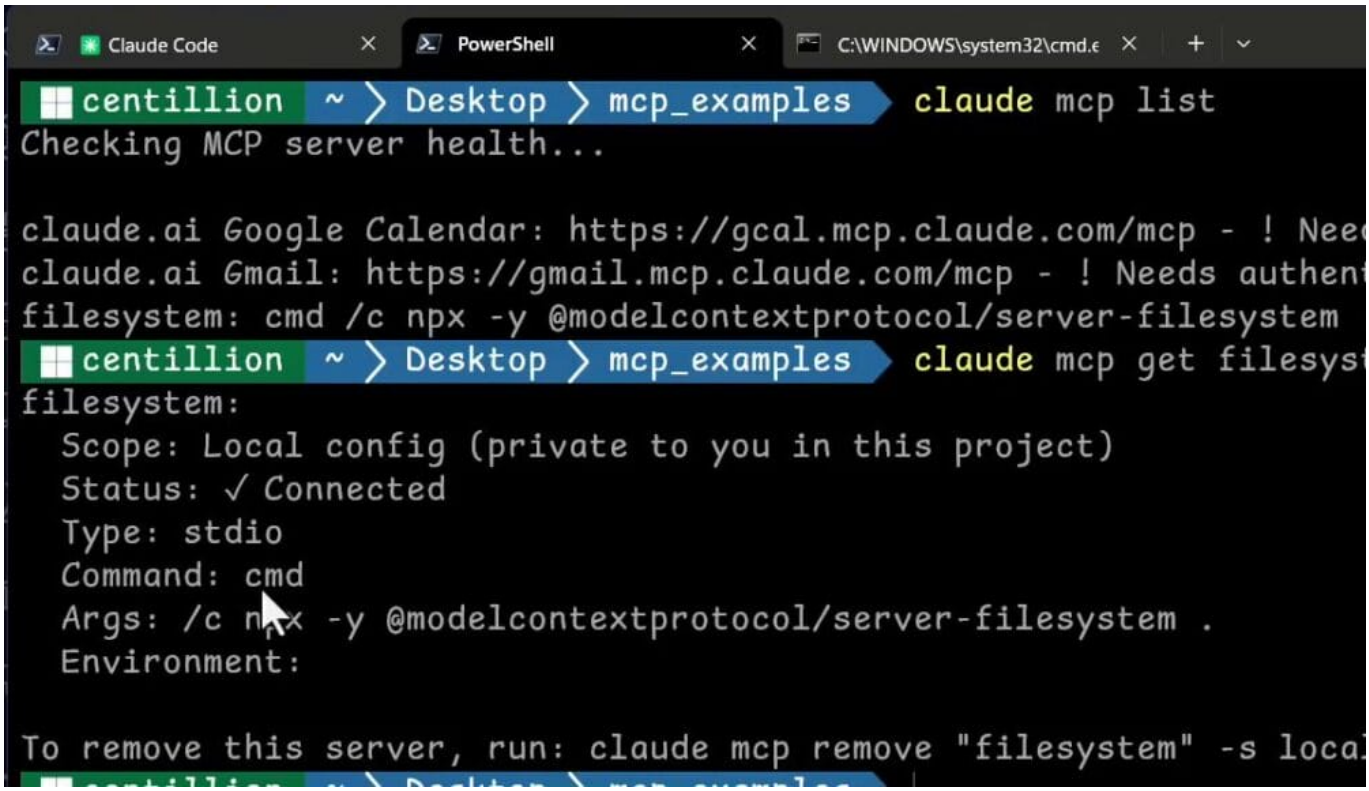
After a few seconds, Claude prints the list of servers it knows about. The filesystem entry should appear as connected. Other servers, such as the claude.ai Gmail and Google Calendar servers, may also be listed but marked as needing authentication, which is expected.



To get more detail about a specific server, use the get subcommand with the server name:

```
claude mcp get filesystem
```

The output describes how the server is wired up: its scope (local configuration, private to the current project), its connection status, the transport type (stdio, meaning standard input and output), and the command and arguments Claude uses to launch it. At the bottom, Claude also shows the command that would remove the server from the configuration, which is useful to know for cleanup.



Inspecting MCP connections inside Claude Code

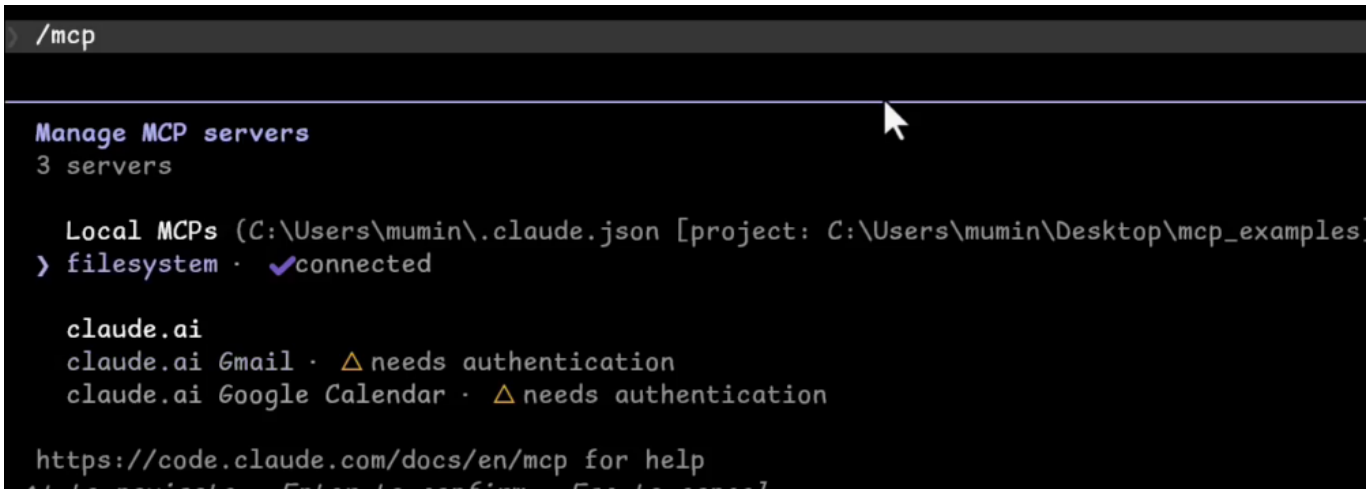
With the server confirmed from the outside, it is time to work with it from inside a Claude Code session. If a session is already running from the previous lesson, it is good practice to restart it so that any recent configuration changes are picked up cleanly. Close the current session and then launch a fresh one with:

```
claude
```

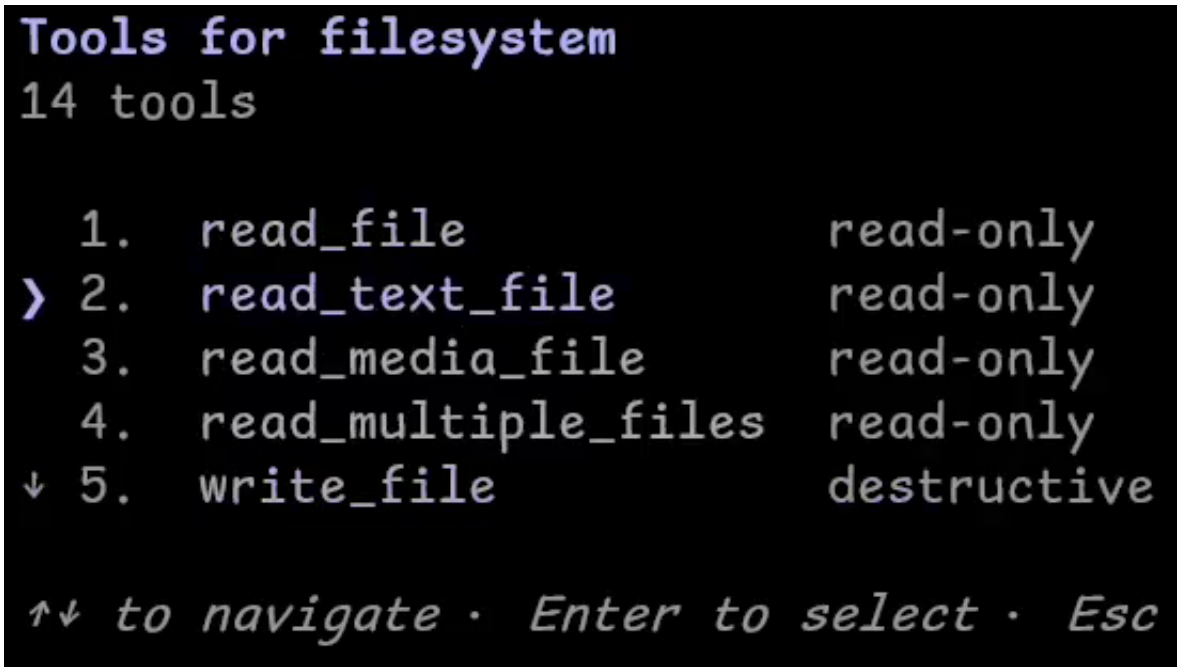
Inside Claude Code, the /mcp slash command is the entry point for managing and inspecting MCP connections. Type it and press Enter:

```
/mcp
```

Claude Code responds with a “Manage MCP servers” view that lists every server it is aware of. The filesystem server appears under the local MCPs section with a connected indicator, while authentication-gated servers are shown with a warning marker. Use the arrow keys to highlight filesystem and press Enter to drill in.



Selecting the filesystem server reveals the tools it provides. For the official filesystem server, this includes entries such as read_file, read_text_file, read_media_file, read_multiple_files, and write_file, among others. Read-only tools are labeled as such, while destructive tools (like write_file) are flagged separately. Seeing these tools listed is confirmation that Claude Code has correctly loaded the server's capabilities.



Creating test files

To exercise the server, a couple of small files are needed in the project directory. From the terminal inside the mcp_examples folder, create two empty files:

```
echo "" > file.txt
echo "" > README.md
```

Then populate them with the same short greeting so there is something meaningful for Claude to read back:


```
Set-Content file.txt "Hello, World!"
Set-Content README.md "Hello, World!"
```

For Mac/Linux, use:

```
echo "Hello, World!" > file.txt
echo "Hello, World!" > README.md
```

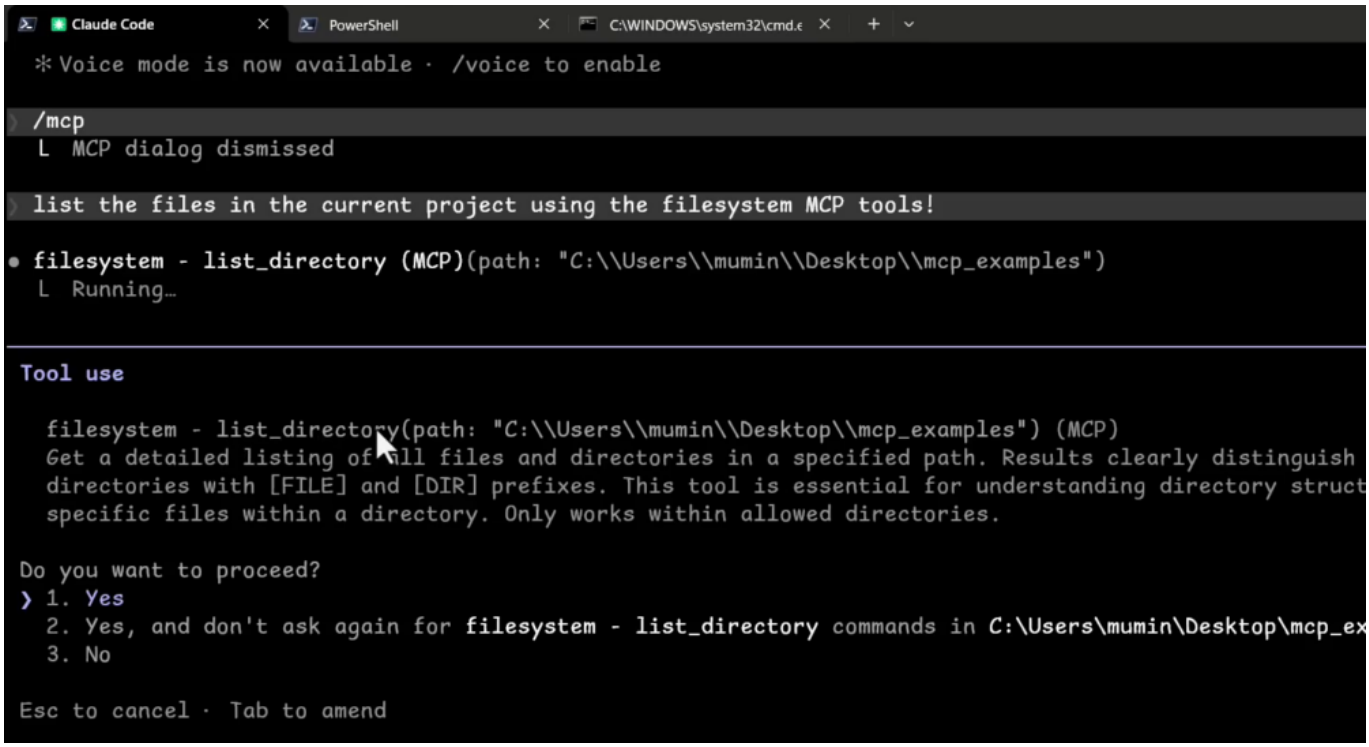
These files are deliberately trivial. The goal is not the content itself, but to give the filesystem server something to list and read during the tests below.

Listing files with the filesystem server

Back in the Claude Code session, and with the working directory still set to mcp_examples, ask Claude to list the project files using the MCP tools. A natural-language prompt is enough:

```
list the files in the current project using the filesystem MCP tools
```

Claude Code recognizes that this task fits the filesystem server's list_directory tool and prepares a tool call against the project path. Before running it, Claude Code shows a tool-use confirmation panel describing what the tool does (a detailed listing of files and directories, marked with [FILE] and [DIR] prefixes) and asks for permission to proceed. The prompt offers three choices: approve once, approve and remember the decision for this tool in this directory, or decline.



Choosing the "yes, and do not ask again" option skips the prompt for future list_directory calls in the same directory, which keeps the flow quick during experimentation. After approval, Claude returns a

table of entries in the folder. The listing includes the .claude configuration directory along with the two files created earlier: file.txt and README.md.

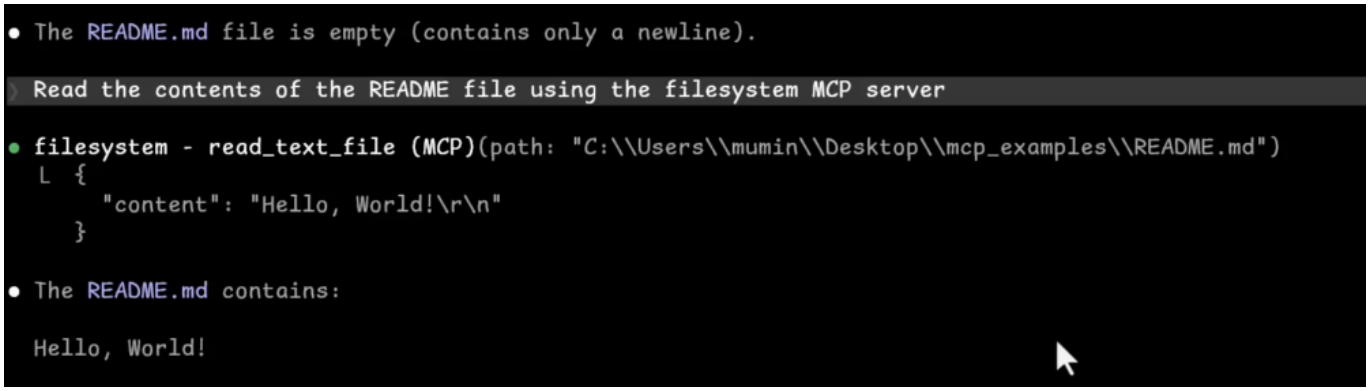
Reading file contents with the filesystem server

The second test confirms that Claude can not only see files but also read their contents. From the same session, enter:

```
read the contents of the README file using the filesystem MCP server
```

Claude Code selects an appropriate filesystem tool for the job. It may first consider read_file, which is marked as deprecated, and instead route the call through read_text_file, the current replacement for reading files as text. As with the listing step, Claude Code asks for permission before invoking the tool.

Once approved, the tool reads README.md and returns its contents. If the file had been left empty by mistake, Claude reports that cleanly (for example, “the README.md file is empty”), which is a useful cue to go back and populate it with Set-Content. After filling it in and re-running the prompt, the tool returns the expected payload and Claude reports that the file contains Hello, World!.



Wrapping up

If Claude Code can inspect the project directory and read a file through the MCP-connected filesystem tools, the setup is alive and working end to end. That small success matters more than it looks: it proves the registration, the transport, the tool permissions, and the project scoping are all aligned. With the mental model clear and the first MCP server registered and tested, the next session moves on to the GitHub MCP, using this same foundation to work with issues, pull requests, and repositories.

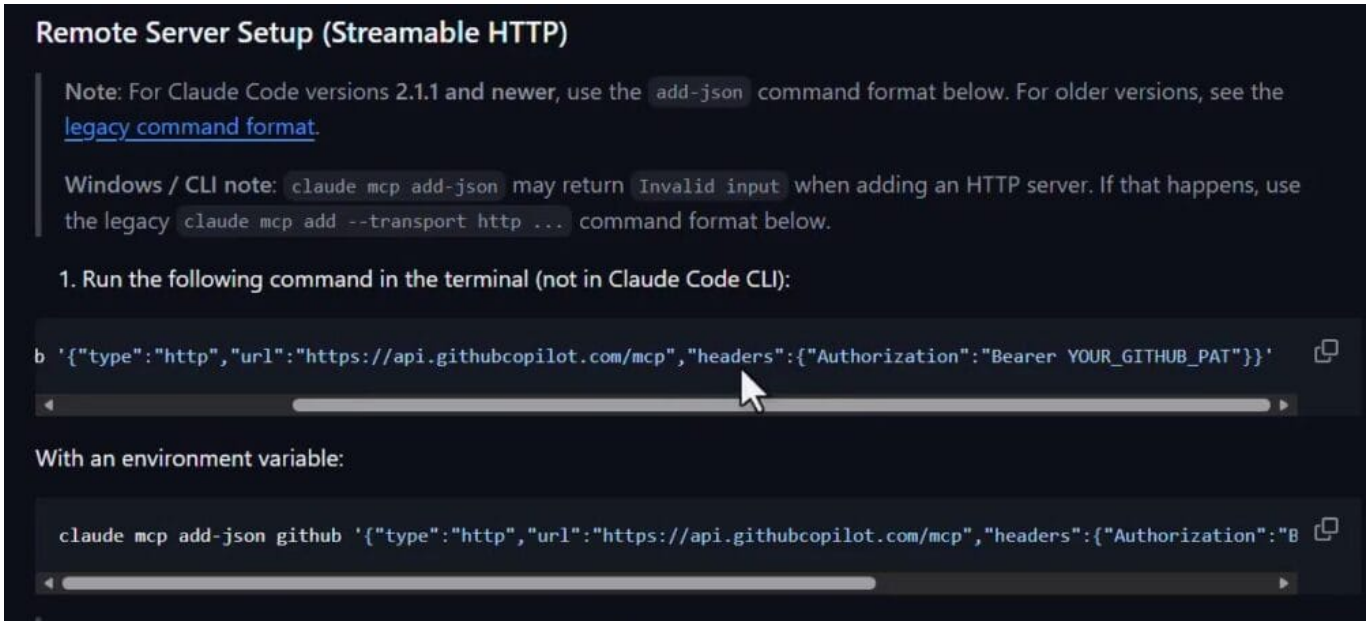
In the previous lesson, the File System MCP was used to confirm that Claude Code can discover and invoke external tools correctly. This lesson takes the next step and connects Claude Code to a service that is much closer to real development work: GitHub. Once the connection is in place, Claude will be able to inspect issues, create branches, implement features in a local repository, commit the work, and open pull requests, all through the same MCP mechanism already covered in earlier lessons.

Locating the GitHub MCP server reference

Before installing anything, it is worth knowing where the authoritative documentation lives. Searching for github mcp server on GitHub leads to the official github/github-mcp-server repository, which contains installation instructions for many Claude applications. This page is the main reference for configuring the GitHub MCP server, and it may be updated over time as the project evolves.

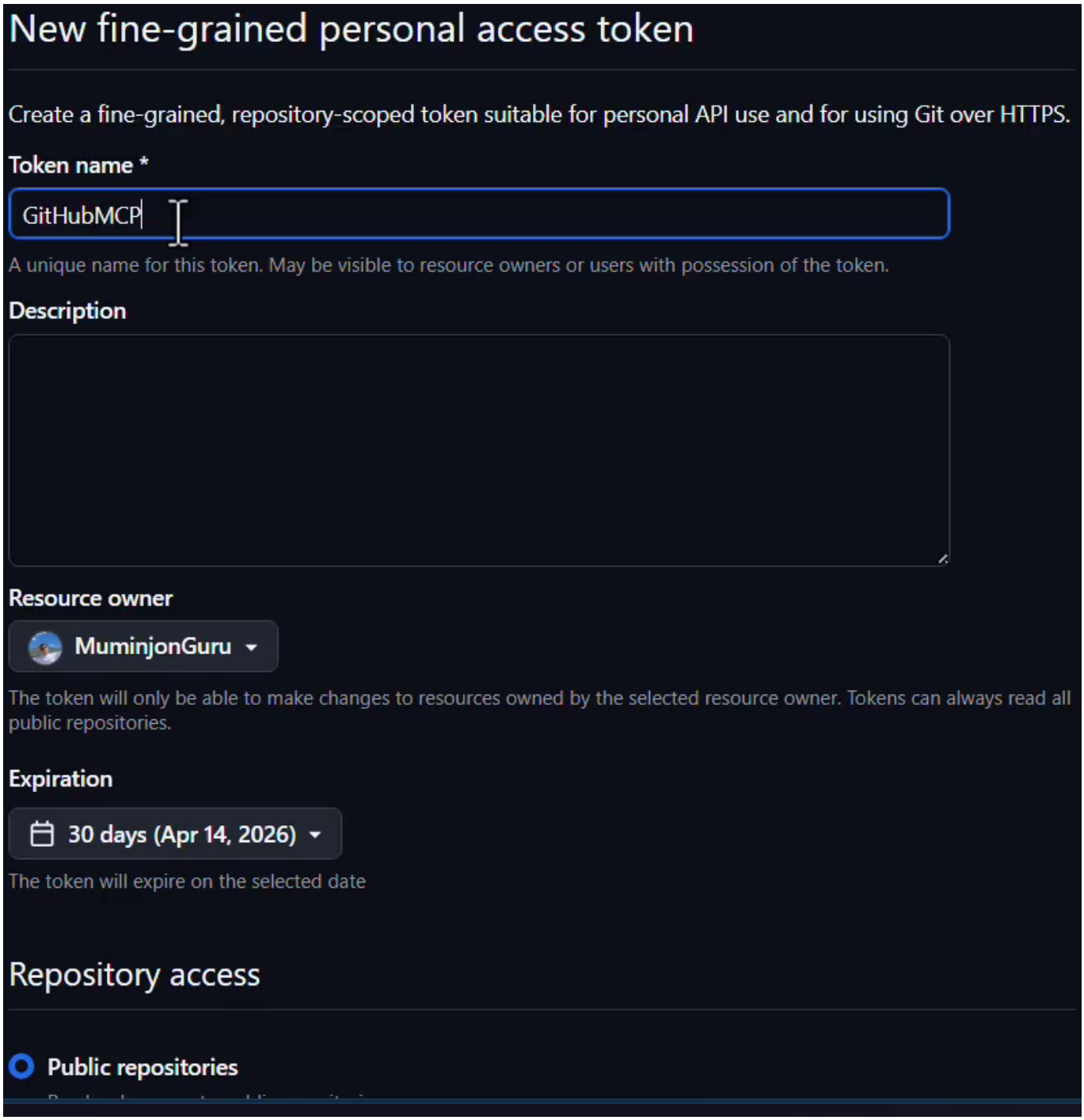
A naive attempt to install it the same way the File System MCP was installed, for example with a command like claude mcp add github, is not enough on its own. GitHub's API requires proper authentication, specifically an OAuth-style bearer token, so the configuration needs to include headers that carry a GitHub Personal Access Token (PAT).

The documentation's "Remote Server Setup (Streamable HTTP)" section shows the exact command to use. It specifies the MCP type, the GitHub Copilot MCP URL, and an Authorization header with the PAT.



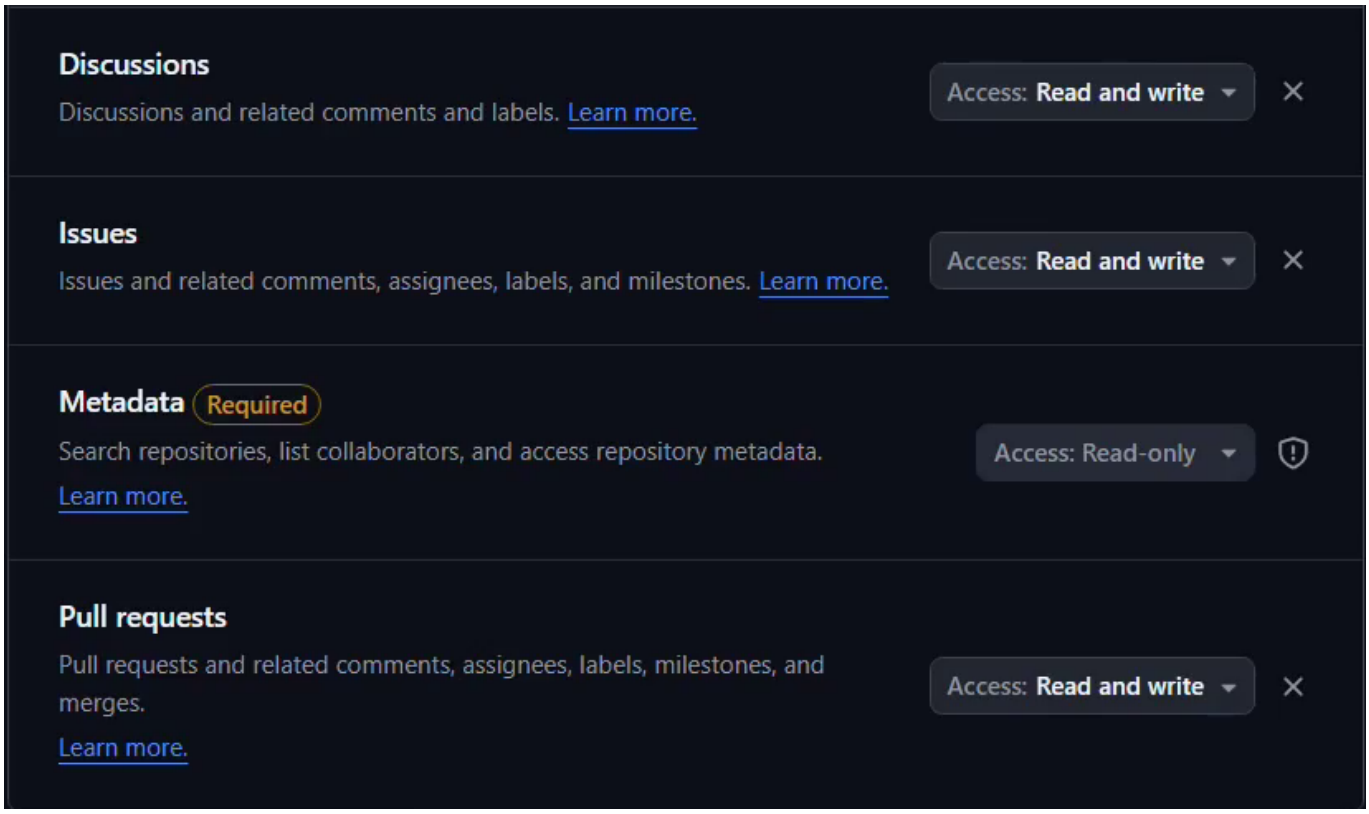
Generating a GitHub personal access token

Since the GitHub MCP server needs a PAT to authorize requests, the next step is to create one. From a signed-in GitHub account, open the developer settings for fine-grained personal access tokens. The token creation form asks for a name, expiration, and resource owner.

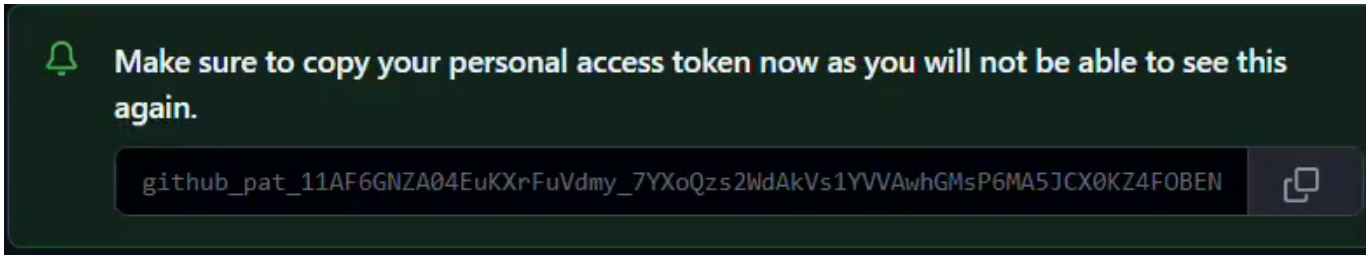


A descriptive name such as GitHubMCP makes it easy to identify later. The expiration is expressed in days and should match your own security preferences. For repository access, the token can be scoped to all repositories or to a single repository; this lesson uses a dedicated demo repository named github-mcp-demo-repo.

Further down the form, GitHub asks which permissions the token should carry. You can leave these empty for a minimal token, but since later lessons will use the MCP to act on issues, pull requests, and branches, it is practical to grant a few relevant permissions up front.



After selecting the permissions, clicking the “Generate token” button creates the token and shows it on screen.



GitHub displays the token value only once, as the green banner on the page warns. Copy the token immediately and keep it somewhere safe, because it will not be visible again after leaving this page.

Installing the GitHub MCP server in Claude Code

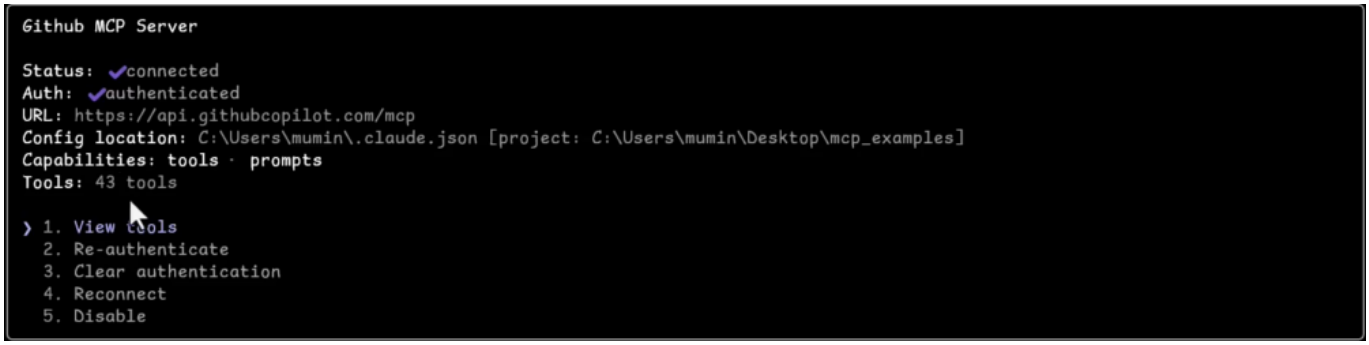
With a PAT in hand, the GitHub MCP server can be added to Claude’s local configuration. The documentation instructs that this command must be run in the operating system’s terminal, not inside the Claude Code CLI. The base command uses claude mcp add-json with a JSON configuration string, and the Authorization header carries the PAT as a bearer token.

```
claude mcp add-json github '{"type":"http","url":"https://api.githubcopilot.com/mcp",
"headers":{"Authorization":"Bearer YOUR_GITHUB_PAT"}}'
```

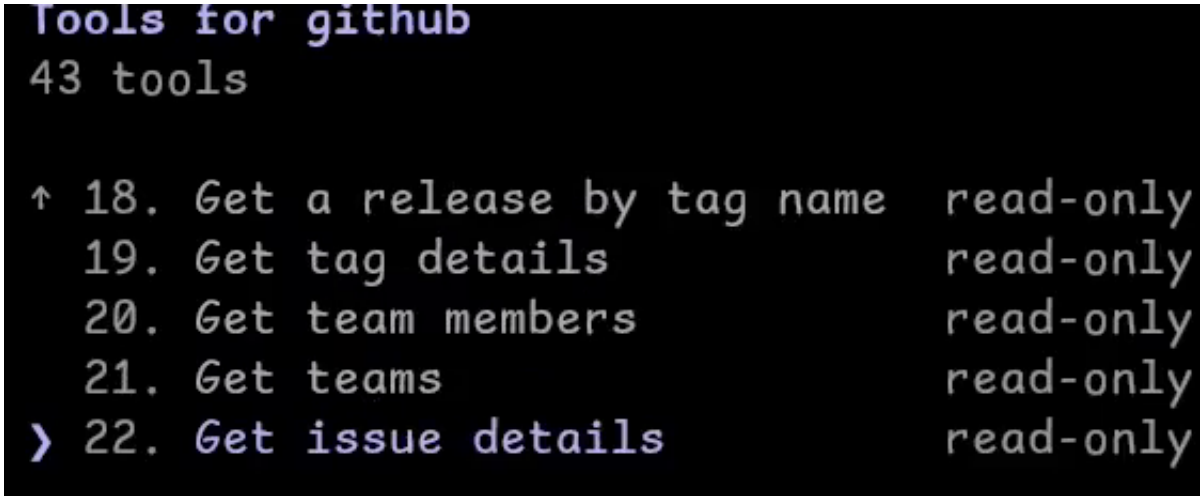
Replace YOUR_GITHUB_PAT with the actual token that was just generated, paste the complete command into a terminal, and press Enter. If the command succeeds, Claude Code prints a confirmation message indicating that the GitHub MCP server was added to the local configuration.

Verifying the connection

Once the command has finished, starting Claude Code and running /mcp lists every configured MCP server. At this point, both filesystem and github should appear, and the GitHub entry should be marked as connected and authenticated. Selecting the GitHub entry reveals its configuration, including the URL and the number of available tools.



From the connection details, choosing “View tools” opens the full list of GitHub operations that Claude Code can now invoke. The server exposes 43 tools at the time of this lesson, covering most of the GitHub operations needed during everyday development work.



The available tools include actions such as:

- add_comment_to_issue
- create_branch
- create_or_update_file
- delete_file_for_repository
- get_comment_details

With the GitHub MCP server connected, Claude Code can now participate in realistic development workflows. In larger setups where multiple agents collaborate, a single agent can be delegated to handle all GitHub-related actions for an entire project, from triaging issues to opening and merging pull requests. The remaining lessons in this section build on top of this connection to drive real

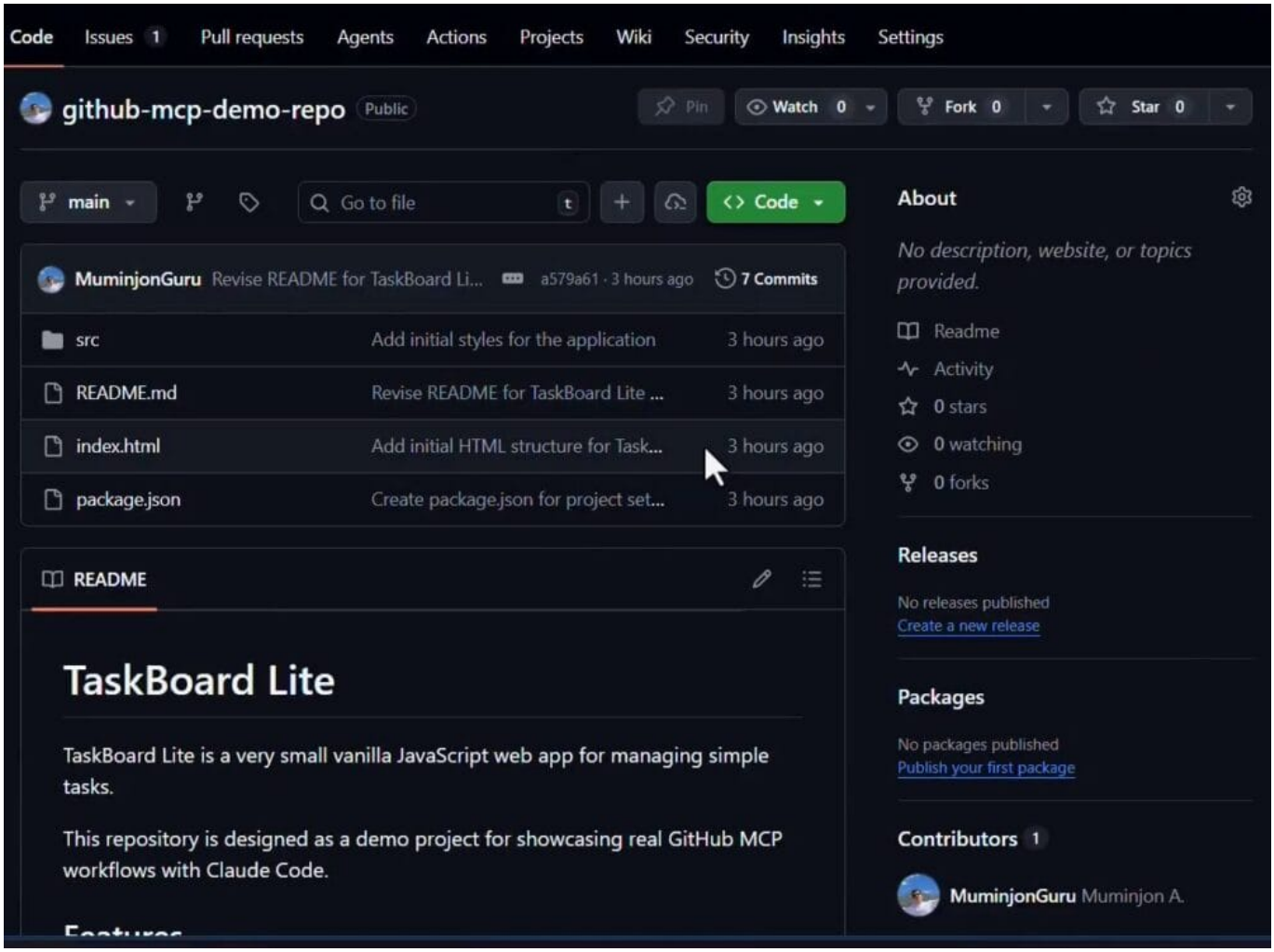
workflows end to end.

Welcome back. In the previous session, we configured our GitHub MCP server. Now it is time to apply those tools inside a real workflow. The goal of this lesson is to let an AI assistant inspect a task from a GitHub repository, understand its requirements, and prepare a local development environment before any code is changed.

Cloning the demo repository

The first step in any development workflow is to obtain a local copy of the project's source code. We will achieve this by cloning a public repository from a GitHub account. Cloning creates a complete copy of the repository, including its history, on your local machine.

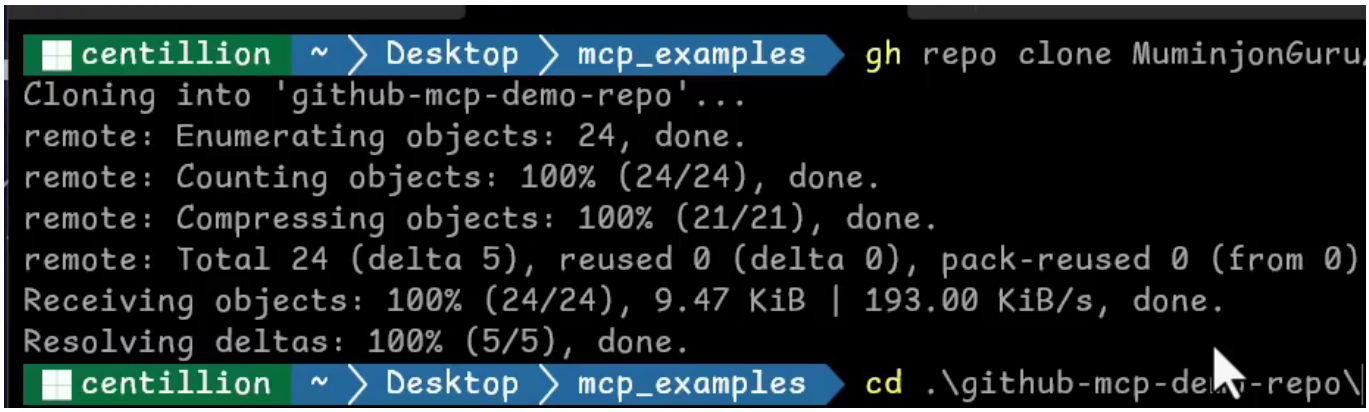
On the GitHub page for the repository, open the green Code dropdown to reveal both the HTTPS clone URL and the GitHub CLI command. In this lesson we use the GitHub CLI form.



Open a terminal and run the clone command, replacing the owner and repository name with the ones you are using:

```
gh repo clone owner/github-mcp-demo-repo
```

After the command completes you will have a new folder named github-mcp-demo-repo in your current directory, with all the project files available locally.



If you don't have *Github CLI* installed, you can run the following equivalent command (which requires no extra installs):

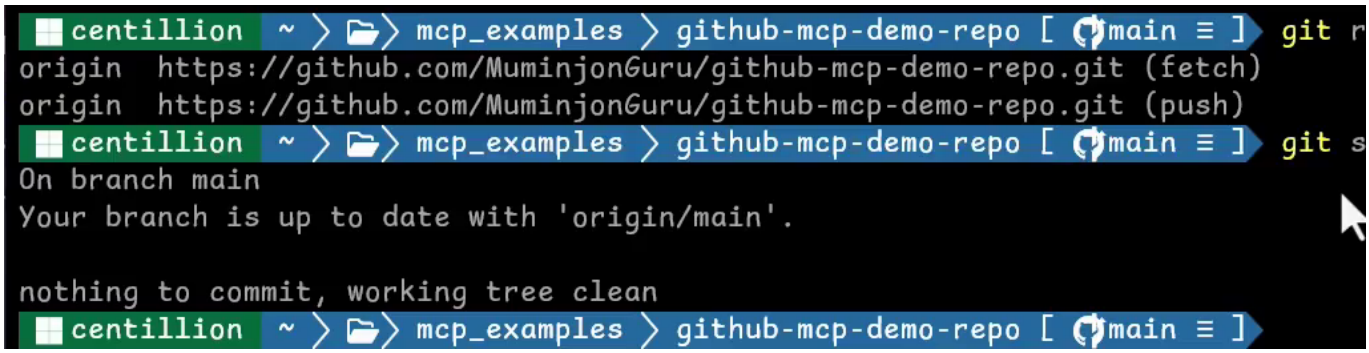
```
git clone https://github.com/owner/github-mcp-demo-repo.git
```

Preparing the local environment

Once the repository is cloned, navigate into its directory and verify its status. This ensures you are on the correct branch and that your local copy is synchronized with the remote on GitHub.

1. Change into the newly created project folder: cd github-mcp-demo-repo
2. Check which remote URLs your local repository is connected to: git remote -v
3. Confirm the working tree is clean and that the main branch is up to date with origin/main: git status

The output of git remote -v should list the origin URL for both fetch and push, and git status should report that the branch is up to date and the working tree is clean. That clean, synchronized state gives Claude Code a stable local starting point, while GitHub MCP continues to provide the remote context from GitHub.



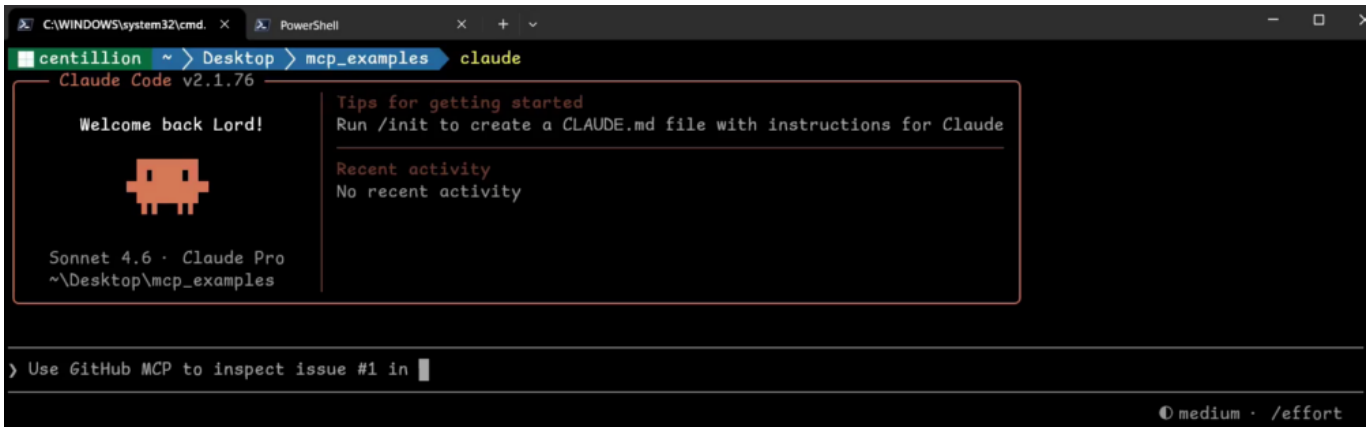
Starting a Claude Code session

With the local environment ready, start a Claude Code session from the same terminal by typing:

```
claude
```

Claude Code loads and displays the active model along with a prompt input area at the bottom,

ready to accept instructions for this project.

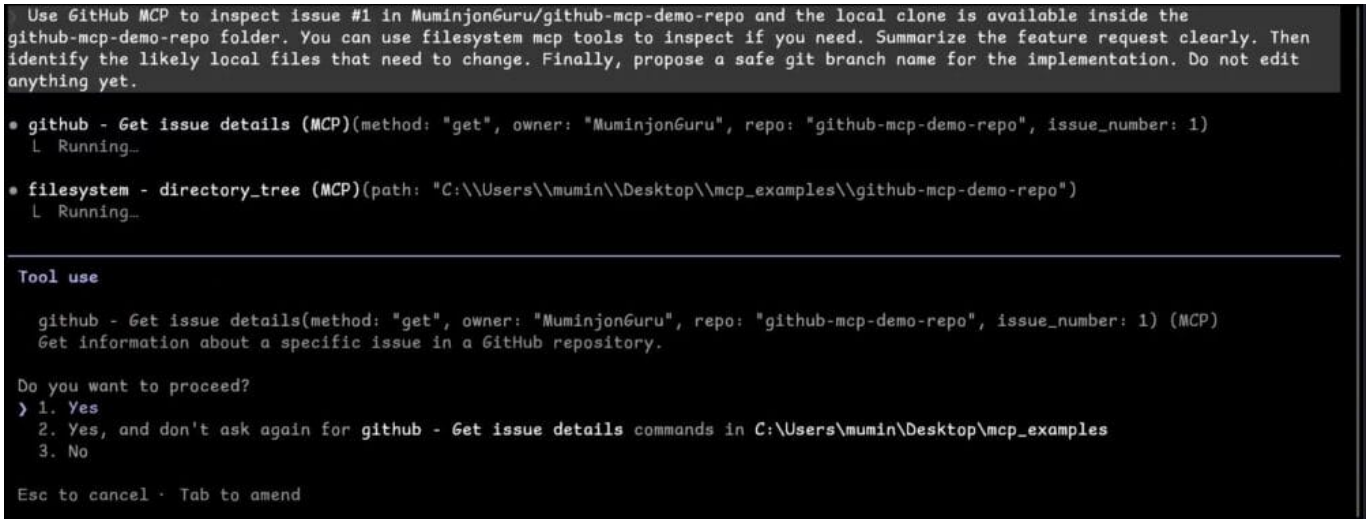


Inspecting a GitHub issue with the MCP tools

Now we instruct the assistant to use the GitHub MCP tools to inspect an existing issue on the repository. The prompt tells Claude exactly what to do: read the issue, summarize the feature, look at the local files, suggest a branch name, and not edit anything yet.

```
Use GitHub MCP to inspect issue #1 and the local clone is available inside the github
-mcp-demo-repo folder. You can use filesystem MCP tools to inspect if you need. Summa
rize the feature request clearly. Then identify the likely local files that need to c
hange. Finally, propose a safe git branch name for the implementation. Do not edit an
ything yet.
```

When Claude runs the prompt, it calls the GitHub MCP tool Get issue details with the owner, repository, and issue number, and it can also use the filesystem MCP tool to walk the local directory tree. Before running either, Claude Code asks for permission so you stay in control of what the tools can do.



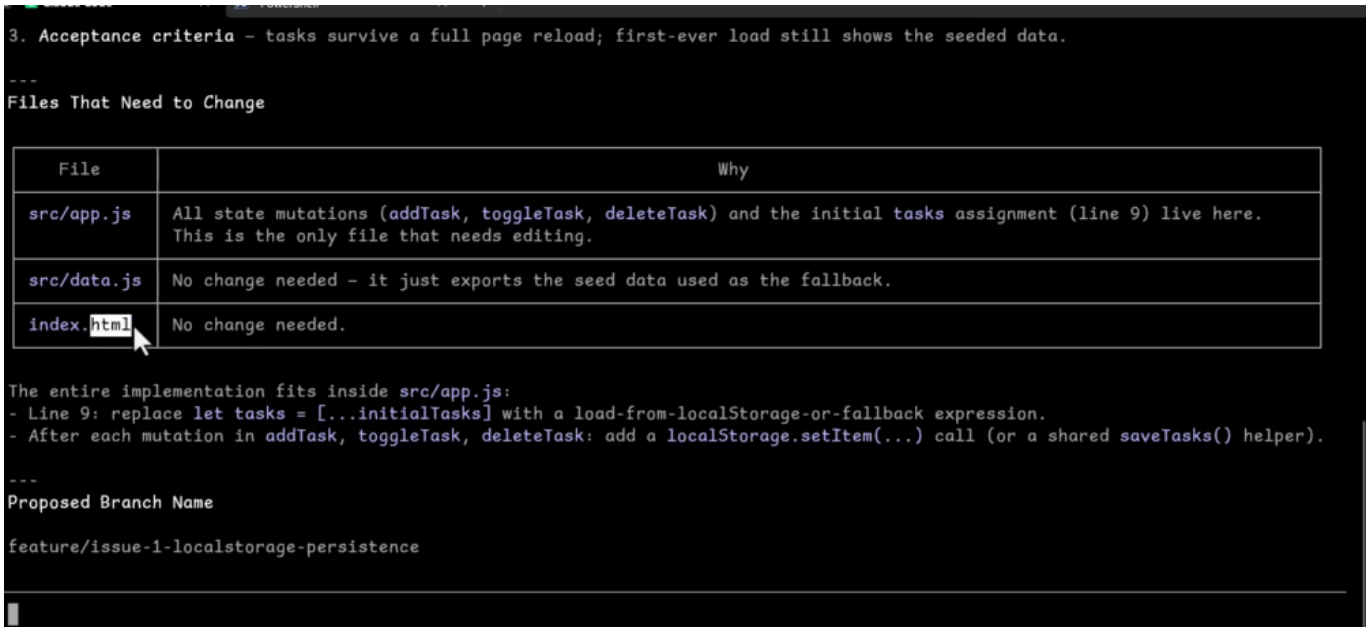
The source of the information is the issue that already exists in the repository on GitHub. In this case, Issue #1 is titled *Process tasks in local storage* and describes a feature request: every page reload currently wipes all user changes, so the task array should be persisted to local storage.

Reviewing Claude's analysis

After the tools return, Claude produces a structured summary that combines what it read from GitHub with what it found in the local project:

- **Feature summary:** the goal is to save tasks to the browser's local storage so they persist across page reloads, and to load them back on startup (with the seeded data still appearing on the very first load).
- **Files that need to change:** after scanning the local project, Claude concludes that the entire implementation fits inside src/app.js. Files like src/data.js and index.html do not need changes for this feature.
- **Proposed branch name:** Claude suggests a descriptive feature branch name, feature/issue-1-localstorage-persistence.

The key point is that Claude is not guessing from a pasted description. It is pulling live issue context directly from GitHub through the MCP server, then combining that with its analysis of the local project structure.



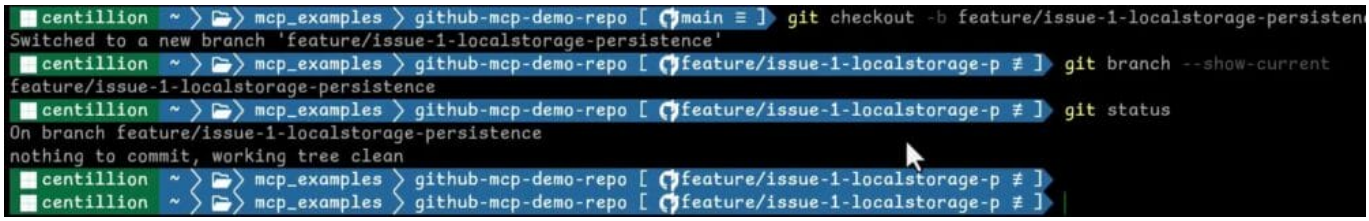
Creating the feature branch

Before any code is written, create a dedicated branch for this work. Isolating new features or fixes on their own branches keeps main stable and makes the changes easier to review and merge later.

Use the branch name that Claude proposed:

1. Create and switch to the new branch: git checkout -b feature/issue-1-localstorage-persistence
2. Verify that you are on the new branch: git branch --show-current
3. Optionally confirm the tree is still clean: git status

The terminal confirms the switch with *Switched to a new branch*, and the verification commands show the new branch name with nothing yet committed. The branch is created locally because the actual code changes still live in your local repository; GitHub MCP continues to provide the remote context, but the implementation work happens on your machine.



With the issue understood, the target files identified, and a feature branch in place, you have a clean foundation for the next step: using Claude Code, guided by the GitHub MCP context, to actually implement the change in src/app.js.

In this lesson, we will put the GitHub MCP server to work alongside Claude Code to drive a real development task from start to finish. We will prompt the AI assistant to implement an open issue, review the changes it produces, capture a concise summary, run the local Git workflow by hand, and finally create the pull request through the MCP tools. Along the way we will see how Claude Code can even diagnose and recover from a small error during the process.

Implementing the feature with Claude Code

We start inside the Claude Code session with our repository already open. The goal is to implement issue #1, which asks for task persistence in localStorage so that the user's task list survives a page reload and falls back to the seeded tasks when nothing is saved yet.

Rather than typing out the full specification, we write a short prompt that references the issue by number and lists the key requirements. In a real project the prompt could be much longer, adding constraints such as not refactoring unrelated code, reusing the current code style, and keeping changes tightly scoped, but for this demo a compact prompt is enough:

```
Implement issue #1 from [your-repo-name] in this local repository. Follow the require
ments: persist tasks in localStorage, load tasks from localStorage if present and fal
l back to seeded tasks if nothing is saved, and also follow the constraints.
```

When Claude Code asks for permission, we select the option to allow all edits during the session so it can modify files without interrupting. Because the GitHub MCP tools were used earlier to pull the issue into context, Claude already knows what problem it is solving and which files belong to the project, so it can make targeted edits.

After a short pause, Claude reports that the work is done. It has only touched src/app.js, which is exactly where the task state lives. The key changes include:

- A new STORAGE_KEY constant that identifies our entry in localStorage.
- A loadTasks function that reads the saved JSON from localStorage, or falls back to the seeded initialTasks when nothing is there.
- A saveTasks function that serializes the current tasks array back to localStorage.
- Updates to addTask, toggleTask, and deleteTask so each one calls saveTasks after mutating the list.

The new helpers and the replacement for the original in-memory initialization look like this. Your exact code may differ slightly because AI output is not perfectly deterministic.

```
const STORAGE_KEY = "tasks";

// Load tasks from localStorage or use initial data
function loadTasks() {
  const saved = localStorage.getItem(STORAGE_KEY);
  return saved ? JSON.parse(saved) : [...initialTasks];
}

// Save current tasks array to localStorage
function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

let tasks = loadTasks();
```


The three mutation functions are then updated so that every change is persisted immediately:

```
// Add a new task to the beginning of the tasks array
function addTask(title, category) {
  const newTask = {
    id: Date.now(),
    title,
    category,
    completed: false
  };

  tasks.unshift(newTask);
  saveTasks();
  renderTasks();
}

// Toggle the completed status of a task by ID
function toggleTask(taskId) {
  tasks = tasks.map((task) =>
    task.id === taskId
      ? { ...task, completed: !task.completed }
      : task
  );

  saveTasks();
  renderTasks();
}

// Remove a task by ID
function deleteTask(taskId) {
  tasks = tasks.filter((task) => task.id !== taskId);
  saveTasks();
  renderTasks();
}
```

With those edits in place, all three acceptance criteria are satisfied: tasks survive a page reload, toggling and deleting are persisted, and a fresh load with an empty storage still shows the three seeded tasks.

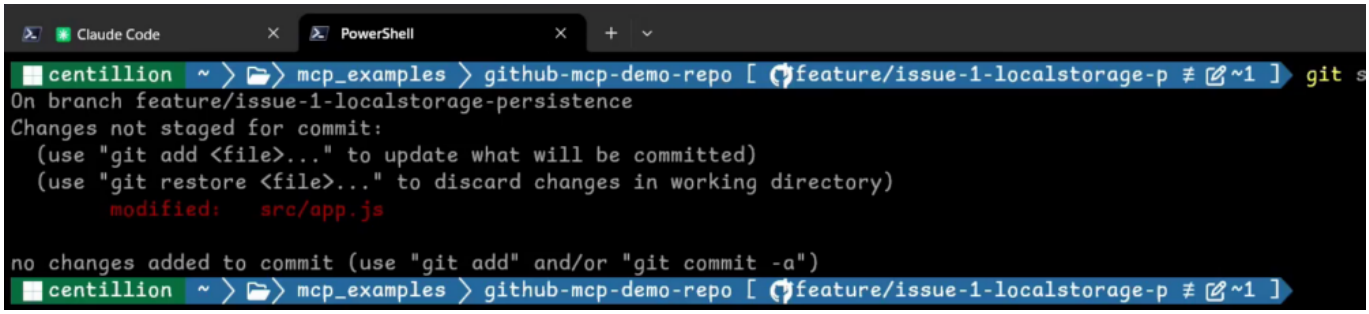
Reviewing the changes with Git

AI assistants are fast, but trusting a robot without looking at the diff is not a good habit. Before we commit anything, we switch to the integrated terminal and use plain Git commands to confirm exactly what changed.

First we check the working tree:

```
git status
```

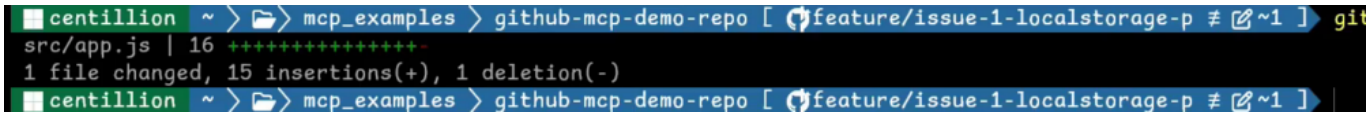

The output confirms that we are on the feature branch and that only src/app.js is modified, which matches what Claude told us.



Next, we ask for a short numerical summary of the diff:

```
git diff --stat
```

This reports one file changed with fifteen insertions and one deletion, which is consistent with adding the two helper functions and replacing a single line of initialization.



Finally, we inspect the actual hunks with:

```
git diff
```

We always stop here and read the real changes line by line. Claude can move fast, but we want to see the exact difference before anything lands in the repository. To close the diff pager we press Q.

Generating an implementation summary

Now that we know the code is correct, we ask Claude for a short human-readable summary we can reuse for the commit message and the pull request description. We switch back to the Claude Code session and send a prompt along these lines:

```
Summarize the implementation for issue #1 in [your-repo-name]. Give me a short human-readable summary: the files changed, the key behavior added, and any assumptions or limitations.
```

Claude replies with a clean paragraph listing the added helpers, the modified mutation functions, and any caveats (for example, that JSON.parse is not wrapped in error handling and that cross-tab synchronization is not implemented). This gives us ready-made language for the commit and the PR body, without having to write it from scratch.

Staging, committing, and pushing

With the review and summary in hand, we run the standard Git workflow by hand. Doing these steps

manually keeps the workflow visible and easy to follow. The clever part of the lesson is not hiding Git behind the AI.

Stage all modified files:

```
git add .
```

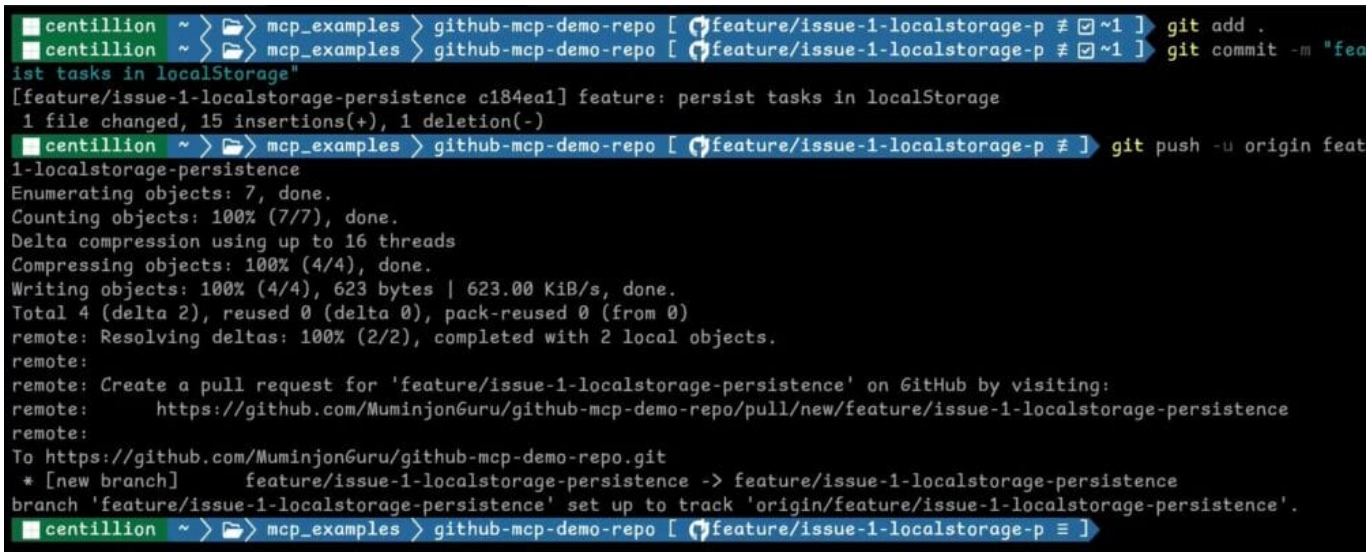
Commit with a concise message that matches the summary:

```
git commit -m "feature: persist tasks in localStorage"
```

Push the feature branch and set it to track the remote:

```
git push -u origin feature/issue-1-localstorage-persistence
```

GitHub responds with a helpful hint that includes a direct link for opening a pull request from the new branch.



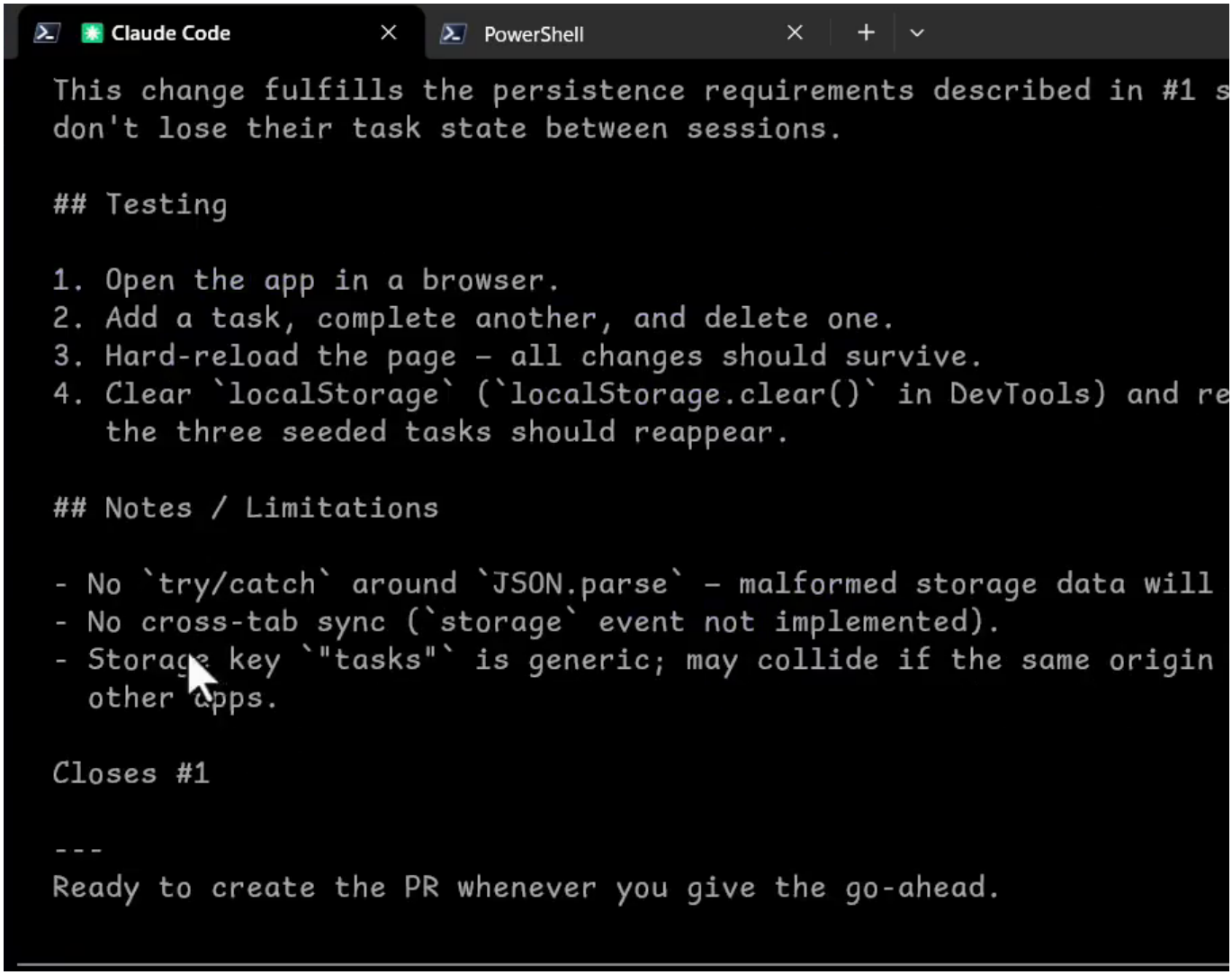
Drafting the pull request first

At this point we could open the pull request directly from the GitHub website, but the whole point of this lesson is to stay inside the editor and let the AI drive the remote actions through the GitHub MCP tools.

Instead of asking Claude to create the pull request immediately, we first ask it to draft the title and body so we have a chance to review the wording before any external action happens. The prompt looks like this:

```
Using GitHub MCP and the current implementation for issue #1 in [your-repo-name], dra
ft a pull request title and body. Include what changed, why it changed, a brief testi
ng note, and a closing reference to the issue. Do not create the pull request yet.
```


Claude produces a title such as feat: persist tasks in localStorage (closes #1) along with a body that explains the change, lists testing steps, notes limitations, and ends with Closes #1.



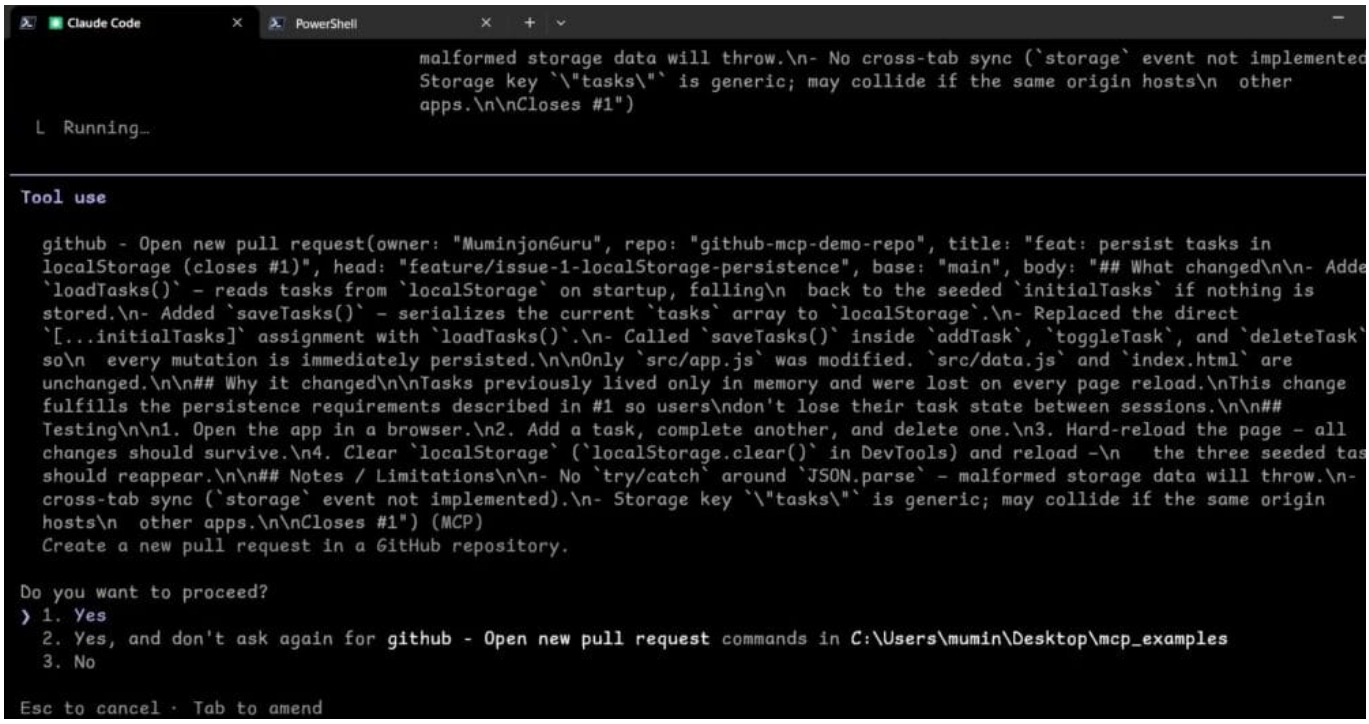
This pause point is valuable: it lets us adjust the text before anything is pushed to GitHub, and it keeps the human firmly in the loop on anything that is visible to collaborators.

Creating the pull request with GitHub MCP

Once we are happy with the draft, we ask Claude to actually open the pull request through the GitHub MCP tools, explicitly specifying the base and head branches:

```
Now use GitHub MCP to create the pull request in this repo. Use base branch main and
head branch feature/issue-1-localstorage-
persistence, and use the title and body we just drafted.
```

Because this is an external action, Claude Code surfaces a permission prompt showing the full payload it is about to send: the owner, repository, base branch, head branch, title, and body. This is exactly the kind of confirmation step you want for anything that leaves your machine.

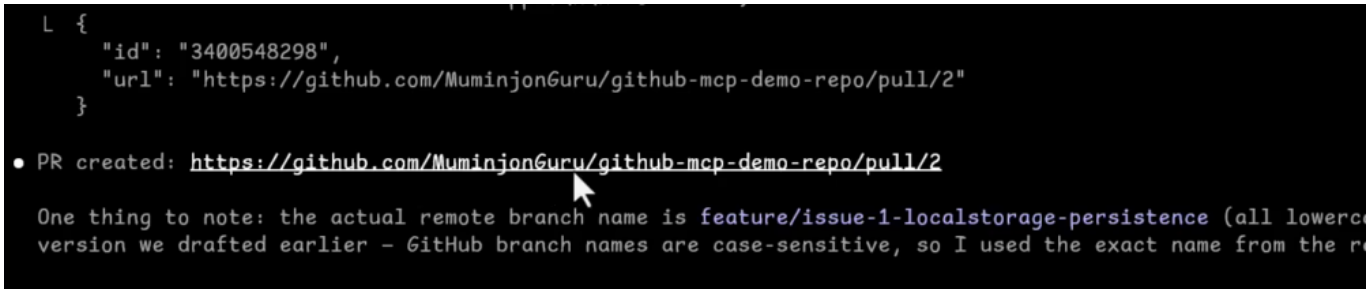


Handling a branch name error

The first attempt fails with a validation error: the head branch cannot be found on the remote. This is a useful real-world moment, because it shows how the AI behaves when something does not go as planned.

Claude investigates by listing the remote branches and notices that the branch does exist, but with a slightly different name. In the draft prompt we referred to feature/issue-1-localStorage-persistence with a capital S, while the actual branch pushed earlier is feature/issue-1-localstorage-persistence, all lowercase. GitHub branch names are case-sensitive, which is what caused the mismatch.

Claude corrects the branch name, retries the MCP call, and this time the pull request is created successfully. It reports back with the new PR URL, ready to open in the browser.



Why this workflow matters

This full loop shows the real value of combining GitHub MCP with Claude Code. The AI used GitHub MCP at the beginning to understand the issue, then implemented the feature locally while we reviewed every change with standard Git commands, and finally used GitHub MCP again to package that work back into GitHub as a pull request. We went from issue, to branch, to implementation, to pull request, which is the workflow most developers actually care about, now driven by a small set of well-defined tools rather than a big black box.

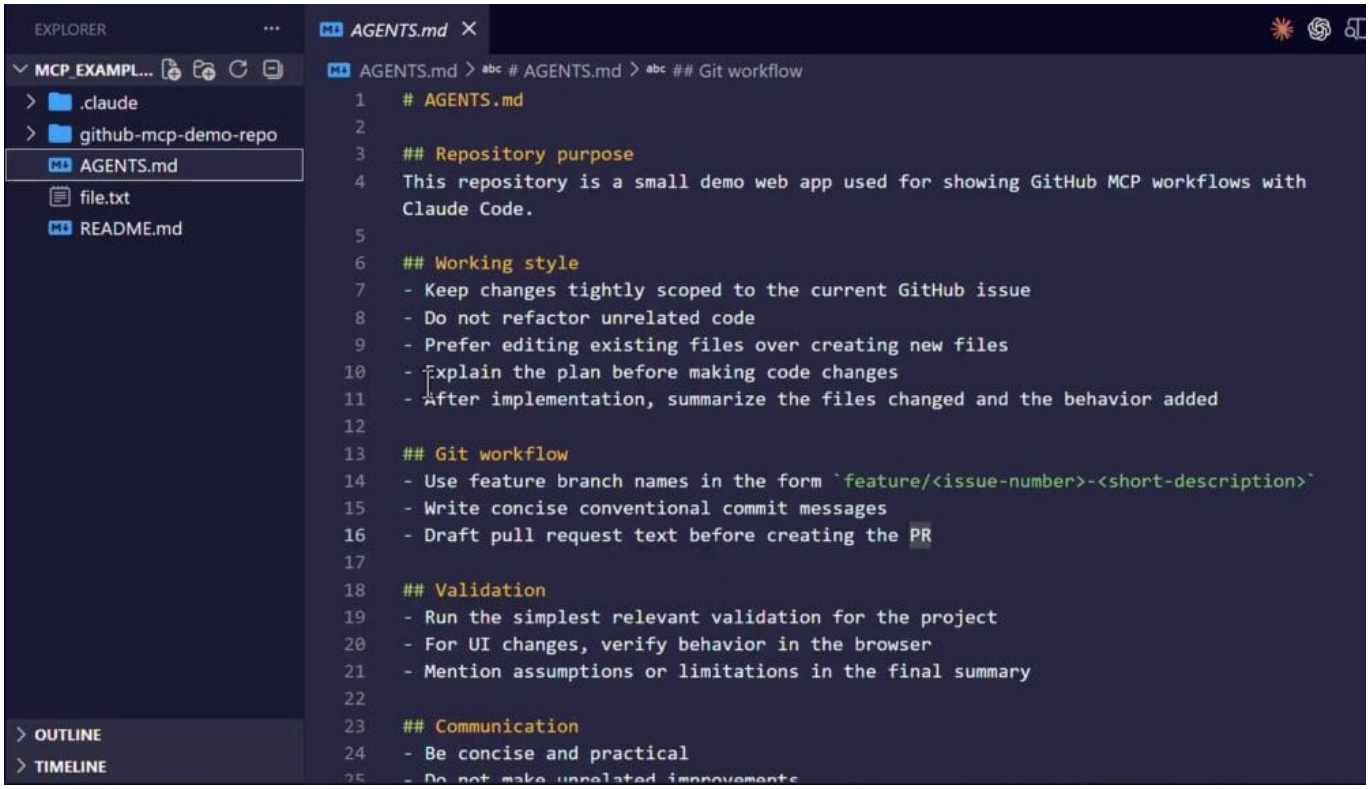
In an earlier lesson, we saw GitHub MCP used in a real workflow, where Claude Code read a GitHub issue, helped implement the requested feature in the local repository, and then turned that work into a pull request. That is the exciting part, but once an agent can actually do useful work, the next question becomes how to keep the workflow controlled. This lesson addresses that question by introducing a lightweight guidance layer for AI agents.

The tool we will use is a small file called **AGENTS.md**. Adding this file to the repository helps agents stay focused, follow repository-specific workflow expectations, and avoid going off on unrelated side quests in the code base.

What is AGENTS.md?

The **AGENTS.md** file is not a hidden or magical Claude Code configuration file. A better way to think about it is as a readme written for coding agents instead of for people. It is an open convention used across different AI products, giving AI coding tools a predictable place to find workflow guidance, coding expectations, validation steps, and general repository rules. The idea is simple: humans get a **README.md**, and agents get an **AGENTS.md**.

For Claude Code specifically, the native persistent instruction file is **claude.md**, not **AGENTS.md**. Claude Code loads **claude.md** files as part of its memory system at the start of a session, and those instructions guide its behavior. However, they are still treated as context rather than hard enforcement, which is why a clear, dedicated **AGENTS.md** can be a useful complement.



Creating a minimal AGENTS.md

The example project used in this lesson sits inside an *MCP_EXAMPLES* root folder, with the actual demo project in a subfolder. At the root of that workspace is a very basic **AGENTS.md**. You can find a copy of this file attached to the lesson project files, or you can take a moment to create your own minimal version with rules that matter for your project.

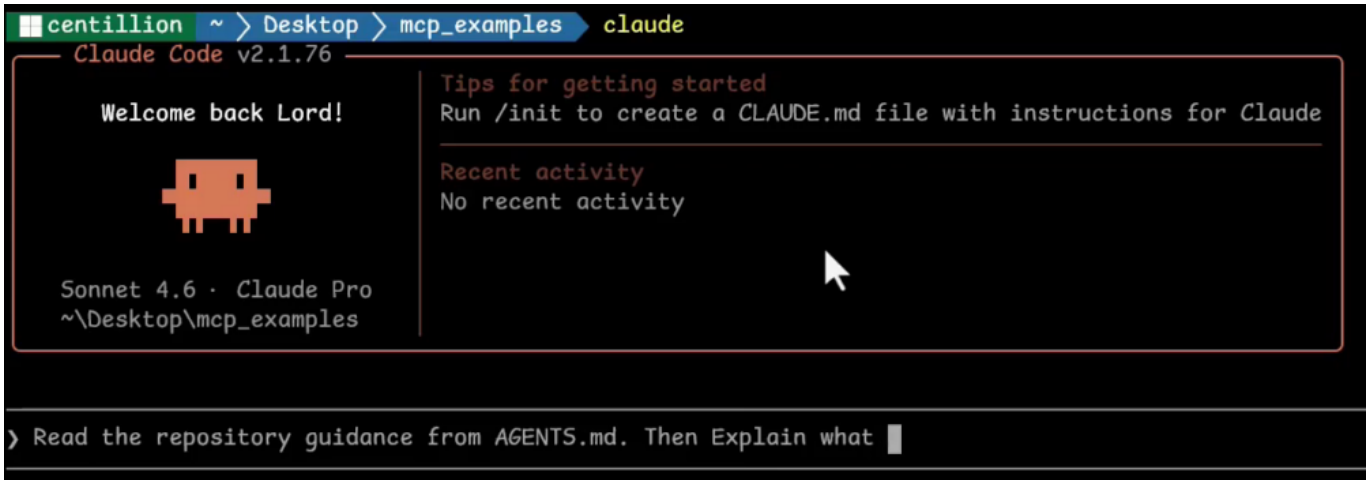
The rules used in this example are intentionally simple but effective:

- **Scope:** keep changes tightly scoped to the current GitHub issue.
- **Refactoring:** avoid unrelated refactors in the code base.
- **Planning:** explain the plan before editing any files.
- **Branch naming:** use a predictable Git branch naming pattern, such as feature/issue-number-short-description, which acts as a template for the agent.
- **Validation:** validate the work in the simplest and most sensible way possible.

Sections for working style, Git workflow, validation, and communication group these ideas together so the agent has a single consistent reference to consult.

Asking Claude Code to read the guidance

With the file in place, we can open a terminal, start Claude Code, and ask it to read and explain the repository guidance. A useful prompt looks like this: read the repository guidance from AGENTS.md, then explain what rules you should follow in this repo, how those rules affect a GitHub MCP workflow, and what kinds of actions you should avoid unless explicitly asked.



Claude Code will read the file and then summarize its new constraints. It identifies the actions it should not perform on its own unless explicitly asked, such as merging pull requests or creating pull requests without first drafting their text. It also confirms how the rules change the way it uses specific GitHub MCP tools. For example, the create branch tool must use the defined branch naming template, and the create pull request tool must draft and show the pull request text before the call is made.

add_issue_comment	Must be concise; no unsolicited architectural opinions
merge_pull_request	Requires explicit user instruction — not done autonomously

---

Actions to Avoid Unless Explicitly Asked

These are things I should **not** do on my own initiative:

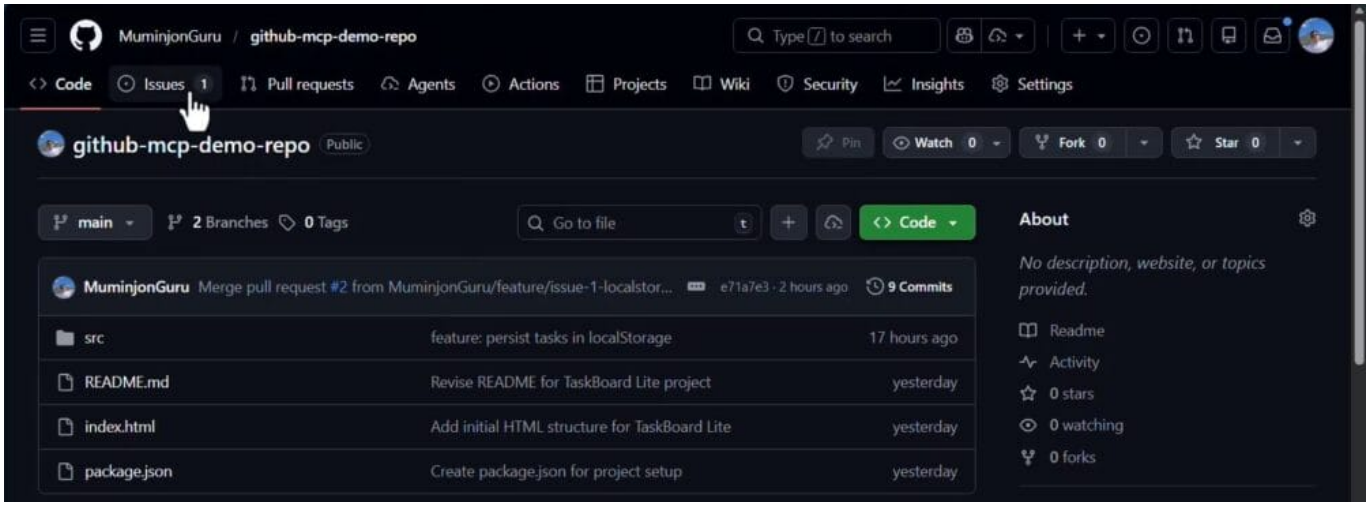
- **Merging PRs** (merge_pull_request) — irreversible shared-state change
- **Creating PRs without drafting** — the PR text must be reviewed first
- **Creating new files** when an existing file can be edited
- **Refactoring unrelated code** even if it's visibly suboptimal
- **Force-pushing or destructive git ops** — reset --hard, push --force
- **Making structural/architectural changes** without asking first
- **Opening issues or comments** that aren't directly related to the task at hand
- **Running broad validation** (full test suite rewrites, adding new CI steps) without being asked

In short: **every GitHub MCP action that affects shared state (branches, PRs, comments, merges) requi**
specific issue — and anything irreversible requires explicit user confirmation first.

The agent also confirms the broader working style, Git workflow, validation, and communication expectations, noting that larger structural changes should be requested rather than attempted autonomously. With a single small guidance file, we have given the agent a clear structure, a plan, a style, and a template for the changes it will make when using Claude Code together with GitHub MCP. The overall framing shifts: instead of charging straight into edits, the agent is nudged toward a specific working style.

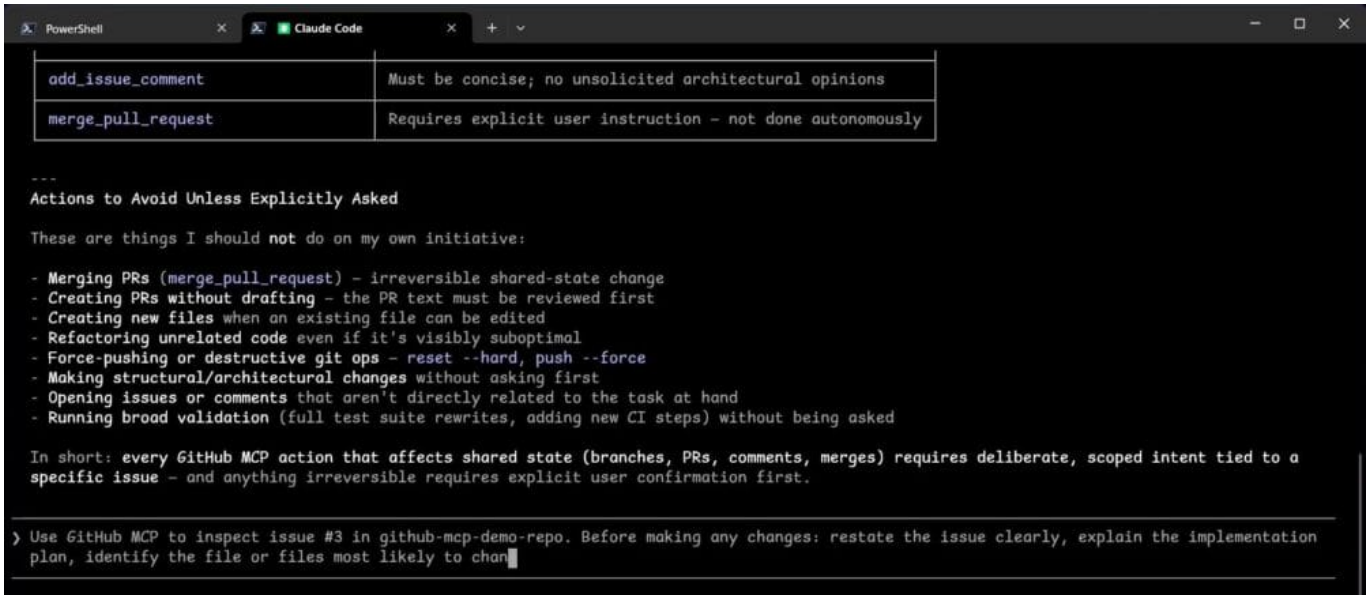
Applying the guidance to a real issue

To see this in action on a real task, we switch to the GitHub repository in a web browser. One previous issue has already been closed and its changes merged, and a new issue has been opened. This new issue is deliberately minimal: it has only a title, *add an empty state when search returns no tasks*, and no description. Its number is 3.



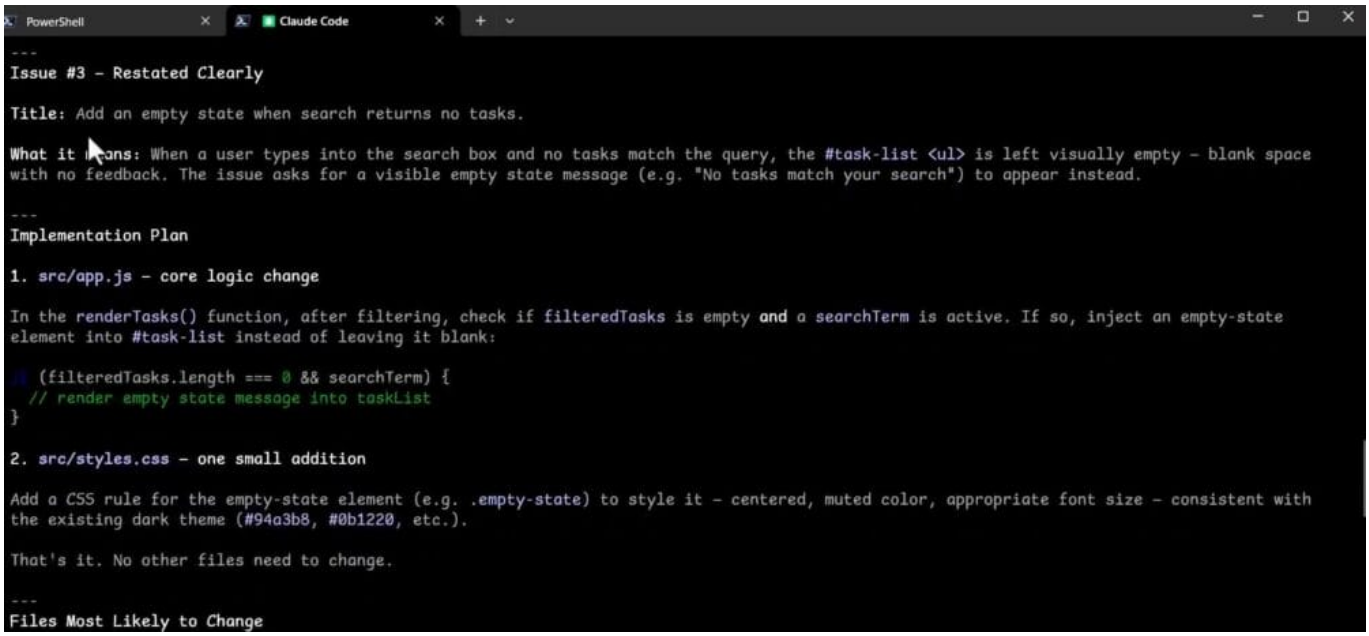
Back in the terminal, we ask Claude Code to inspect issue #3 using GitHub MCP and to produce a plan that follows the AGENTS.md rules. The prompt asks the agent to restate the issue clearly, explain the implementation plan, identify the files most likely to change, and describe how it will

follow the guidance from AGENTS.md during the workflow. Critically, the prompt also asks the agent not to edit anything yet.



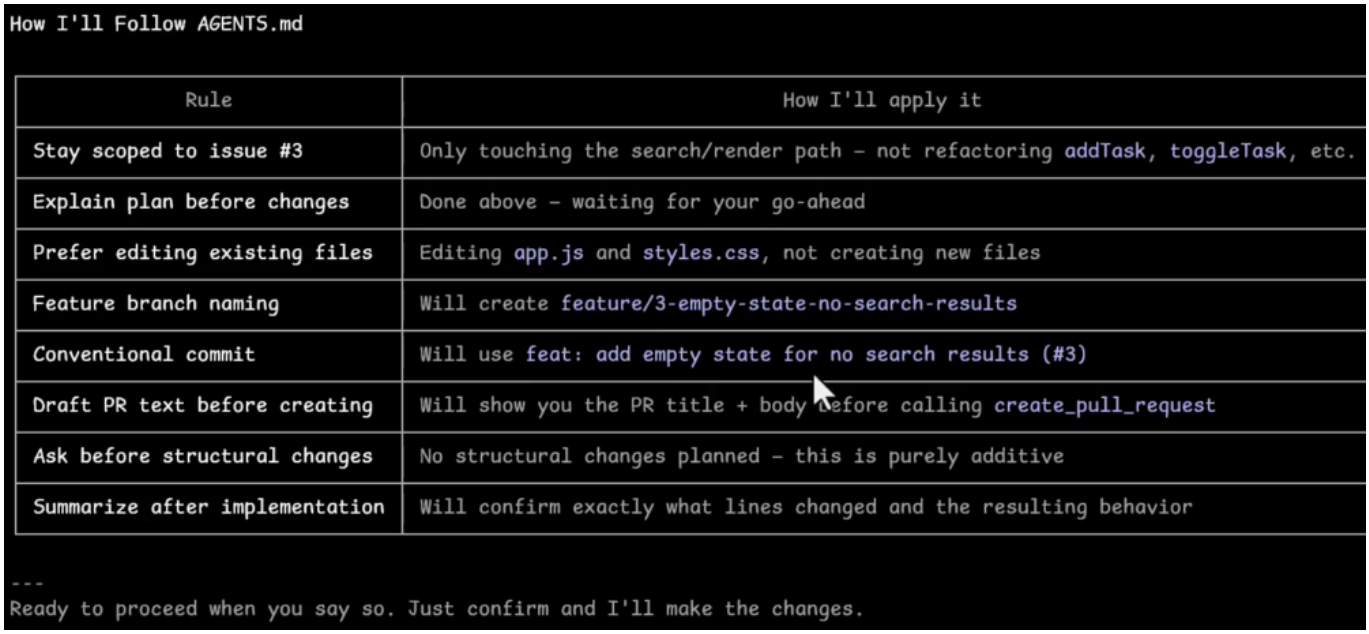
Because the prompt does not include the full repository name, Claude Code first uses a *get my profile* call from GitHub MCP to look up the current user's repositories, and then uses *get issue details* and *get file or directory contents* to read the issue and the relevant files. Once it has the context, it produces a clear restatement of the issue.

The restatement explains the underlying problem: when a user types into the search box and no tasks match the query, the task list is left as blank space with no feedback. The issue is asking for a visible empty state message, something along the lines of *No tasks match your search*, to appear instead.



The implementation plan that follows identifies the files most likely to change, a JavaScript file for the logic that renders the empty state, and a CSS file for its styling. The agent also explicitly walks through how it will follow AGENTS.md during this task. It notes the feature branch name it will use for issue #3, confirms that it will explain changes before editing, and commits to drafting the pull

request text for review before creating the pull request, and to summarizing the work after implementation.



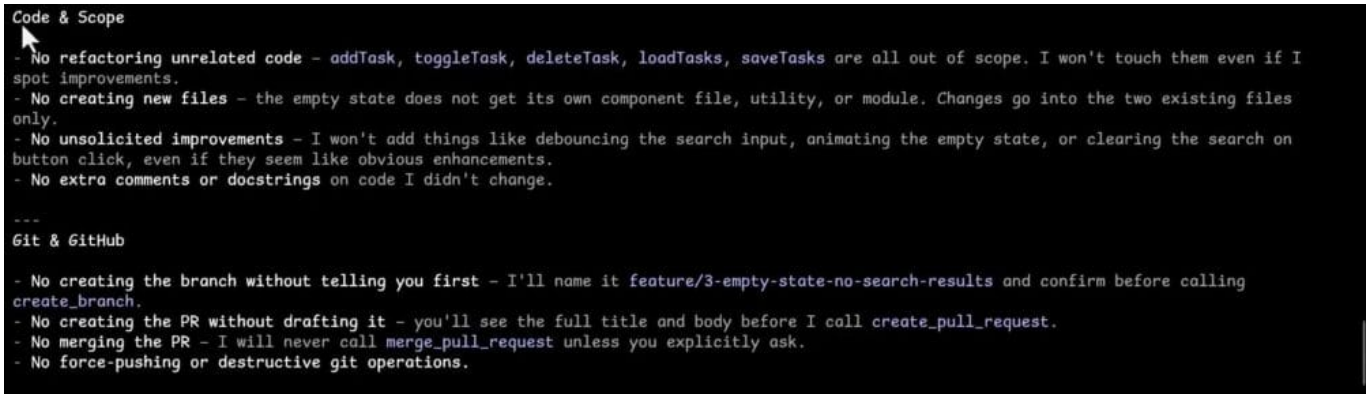
How I'll Follow AGENTS.md

Rule	How I'll apply it
Stay scoped to issue #3	Only touching the search/render path – not refactoring addTask, toggleTask, etc.
Explain plan before changes	Done above – waiting for your go-ahead
Prefer editing existing files	Editing app.js and styles.css, not creating new files
Feature branch naming	Will create feature/3-empty-state-no-search-results
Conventional commit	Will use feat: add empty state for no search results (#3)
Draft PR text before creating	Will show you the PR title + body before calling create_pull_request
Ask before structural changes	No structural changes planned – this is purely additive
Summarize after implementation	Will confirm exactly what lines changed and the resulting behavior

---
Ready to proceed when you say so. Just confirm and I'll make the changes.

Making the guardrails visible

A useful optional follow-up is to ask the agent to list what it will intentionally avoid doing while implementing the issue. The answer groups the constraints into practical categories: code and scope, Git and GitHub, and communication. On the code side, the agent commits to implementing the minimum change that satisfies issue #3, without refactoring unrelated code or making unsolicited improvements. On the Git and GitHub side, it confirms it will only create branches following the agreed naming pattern, will draft pull request text before calling the create pull request tool, and will not merge pull requests on its own. On the communication side, it promises to check in before making larger structural changes.



Code & Scope

- No refactoring unrelated code – addTask, toggleTask, deleteTask, loadTasks, saveTasks are all out of scope. I won't touch them even if I spot improvements.
- No creating new files – the empty state does not get its own component file, utility, or module. Changes go into the two existing files only.
- No unsolicited improvements – I won't add things like debouncing the search input, animating the empty state, or clearing the search on button click, even if they seem like obvious enhancements.
- No extra comments or docstrings on code I didn't change.

Git & GitHub

- No creating the branch without telling you first – I'll name it feature/3-empty-state-no-search-results and confirm before calling create_branch.
- No creating the PR without drafting it – you'll see the full title and body before I call create_pull_request.
- No merging the PR – I will never call merge_pull_request unless you explicitly ask.
- No force-pushing or destructive git operations.

The value of repository guidance

The goal of this exercise is not to repeat the full implementation cycle, we have already seen that in the previous lesson. The goal is to show how guidance shapes the workflow before any code changes begin. Claude Code is still using the same powerful GitHub MCP tools, but now it approaches the work with clearer boundaries, a tighter scope, and more predictable behavior.

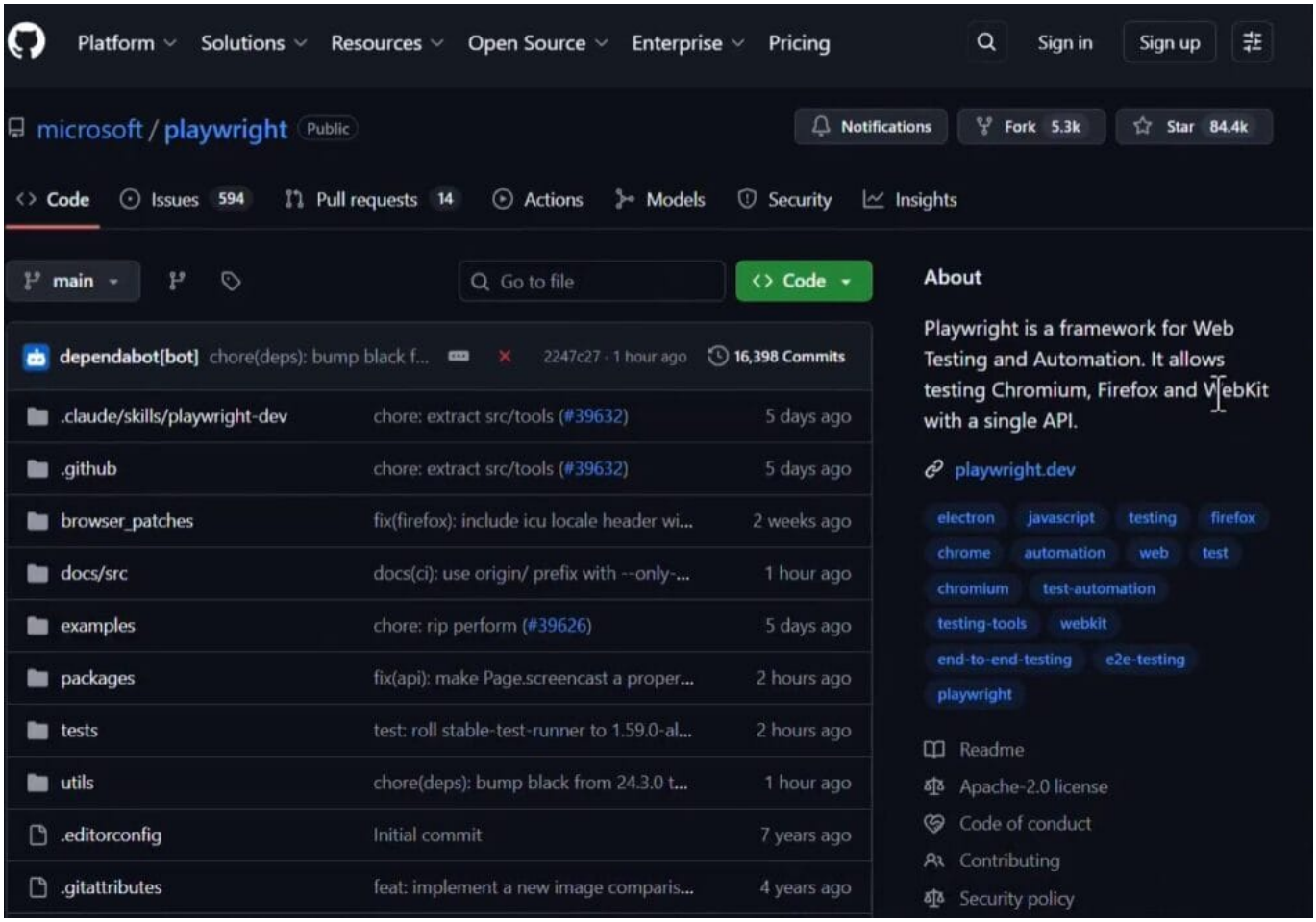
That is the real value of adding an **AGENTS.md** on top of a GitHub MCP setup. With a very small amount of written guidance, we turn a capable but somewhat freewheeling agent into a focused

collaborator that plans, confirms, and narrates its work along the way, which is exactly the kind of workflow most teams actually want from an AI coding assistant.

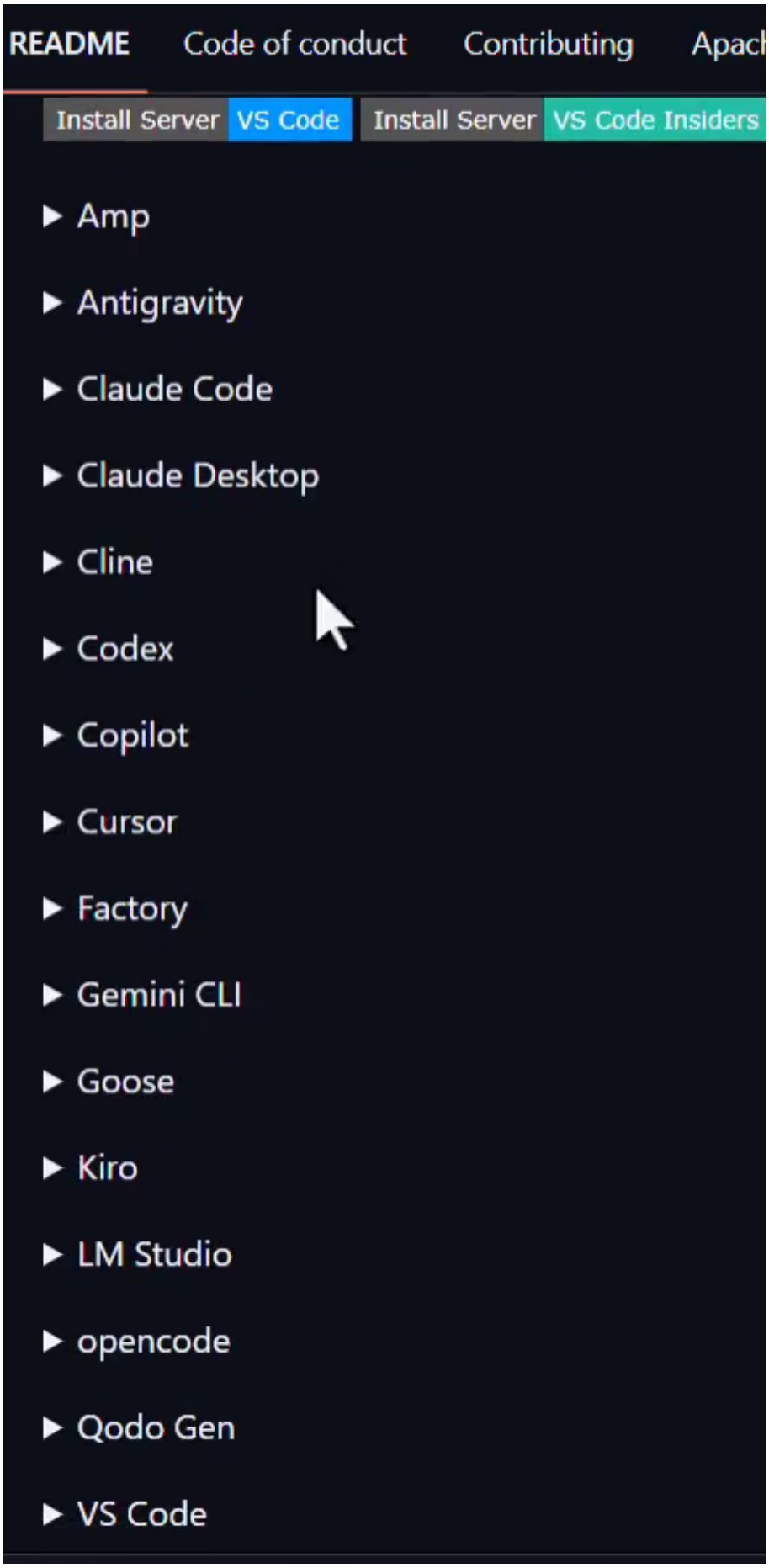
So far in this course, we have used MCP to extend Claude Code beyond the local project. First, we connected a file system MCP server, and then we used the GitHub MCP to work through a real issue-to-pull-request workflow. In this lesson, we move into browser automation with the official Playwright MCP server. This is the point where Claude Code stops simply reading files and repository context and starts interacting with live web pages in a structured way. We will install and configure the Playwright MCP in Claude Code on your system, verify that it is connected, and then use it on a real browser workflow.

A quick look at Playwright

Before installing the MCP server, it helps to understand what Playwright is. Playwright is a framework, maintained by Microsoft, for web testing and automation. It allows testing Chromium, Firefox, and WebKit browsers through a single API, which makes it a popular choice for end-to-end browser automation.



For Claude, there is an official Playwright MCP server that exposes this functionality as a set of tools. The server provides browser automation capabilities using Playwright, and it enables large language models to interact with web pages through structured accessibility snapshots, bypassing the need for screenshots or visually tuned models. In other words, Claude reasons about the page through its accessibility tree rather than by looking at pixel images.



The Playwright MCP repository on GitHub provides installation instructions for several clients, including Claude Code, Claude Desktop, and others. Since we are working with Claude Code inside the CLI, we follow the Claude Code entry, which provides a single command we can copy.

Installing the Playwright MCP in Claude Code

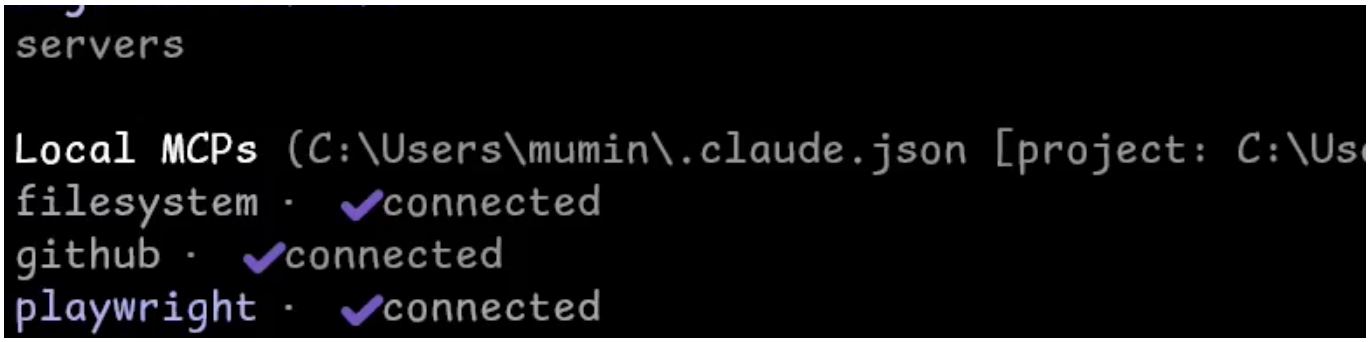
With the command copied, we return to VS Code and open the integrated terminal. In the terminal, paste and run the install command:

```
claude mcp add playwright -- npx @playwright/mcp@latest
```

This registers the Playwright MCP server with Claude Code so that its tools become available in future sessions. After the command completes, we clear the terminal and launch Claude Code itself. From inside Claude Code, we check the list of connected MCP servers using the built-in command:

```
/mcp
```

The resulting panel shows the local MCP servers configured for this project. Alongside the previously configured filesystem and github servers, playwright now appears as connected and ready to use.



Tools provided by the Playwright MCP

Pressing enter on the Playwright entry reveals its full tool list. At the time of this lesson, the server exposes around 22 tools that cover browser control and page interaction. A representative selection includes:

- navigate_to_url for opening a target page
- type_text for typing into input fields
- click for clicking on page elements
- take_screenshot for capturing the current page
- get_console_messages for reading browser console output
- fill_form for populating forms in one step
- press_key, drag_mouse, select_option, upload_file
- Tools to close or resize the browser and to manage tabs

These tools, taken together, let Claude drive a real browser end to end rather than working from static text alone.

Choosing a target website

To try the MCP in practice, we need a target website. For this first workflow, we use Playwright's TodoMVC demo at https://demo.playwright.dev/todomvc/. It is a small React based single page app that supports adding, completing, editing, deleting, and filtering todos using URL hash routing, which makes it a convenient example for demonstrating browser automation.

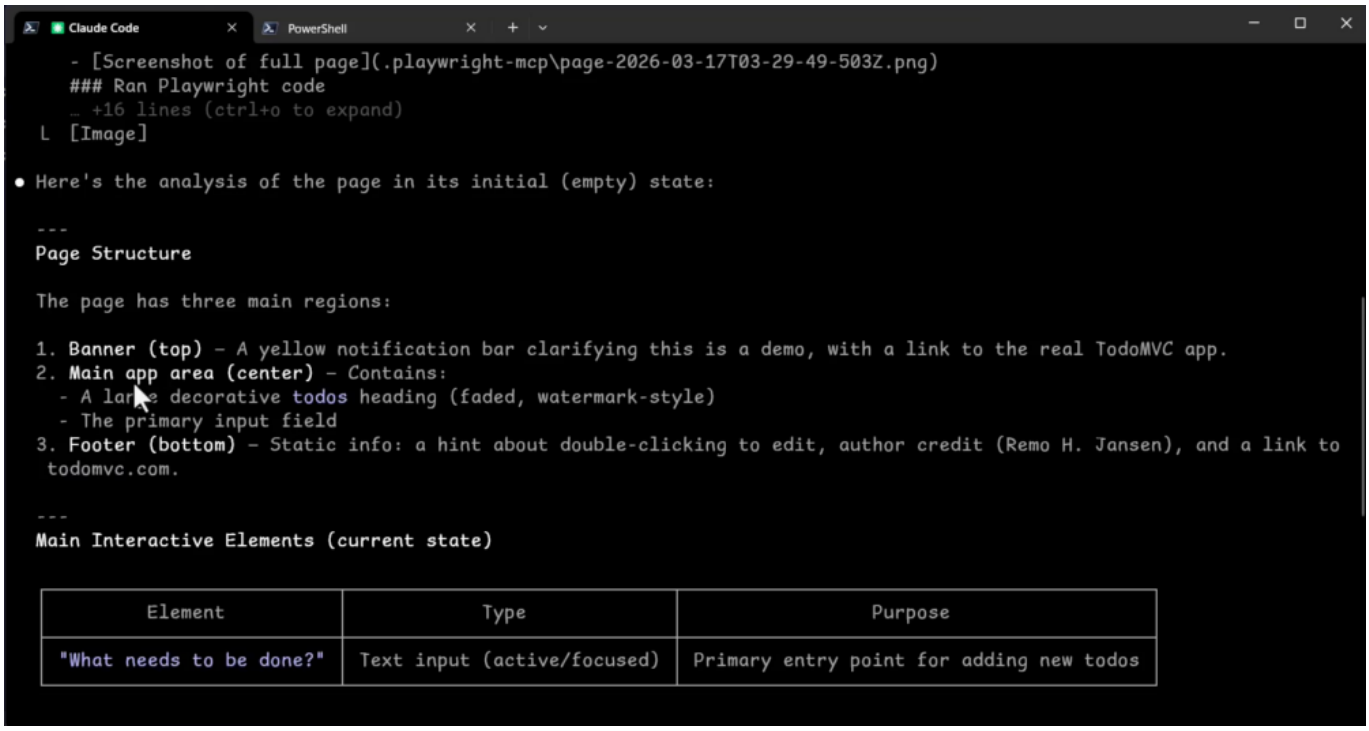
Workflow 1: inspecting the page before acting

Rather than jumping straight into clicks, it is useful to start with an inspection step. This makes the workflow feel intentional instead of throwing clicks at the page and hoping for good results. We write a prompt in Claude Code that asks Playwright MCP to open the page and describe it, without taking any interactive actions yet:

```
Use Playwright MCP to open https://demo.playwright.dev/todomvc

Then, describe the page structure briefly, identify the main interactive elements, ex
plain what user actions this app appears to support. Do not start interacting yet.
```

When we run this prompt, Claude selects the navigate_to_url tool from the Playwright MCP. After we grant permission, a new browser window opens and Claude performs the navigation and then analyzes the page.



The analysis describes three main regions: a banner at the top, a main application area in the center containing a large decorative todos heading and the primary input field, and a footer with static information, a hint about double-clicking to edit, and author credit. It identifies the text input labeled “What needs to be done?” as the primary interactive element. It also notes that additional controls like checkboxes, filter tabs, and the “Clear completed” button are rendered dynamically once items are added. Finally, it explains the supported user actions, including adding, completing, editing, deleting, and filtering todos, and recognizes that the app is a React based single page app using URL hash routing for filter state.

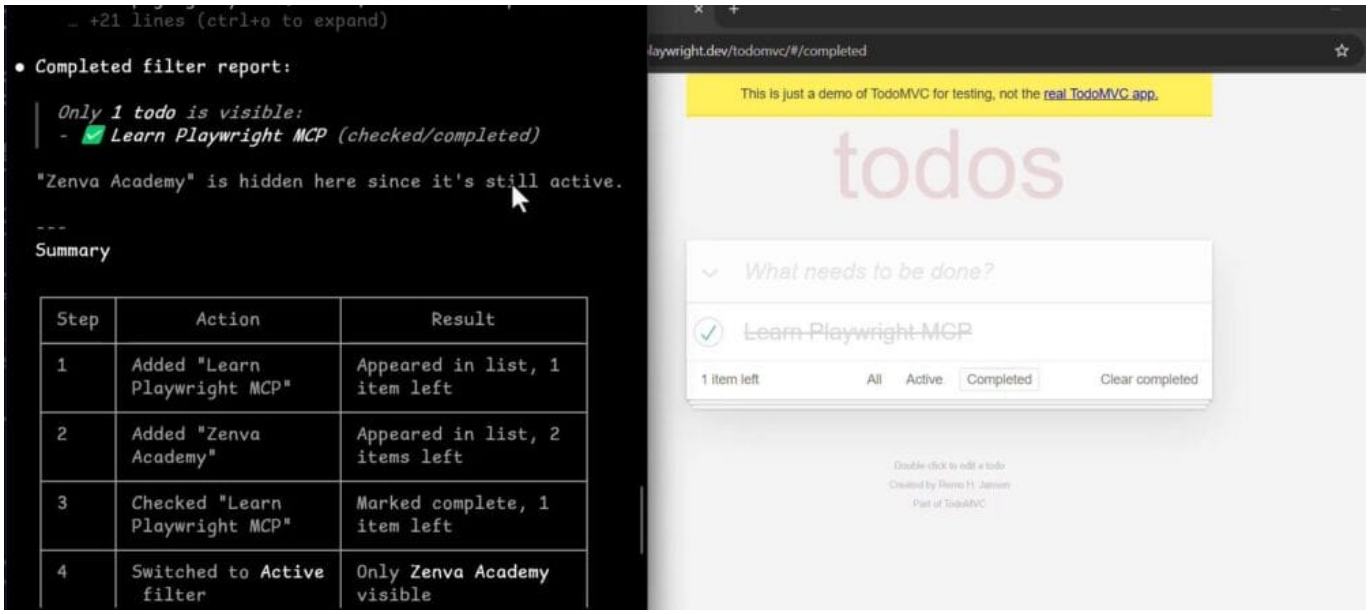
Workflow 2: interacting with the TodoMVC app

Now that we understand the page, we can move on to a more complete workflow that mixes interaction and reasoning. We write a second prompt that asks Claude to run through a sequence of actions and report the state after each one:

```
Using Playwright MCP on the TodoMVC page: add a todo named Learn Playwright MCP, add a second todo named Zenva Academy, mark the first todo as completed, switch to the Active filter, report what todos are available, switch to the Completed filter and finally, report what todos are visible. Explain each step briefly as you go.
```

This is a small workflow, but it is enough to show that Claude can operate a real web app through the Playwright MCP and reason about the state changes that follow each action. After granting permission for each tool call, Claude uses type_text to add the two todos, then click to check off the first item and to switch between the Active and Completed filters. Between actions it inspects the page and reports what is visible.

Once the sequence finishes, Claude summarizes the result. Under the Active filter, only “Zenva Academy” is visible, since “Learn Playwright MCP” has been marked as complete. Under the Completed filter, only “Learn Playwright MCP” is visible, while “Zenva Academy” is hidden because it is still active.



Why this matters

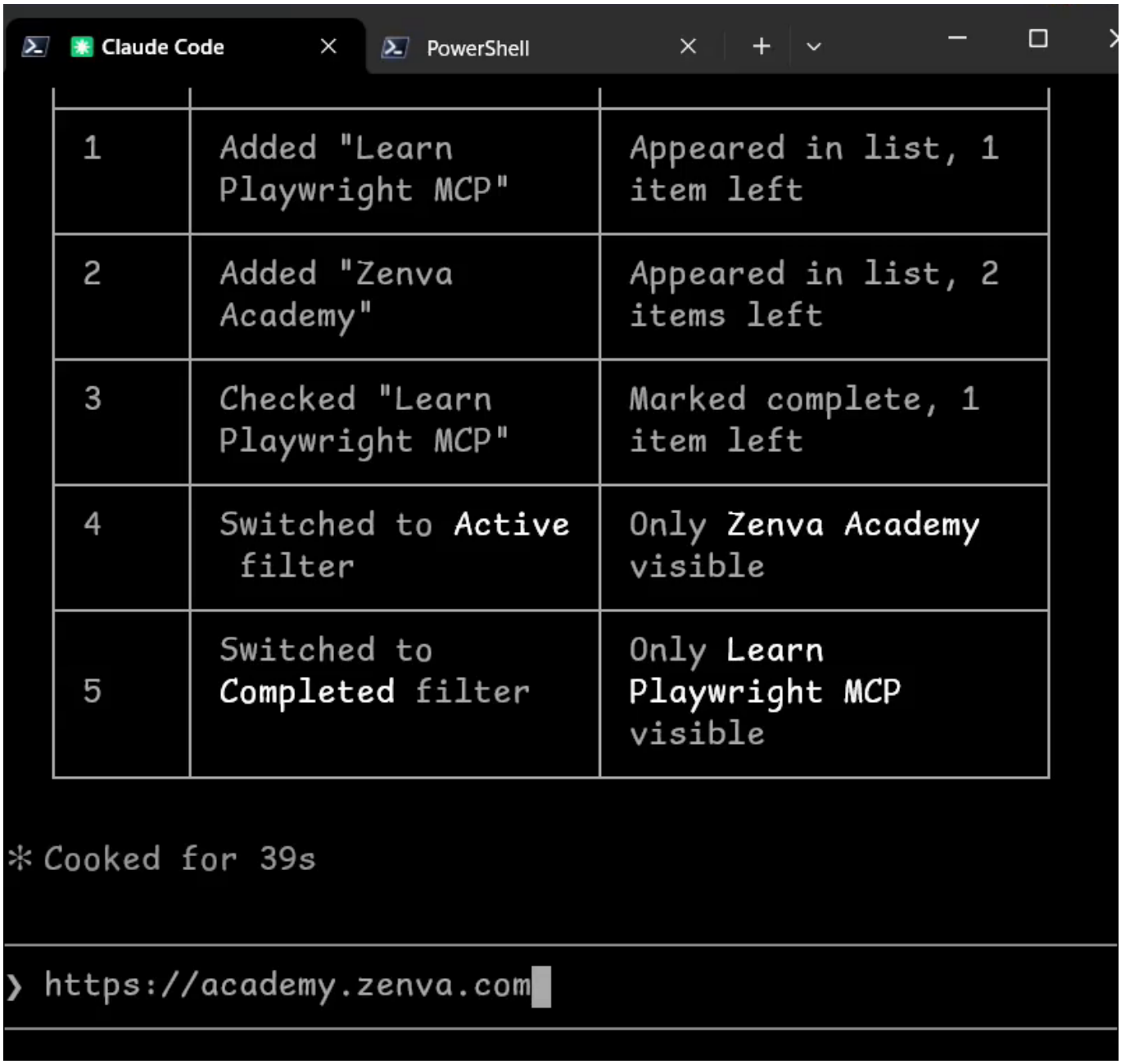
The key takeaway is that the Playwright MCP is not just a way to control a browser. It gives Claude structured access to a live browser environment, which means Claude can inspect, navigate, interact, and reason about the UI state in a way that is far more grounded than a normal text-only workflow. With this foundation in place, we are ready to build more ambitious browser automation tasks on top of it in the following lessons.

This lesson presents a short, hands-on challenge designed to put the Playwright MCP workflow into practice. Rather than introducing new material, it asks you to combine Claude Code with the Playwright MCP server and apply both to a real, public website.

The challenge

Your goal is to use Claude Code together with the Playwright MCP to analyze a real website, the Zenva Academy site, and figure out what kinds of AI-related learning options or AI courses are visible or available from the homepage and its course navigation.

The target website is https://academy.zenva.com.



Approach: guided investigation, not scraping

This exercise is intentionally not about brute force scraping or dumping raw HTML out of the page.

The point is guided browser investigation, where you direct Claude Code through the Playwright MCP to look at the site the way a careful person would.

You should approach the site as a careful user and a careful analyst. A useful sequence of steps is:

1. Inspect the structure of the homepage to understand what is on the page.
2. Navigate the menus that organize courses and topics.
3. Use search if it is available and helpful for surfacing AI-related content.
4. Summarize what you find clearly, focusing on the AI-related options that are actually visible.

What success looks like

A good outcome is a concise, structured summary that describes which AI-focused courses or learning paths are discoverable from the Zenva Academy homepage and its main navigation, and how a new visitor would come across them. Keep the focus on what a normal visitor can see, rather than on extracting large amounts of raw page data.

Take your time with the investigation, and good luck with the challenge.

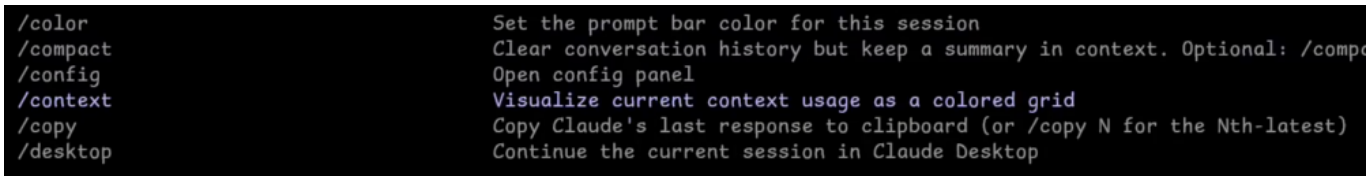
In this lesson we walk through the solution to the challenge of using Claude Code together with the Playwright MCP to analyze a public website. The goal is to see, step by step, how a natural-language prompt can be turned into an automated browser session that inspects the Zenva Academy homepage, collects information about AI-related courses, and produces a concise, structured summary. Along the way we will also see a few useful Claude Code commands for keeping the session clean and readable.

Preparing a clean Claude Code session

Before running a new task, it helps to start with a tidy Claude Code session so that previous conversation history does not consume context. Two slash commands are particularly useful here:

- **/clear**, which clears the conversation history and frees up the context window.
- **/context**, which visualizes the current context usage as a colored grid so you can see how much space is still available.

Running /context reveals the available free space (for example, around 72% free in this session). Even when there is plenty of room, running /clear before a focused task like this one is a good habit, since it keeps Claude's attention on the new instructions rather than on older, unrelated turns.



Writing the prompt for Playwright MCP

With a clean session ready, the next step is to write a prompt that tells Claude Code exactly what to do with the Playwright MCP. A good prompt for this kind of task has three parts: a clear objective, a numbered list of steps, and a short list of constraints.

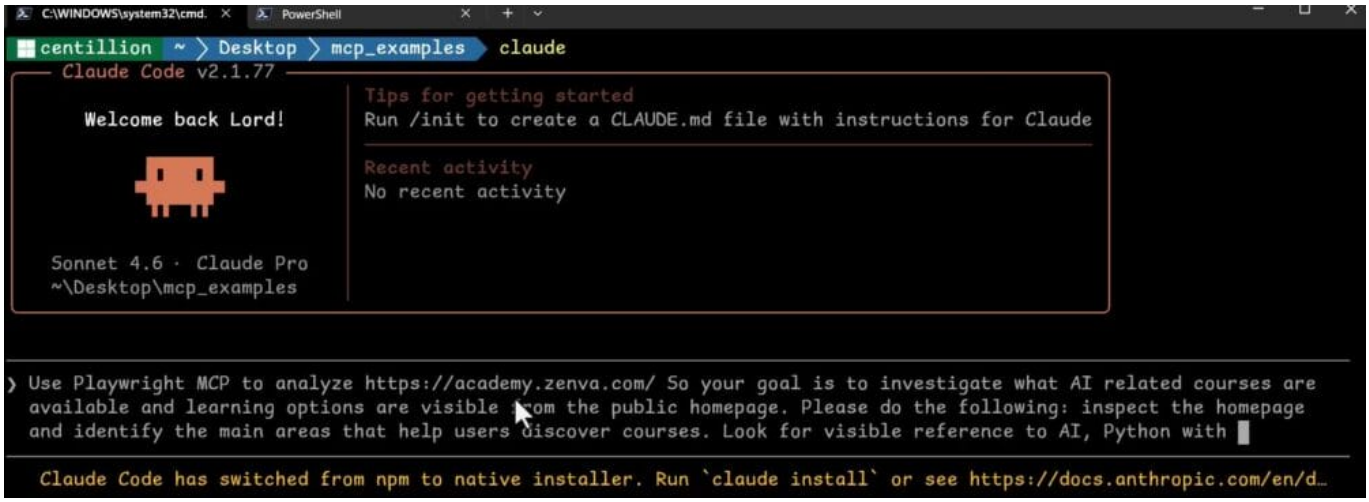
The objective is to analyze the Zenva Academy homepage and investigate which AI-related courses and learning options are visible to a public visitor. From there, the prompt breaks the task into concrete steps:

1. Inspect the homepage and identify the main areas that help users discover courses.
2. Look for visible references to AI, Python, or "Python with AI" and similar topics.
3. Open the courses navigation menu and identify categories or subcategories related to AI.
4. Use the course search, if needed, to check how a learner might look for AI courses.
5. Summarize the findings in a clear, structured report.

The constraints keep the run safe and focused:

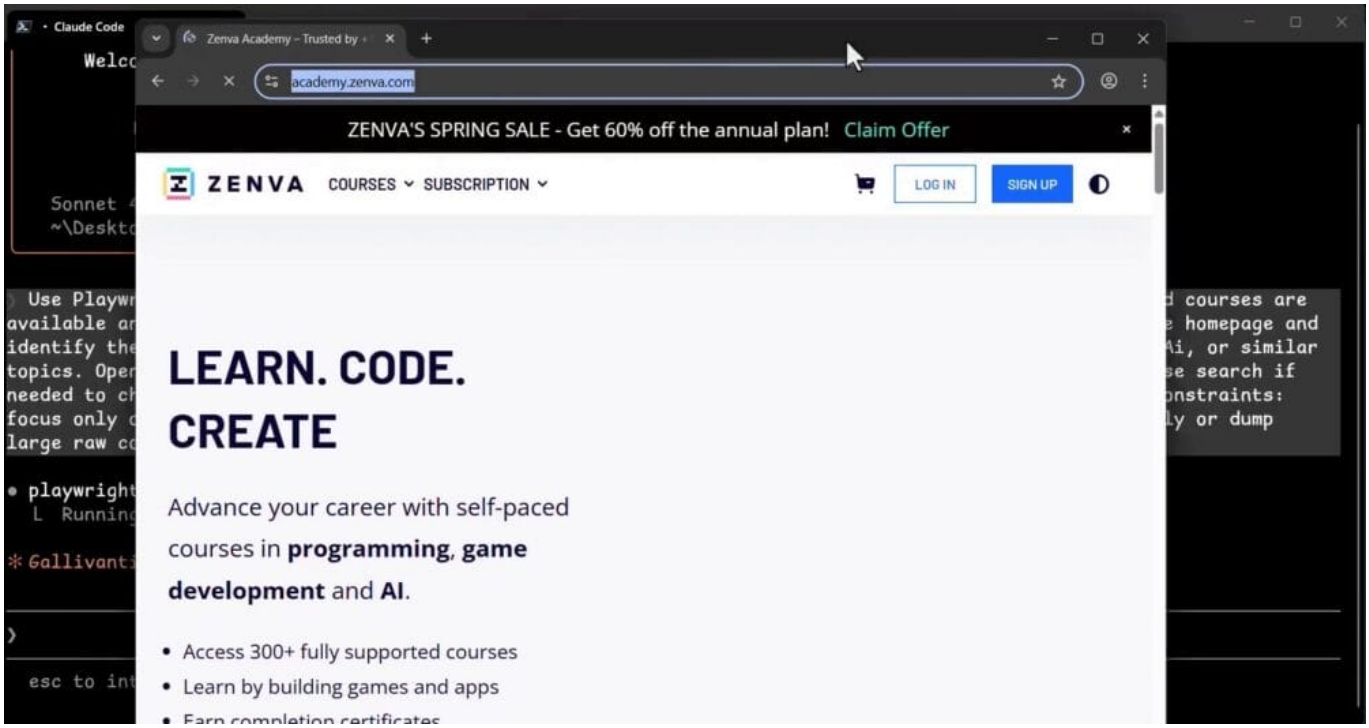
- Focus only on public, visible content.
- Do not attempt to log in or access private areas.
- Do not scrape excessively or dump large amounts of raw content.
- Be concise and structured in the final summary.

Typing these instructions into Claude Code gives the agent enough guidance to drive the browser responsibly and produce a useful report at the end.



Watching the automation run

After submitting the prompt, the windows are arranged so that Claude Code and the Playwright browser are visible side by side. The Playwright MCP opens a browser, navigates to academy.zenva.com, and begins inspecting the homepage. You can watch the automation click around the navigation, open the relevant menus, and read the visible content, all while Claude Code streams its reasoning and progress in the terminal. Because both views are visible at once, it becomes easy to follow what the agent is doing and why.

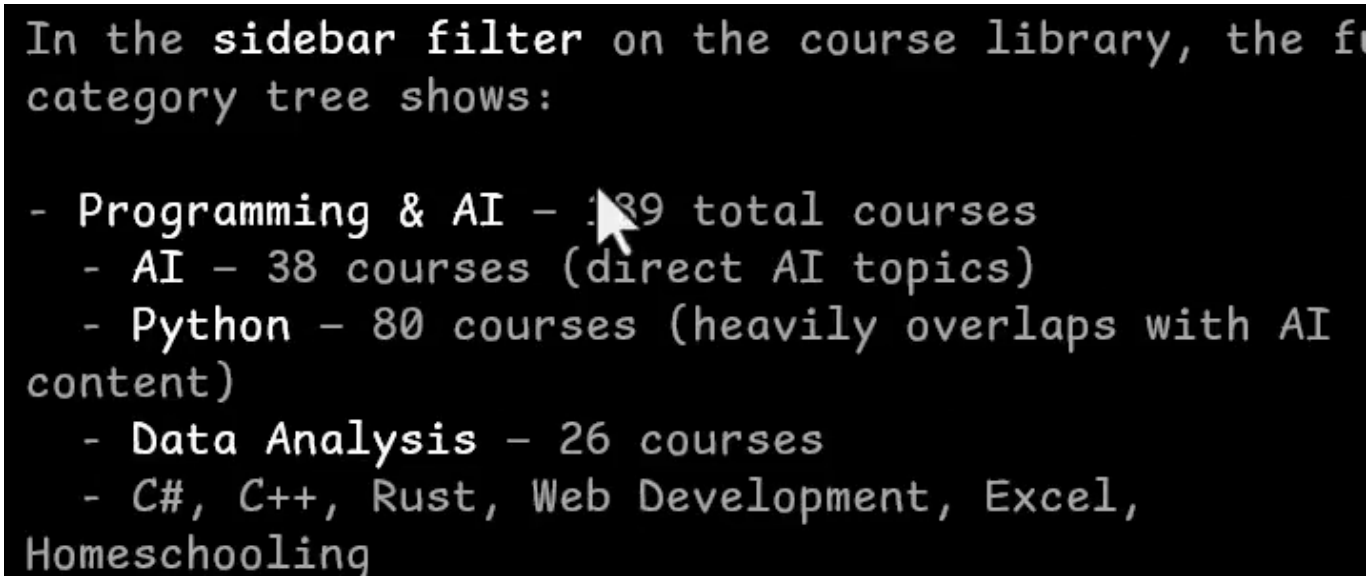


Reviewing the findings

Once the browser walkthrough is complete, Claude Code produces a structured report. The homepage summary identifies the main tagline, the sale banner, and the primary courses navigation, including a direct link to “Programming with AI”.

Looking into the courses navigation, the report shows that the broader “Programming and AI” category contains 189 total courses, of which 38 are specifically AI-tagged. The remaining courses in

this category cover related topics such as Python, data analysis, C#, and web development, giving a clear sense of how AI content fits alongside other programming areas on the site.



AI tooling and assistant courses

Within the AI and Data Science listing, Claude Code groups the results by theme. The first group, “AI Tooling and Assistants”, highlights newer or trending courses aimed at learners who want to work with AI coding assistants directly. The table shows each course along with its level and duration, making it easy to pick something that matches a learner’s background.

Notable entries in this group include a new beginner course on best practices for AI-powered coding with Claude Code, another beginner course on AI-assisted development with Claude Code, a short introduction to vibe coding with Cursor, and two entry-level courses covering prompt engineering for programmers and building customizable chatbots with OpenAI GPTs. Between them, these titles cover the main skills someone would need to start pairing an IDE or terminal with an AI assistant.

Course	Level	Duration
Best Practices for AI-Powered Coding with Claude Code *(New)*	Beginner	1h 15m
AI-Assisted Development with Claude Code *(New)*	Beginner	1h 16m
Learn Vibe Coding with Cursor *(New)*	Beginner	47m
Prompt Engineering for Programmers	Entry Level	41m
Build Customizable AI Chatbots using OpenAI GPTs	Entry Level	50m

Intermediate Python and AI courses

Moving on from tooling, the analysis identifies a second group of courses for learners who want to build AI-powered applications in Python. These are intermediate-level options and include a course on Python AI development with Flask and Llama, as well as a course on retrieval augmented generation (RAG) with LlamaIndex. Together they give a practical path from basic AI tooling into real application development, where learners combine web frameworks, language models, and vector search in a single project.

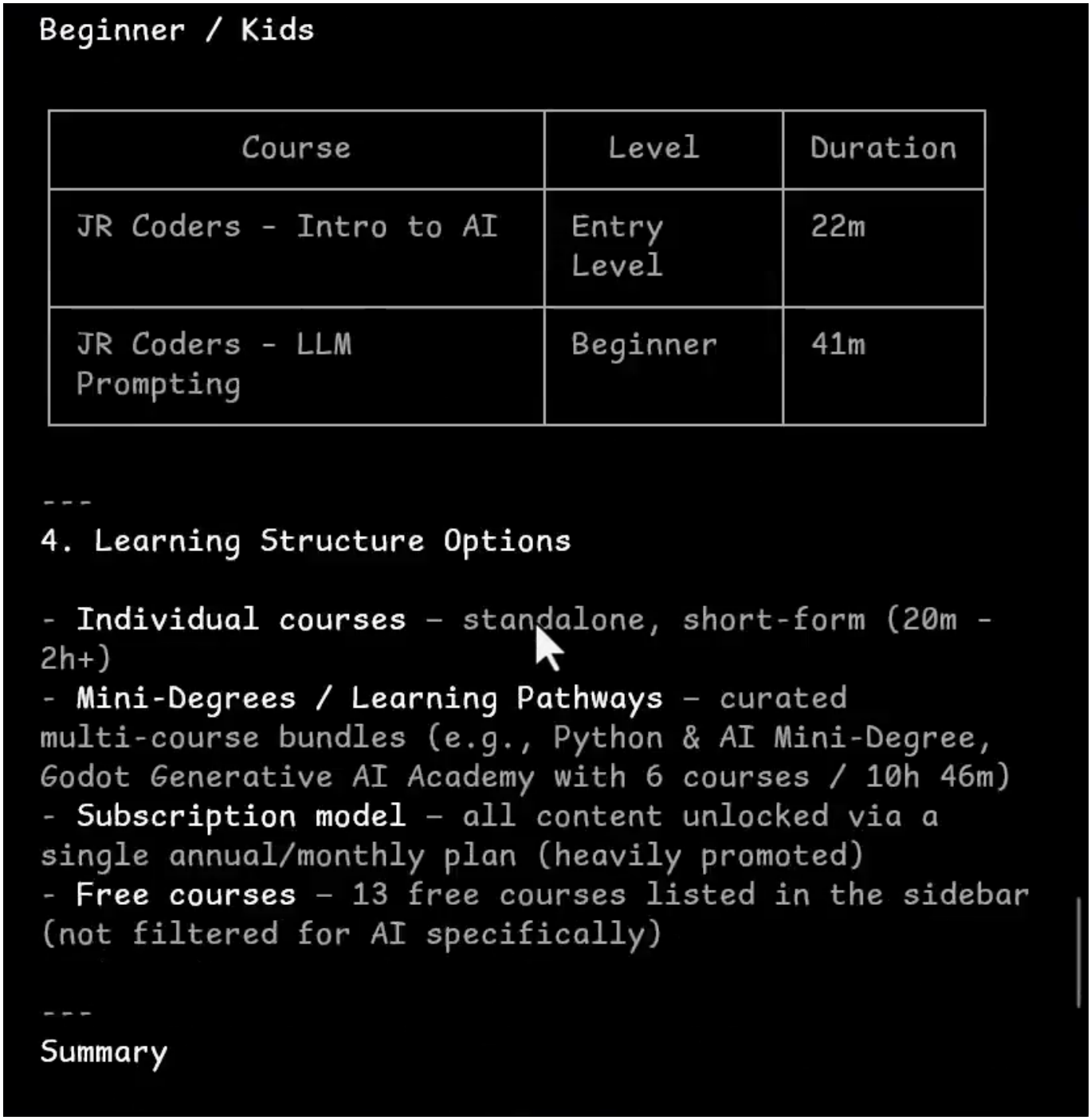
Learning structure options and skill levels

The final section of the report focuses on how courses are organized. Claude Code identifies four main ways a learner can access content on the platform:

- **Individual courses**, which are standalone and usually run between about 20 minutes and a couple of hours.
- **Mini-degrees and learning pathways**, which bundle several related courses into a longer, curated track (for example, a Python and AI mini-degree or a multi-course pathway for generative AI in Godot).
- **Subscription model**, where all content is unlocked through a single annual or monthly plan.
- **Free courses**, with a set of free options available from the sidebar.

The analysis also calls out that the 38 AI-tagged courses span different skill levels. There are entry-level and beginner options, including kid-friendly tracks such as JR Coders Intro to AI and JR Coders

LLM Prompting, alongside more advanced courses for experienced developers. This range means a complete beginner and a working engineer can both find a reasonable starting point.



Key takeaways and next steps

This challenge solution shows that Claude Code becomes much more capable once it is connected to the right MCP servers. Over the course of the module we started with the fundamentals, moved into real setup and tool registration, explored local file system access, used the GitHub MCP for an issue-to-pull-request workflow, added guidance through AGENTS.md, and now used the Playwright MCP for browser automation with real-world analysis.

The bigger idea across all of these sessions is that Claude Code is not just a chat interface for code.

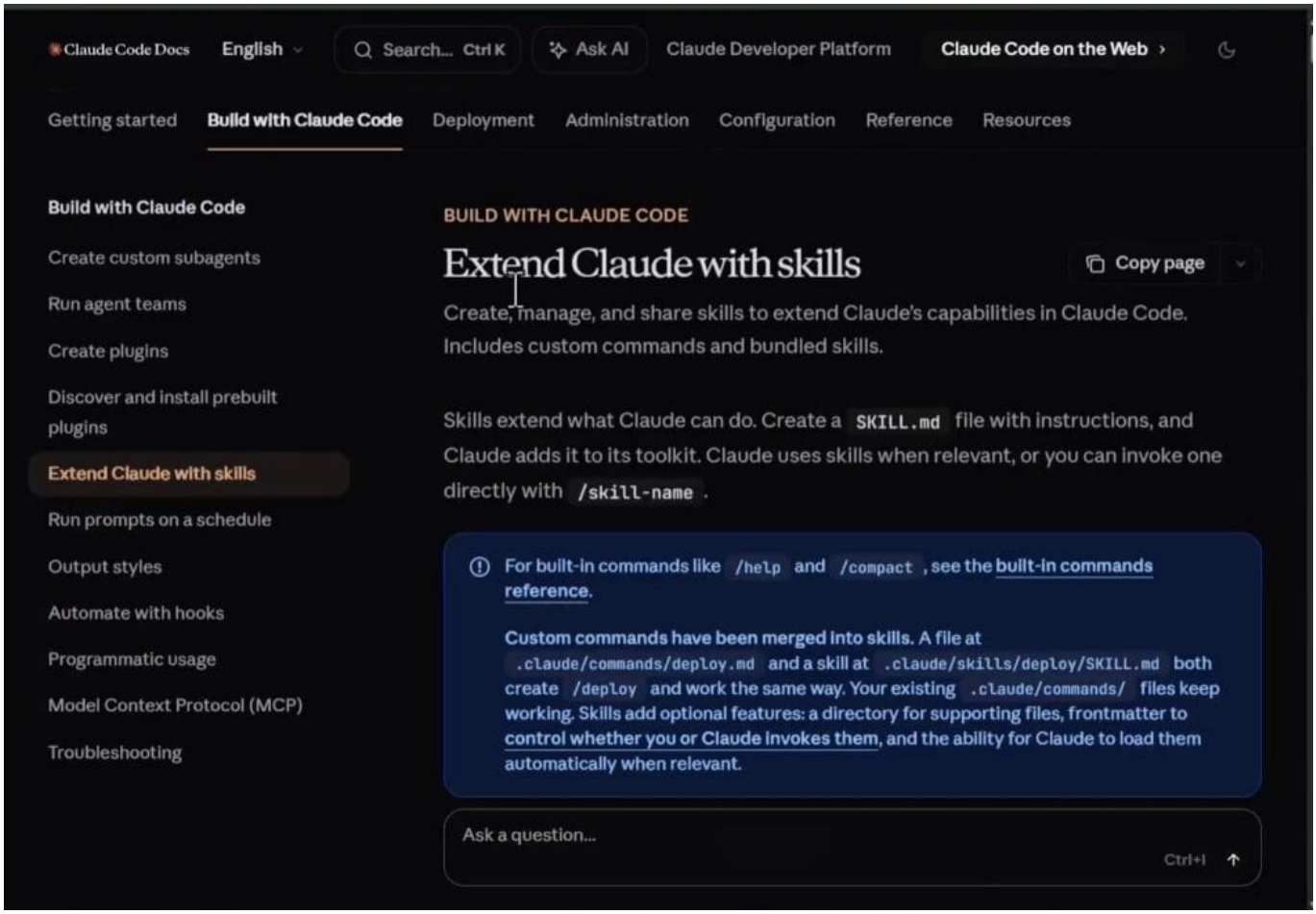
With the right structure, it becomes a practical development environment that can reason across files, repositories, workflows, and even live browser sessions. The goal has never been to memorize a list of tools, but to combine them into workflows that are useful, controlled, and repeatable.

From here, the next step is to build your own workflows, experiment with the MCP servers that best fit your projects, and learn how to shape Claude Code into a more reliable engineering partner.

So far in this course, we have connected Claude Code to external capabilities through MCP. However, one key piece is still missing: repeatability. If you find yourself running the same kind of investigation or task again and again, rewriting a large, detailed prompt every time is inefficient. This is exactly the problem that Skills are designed to solve.

What is a skill?

A Skill lets you package a reusable workflow so that Claude can apply it when it recognizes a relevant task, or so that you can invoke it directly for a specific behavior. According to the official Claude Code documentation, skills are folders of instructions, scripts, and resources that Claude loads dynamically to improve performance on specialized tasks. In short, skills teach Claude how to complete specific tasks in a repeatable, structured way.



Skills can be used in many different contexts, for example:

- Creating documents that follow your company's brand guidelines.
- Analyzing data with your organization's specific workflows.
- Automating personal tasks that involve a set of rules and constraints.

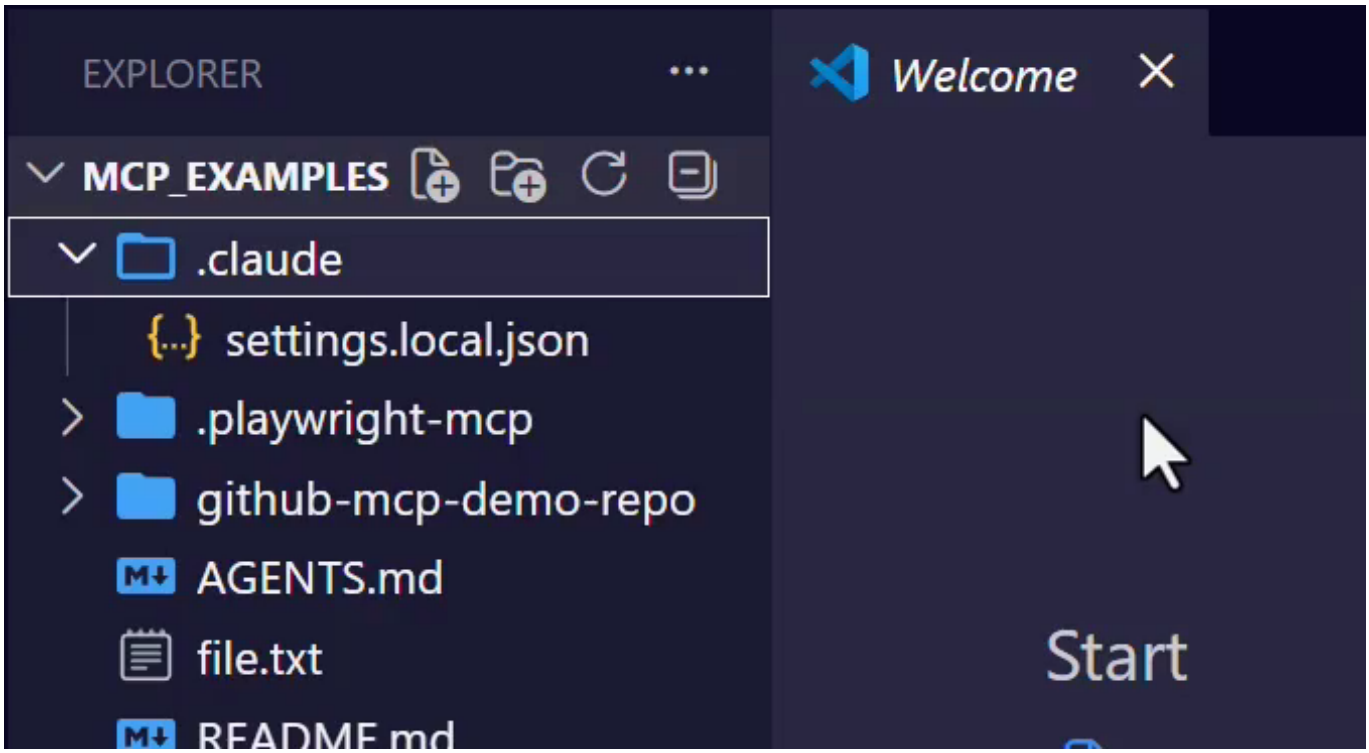
Anthropic also provides a public GitHub repository at anthropics/skills that collects official examples and references for how skills are built and used.

Instead of stuffing every specialized instruction into your CLAUDE.md file, you can create a focused skill for a specific kind of task. This is useful because CLAUDE.md is loaded at the start of every session, while skills are loaded on demand when they are actually used. This practice, also recommended by Anthropic's cost guidance, helps reduce token usage with your LLM provider.

Creating a project-level skill

Let’s build a practical example: a skill named playwright-site-investigator. The goal is simple. Whenever we want Claude to analyze a public website with the Playwright MCP, we want it to follow a predefined, structured workflow. The skill will act as a playbook for how to use the Playwright MCP consistently.

We are going to work inside a project folder that already has a Claude Code session initialized, so a .claude folder exists. Inside that folder, we will create the skill.



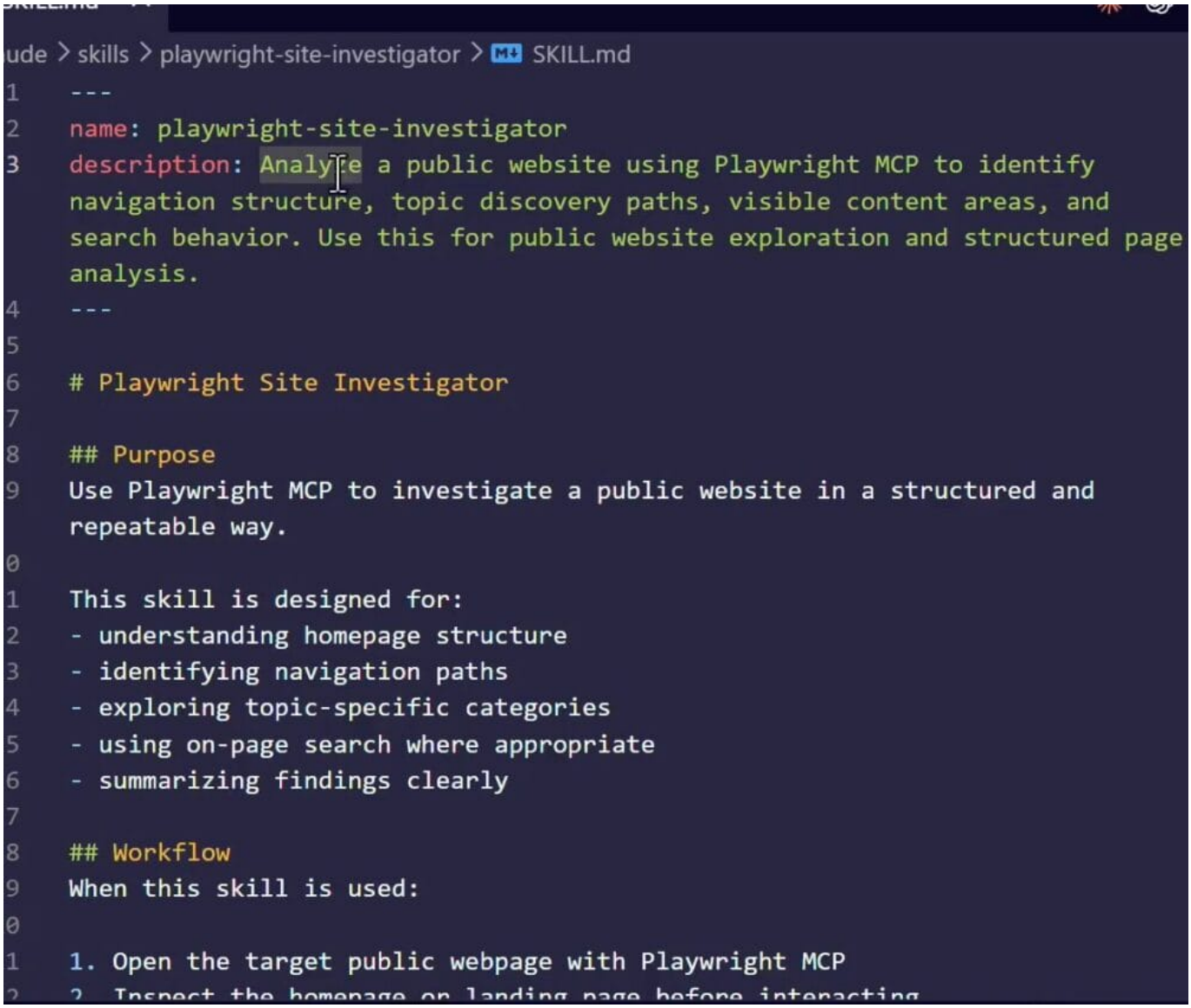
From the project root, follow these steps in VS Code or your editor of choice:

1. Open the .claude folder, which was created when you initialized the Claude Code session in your project.
2. Inside .claude, create a new folder named skills.
3. Inside skills, create another folder for your specific skill. We will name it playwright-site-investigator.
4. Finally, inside that folder, create a new file named SKILL.md.

The resulting path should be .claude/skills/playwright-site-investigator/SKILL.md.

Writing the SKILL.md file

The SKILL.md file is where you define the behavior of your skill. It uses YAML front matter at the top for metadata (name and description), followed by a markdown body that describes the skill’s purpose, workflow, rules, and output format. This structure is what Claude reads when deciding whether to apply the skill and how to execute it.



Here is the full contents of the file for our Playwright Site Investigator skill:

```
---
name: playwright-site-investigator
description: Analyze a public website using Playwright MCP to identify navigation str
ucture, topic discovery paths, visible content areas, and search behavior. Use this f
or public website exploration and structured page analysis.
---

# Playwright Site Investigator

## Purpose
Use Playwright MCP to investigate a public website in a structured and repeatable way
.

This skill is designed for:
- understanding homepage structure
- identifying navigation paths
- exploring topic-specific categories
- using on-page search where appropriate
```



```
- summarizing findings clearly

## Workflow
When this skill is used:

1. Open the target public webpage with Playwright MCP
2. Inspect the homepage or landing page before interacting
3. Identify major navigation areas, menus, search fields, and topic discovery element
s
4. Look for content relevant to the requested topic
5. Use search only if it helps clarify discoverability
6. Summarize findings in a concise and structured way

## Rules
- Focus only on public, visible content
- Do not attempt login or private/account areas
- Do not dump large raw page content
- Do not scrape excessively
- Prefer concise summaries over verbose transcripts
- Explain what was found and where it was found in the UI

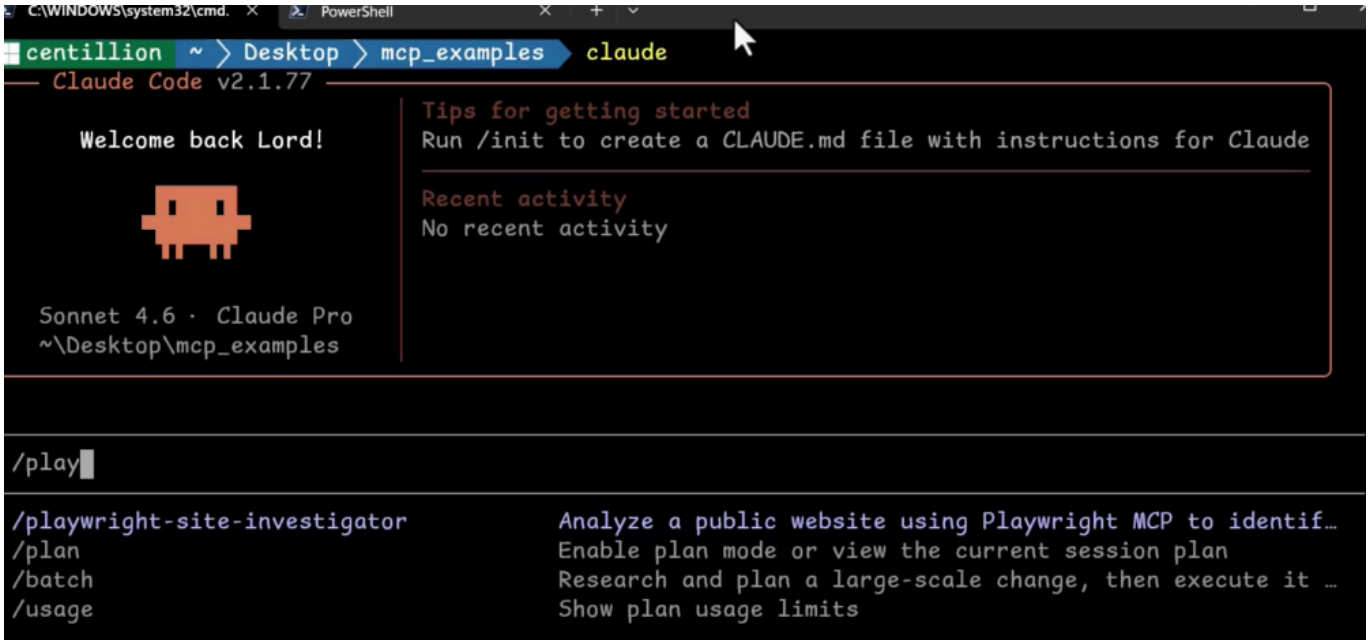
## Output format
Return:
1. a short summary of what the site offers related to the topic
2. where the relevant content was found
3. how a new user could discover it
4. any observations about clarity or discoverability
```

This is a very natural fit for the Playwright MCP, because browser investigation tends to follow the same pattern every time: open the page, inspect the structure, look at the navigation, search if needed, and summarize the findings. Instead of rewriting that whole strategy for every new prompt, we package it once as a skill and reuse it.

Invoking the skill

Once the skill is saved, we can invoke it directly from a Claude Code session. Open a terminal inside the same project folder (so that Claude can see the local .claude directory) and start a session with the claude command.

At the prompt, type a forward slash followed by the skill name. Claude Code will detect the skill and offer it as an autocomplete suggestion, because the skills folder lives inside .claude.



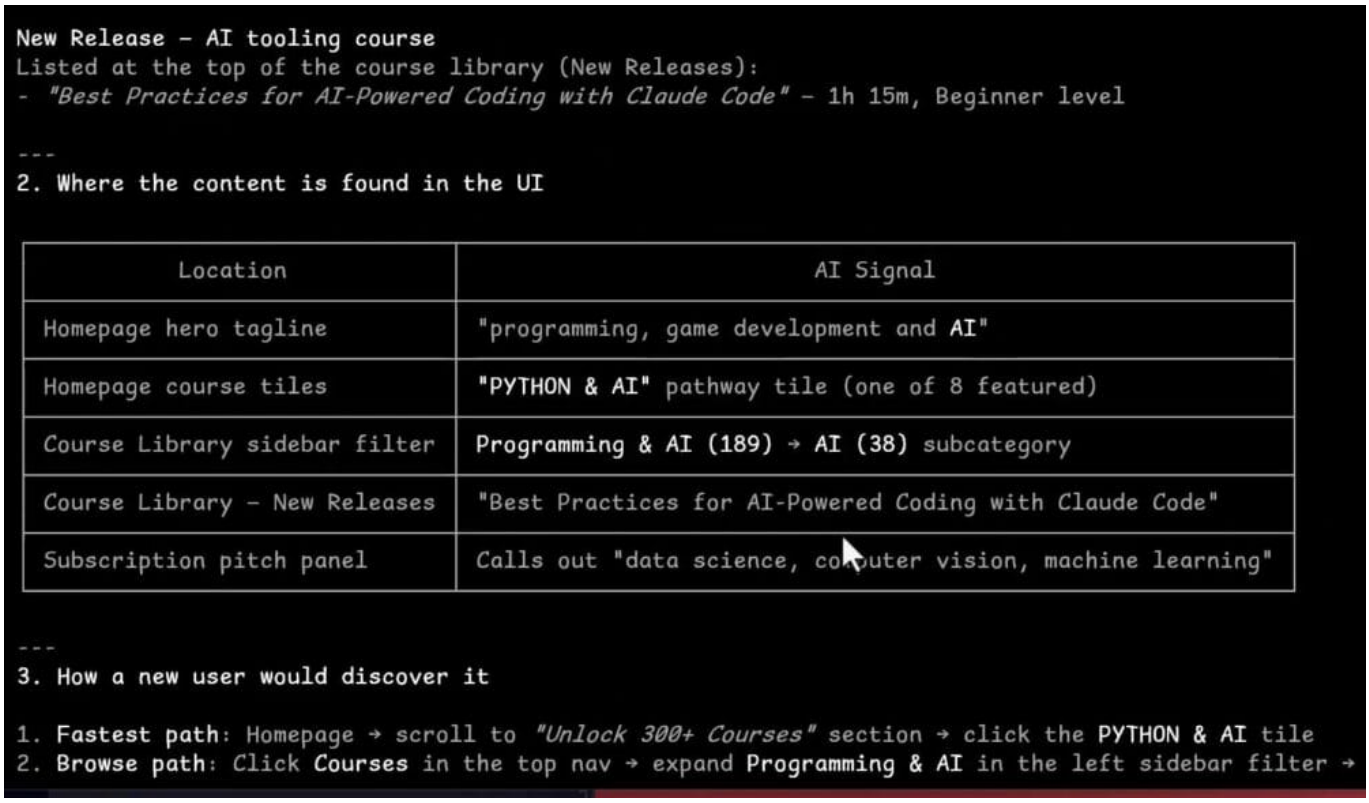
Select the suggestion by pressing Tab, then type the specific request after the skill name. For this example, we will ask Claude to investigate the Zenva Academy website:

```
/playwright-site-investigator Analyze https://academy.zenva.com/ and determine what AI related learning options are visible from the public home page and navigation.
```

This is the same kind of investigation we explored earlier in the course, but now the workflow is packaged as a reusable skill instead of being spelled out from scratch in one giant prompt.

Running the skill

Once submitted, Claude runs the investigation. The browser capability still comes from the Playwright MCP, but the approach, the sequence of steps, and the output structure are all driven by the skill we wrote. The output is organized according to the format we defined: a short summary of what was found, where it was found in the UI, how a new user could discover it, and observations about clarity and discoverability.



New Release – AI tooling course
Listed at the top of the course library (New Releases):
- *"Best Practices for AI-Powered Coding with Claude Code"* – 1h 15m, Beginner level

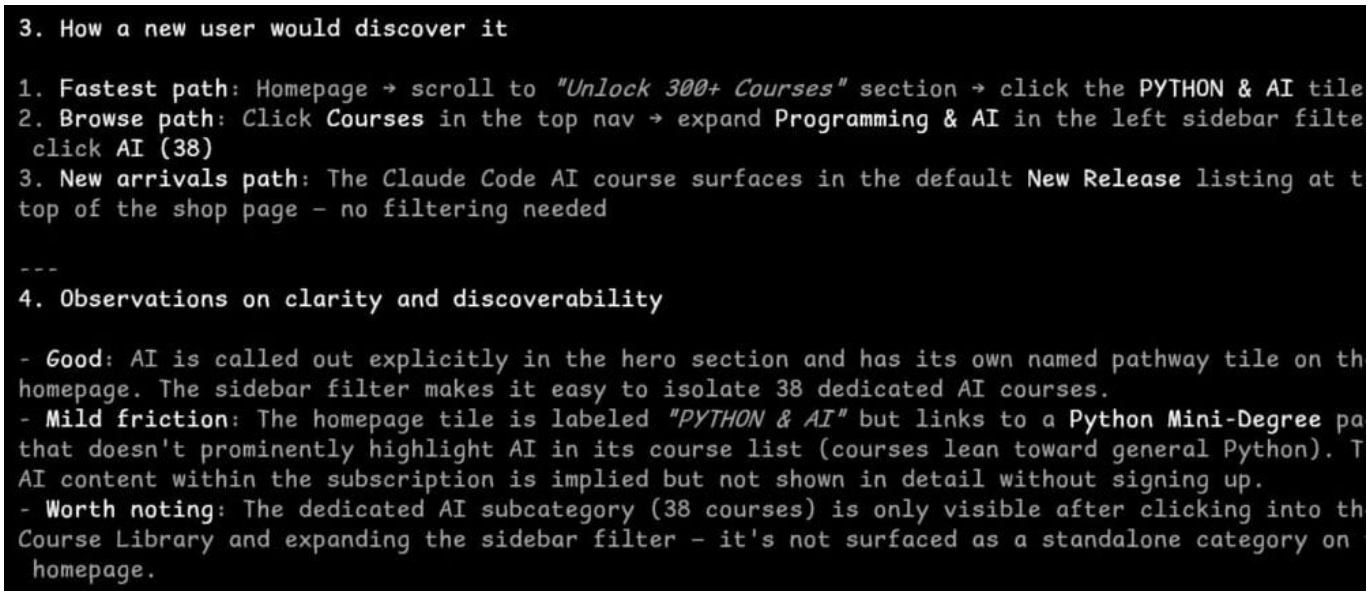
---
2. Where the content is found in the UI

Location	AI Signal
Homepage hero tagline	"programming, game development and **AI**"
Homepage course tiles	**"PYTHON & AI"** pathway tile (one of 8 featured)
Course Library sidebar filter	**Programming & AI (189)** → **AI (38)** subcategory
Course Library – New Releases	"Best Practices for AI-Powered Coding with Claude Code"
Subscription pitch panel	Calls out "data science, computer vision, machine learning"

---
3. How a new user would discover it

1. **Fastest path**: Homepage → scroll to *"Unlock 300+ Courses"* section → click the **PYTHON & AI** tile
2. **Browse path**: Click **Courses** in the top nav → expand **Programming & AI** in the left sidebar filter →

The final sections of the output present the discovery paths and the observations, exactly in the structure described in the skill's output format section:



3. How a new user would discover it

1. **Fastest path**: Homepage → scroll to *"Unlock 300+ Courses"* section → click the **PYTHON & AI** tile
2. **Browse path**: Click **Courses** in the top nav → expand **Programming & AI** in the left sidebar filte
click **AI (38)**
3. **New arrivals path**: The Claude Code AI course surfaces in the default **New Release** listing at t
top of the shop page – no filtering needed

---
4. Observations on clarity and discoverability

- **Good**: AI is called out explicitly in the hero section and has its own named pathway tile on th
homepage. The sidebar filter makes it easy to isolate 38 dedicated AI courses.
- **Mild friction**: The homepage tile is labeled *"PYTHON & AI"* but links to a **Python Mini-Degree** pa
that doesn't prominently highlight AI in its course list (courses lean toward general Python). T
AI content within the subscription is implied but not shown in detail without signing up.
- **Worth noting**: The dedicated AI subcategory (38 courses) is only visible after clicking into th
Course Library and expanding the sidebar filter – it's not surfaced as a standalone category on
homepage.

Conclusion

This is the real advantage of skills. Instead of remembering a big specialized prompt every time, or keeping it pasted into a notepad or note-taking application, you now have a named workflow you can call whenever you need it. We started by giving Claude Code external capabilities through MCP, then added workflow guidance with AGENTS.md, and now with skills we are packaging repeatable expertise on top of those capabilities. The full stack comes together: instructions, external tools, guardrails, and reusable workflows, all working from within a single Claude Code session.

This lesson closes out the course by reflecting on the journey you have taken through Claude Code and the Model Context Protocol (MCP). It restates the core idea that ties every lesson together, outlines the next steps to apply the material to your own work, and points you toward further learning.

A recap of the course

The course followed a progression designed to build your understanding step by step. You began with the foundations of Claude Code and MCP, then moved into setup and practical tool registration so you could start using MCP servers in a real environment.

From there, the material expanded into concrete integrations, covering the following areas:

- The file system MCP server, used for reading and working with files through Claude Code.
- GitHub MCP, applied to a realistic issue-to-pull-request workflow.
- Playwright MCP, used for browser automation and validation tasks.
- AGENTS.md, introduced as a way to add guidance and control to how Claude Code behaves.
- Claude Code skills, used to package repeatable workflows so they can be reused reliably.

The core idea: integration over isolation

The big takeaway across every lesson is that Claude Code becomes much more useful when it is not working in isolation. On its own, it is a capable coding assistant, but its real strength shows once it can interact with its surroundings.

Once Claude Code can reach files, repositories, browser sessions, and reusable workflows, it starts to feel far more like a structured engineering environment than a simple coding assistant. That shift, from isolated assistant to integrated engineering partner, is the central theme of the course.

Applying what you have learned

The natural next step is to take these ideas and apply them to your own projects. A good way to start is to focus on workflows you already run and look for opportunities to bring Claude Code into them.

1. Connect Claude Code to the MCP servers that match your real workflow, rather than using tools only because they exist.
2. Build small but useful skills that wrap the tasks you already perform regularly.
3. Experiment with different browser investigation or repository management workflows to see what combinations are genuinely helpful.

Most importantly, keep thinking carefully about structure, scope, and control. The real value is not just in adding more tools, but in learning how to use them, and how to control them, well.

Where to go next

By this point, you should have a strong practical foundation in Claude Code, MCP, and the workflow patterns that make them useful together. From here, continued practice on your own projects is what will turn that foundation into durable skill.

If you want to keep learning and build more projects, Zenva Academy offers a broad catalog of courses. Zenva serves over a million learners, with foundational courses and specialized tracks in game development, web development, and AI tools. You can continue learning through step-by-step video instruction or through concise lesson summaries directly on the platform.

Conclusion

Thank you for going through this course. The goal has been to give you a solid starting point for building your own real-world Claude Code workflows, and you now have the concepts, tools, and patterns to do exactly that.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

AGENTS.md

Found in folder: **/**

This file documents the project's development workflow and expected conventions. It outlines the purpose, working style, git process, validation steps, and communication guidelines for contributors. It serves as a reference for maintaining consistency during development with AI.

```
# AGENTS.md

## Repository purpose
This repository is a small demo web app used for showing GitHub MCP workflows with Cl
aude Code.

## Working style
- Keep changes tightly scoped to the current GitHub issue
- Do not refactor unrelated code
- Prefer editing existing files over creating new files
- Explain the plan before making code changes
- After implementation, summarize the files changed and the behavior added

## Git workflow
- Use feature branch names in the form `feature/-`
- Write concise conventional commit messages
- Draft pull request text before creating the PR

## Validation
- Run the simplest relevant validation for the project
- For UI changes, verify behavior in the browser
- Mention assumptions or limitations in the final summary

## Communication
- Be concise and practical
- Do not make unrelated improvements
- Ask before making larger structural changes
```

README.md

Found in folder: **/**

This code displays the text "Hello, World!" when rendered. It serves as a simple greeting or introductory message. The content is plain text without any formatting.

```
Hello, World!
```


SKILL.md

Found in folder: **/.claude/skills/playwright-site-investigator**

This code defines a structured workflow for analyzing public websites using Playwright MCP, focusing on navigation, content discovery, and search behavior. It provides specific guidelines for what to examine and how to report findings without accessing private areas or excessive scraping. The output format requires concise summaries of relevant content locations and discoverability observations.

```
---
name: playwright-site-investigator
description: Analyze a public website using Playwright MCP to identify navigation str
ucture, topic discovery paths, visible content areas, and search behavior. Use this f
or public website exploration and structured page analysis.
---

# Playwright Site Investigator

## Purpose
Use Playwright MCP to investigate a public website in a structured and repeatable way
.

This skill is designed for:
- understanding homepage structure
- identifying navigation paths
- exploring topic-specific categories
- using on-page search where appropriate
- summarizing findings clearly

## Workflow
When this skill is used:

1. Open the target public webpage with Playwright MCP
2. Inspect the homepage or landing page before interacting
3. Identify major navigation areas, menus, search fields, and topic discovery element
s
4. Look for content relevant to the requested topic
5. Use search only if it helps clarify discoverability
6. Summarize findings in a concise and structured way

## Rules
- Focus only on public, visible content
- Do not attempt login or private/account areas
- Do not dump large raw page content
- Do not scrape excessively
- Prefer concise summaries over verbose transcripts
- Explain what was found and where it was found in the UI

## Output format
Return:
1. a short summary of what the site offers related to the topic
2. where the relevant content was found
3. how a new user could discover it
4. any observations about clarity or discoverability
```



```
File extension: md
```

file.txt

Found in folder: **/**

This code outputs the text “Hello, World!” when run. It serves as a basic example program.

```
Hello, World!
```

index.html

Found in folder: **/github-mcp-demo-repo**

This HTML file defines the structure of a simple task management web application. It creates a user interface with a form to add new tasks and a section to display and search existing tasks. The page loads external CSS and JavaScript files to handle styling and functionality.

```
<title>TaskBoard Lite</title>
 <!-- External stylesheet for application styling -->


<div class="app-shell">
  <header class="app-header">
    <div>
      <p class="eyebrow">Demo Project</p>
      <h1>TaskBoard Lite</h1>
      <p class="subtitle">A tiny task manager for GitHub MCP demos</p>
    </div>
  </header>

  <main class="main-grid">
    <!-- Section containing the form for adding new tasks -->
    <section class="card">
      <h2>Add a Task</h2>

        <label for="task-title">Task title</label>


        <label for="task-category">Category</label>

          General
          Study
          Work
          Personal


        <button type="submit">Add Task</button>
```



```
      </section>

      <!-- Section for displaying and searching the list of tasks -->
      <section class="card">
        <div class="tasks-header">
          <div>
            <h2>Tasks</h2>
            <p class="small-text">Manage your current task list</p>
          </div>

          <div class="search-box">
            <label class="visually-hidden" for="search-input">Search tasks</label>

          </div>
        </div>

        <ul id="task-list" class="task-
list"></ul> <!-- Container where tasks will be dynamically inserted -->
      </section>
    </main>
  </div>

   <!-- Main application logic -->
```

styles.css

Found in folder: **/github-mcp-demo-repo/src**

This CSS styles a task management application with a dark theme, defining the layout, colors, and responsive behavior for the user interface. It sets up a two-column grid for desktop that becomes a single column on mobile, and styles form elements, cards, and a list of tasks. The visual design includes specific spacing, borders, shadows, and states for interactive components like buttons and completed tasks.

```
/* Global box-sizing rule for consistent sizing */
* {
  box-sizing: border-box;
}

/* Base styles for the page */
body {
  margin: 0;
  font-family: Arial, Helvetica, sans-serif;
  background: #0f172a;
  color: #e2e8f0;
}

/* Centered container for the main application */
.app-shell {
  max-width: 1000px;
  margin: 0 auto;
  padding: 32px 20px 60px;
```



```
}

.app-header {
  margin-bottom: 24px;
}

/* Styling for a small uppercase label */
.eyebrow {
  margin: 0 0 8px;
  text-transform: uppercase;
  letter-spacing: 0.12em;
  font-size: 12px;
  color: #94a3b8;
}

h1 {
  margin: 0;
  font-size: 40px;
}

.subtitle {
  margin-top: 10px;
  color: #cbd5e1;
}

/* Main layout grid: sidebar and content area */
.main-grid {
  display: grid;
  grid-template-columns: 320px 1fr;
  gap: 20px;
}

/* Card component styling */
.card {
  background: #111827;
  border: 1px solid #1f2937;
  border-radius: 16px;
  padding: 20px;
  box-shadow: 0 12px 30px rgba(0, 0, 0, 0.22);
}

.card h2 {
  margin-top: 0;
}

/* Form layout for adding tasks */
.task-form {
  display: flex;
  flex-direction: column;
  gap: 10px;
}

/* Shared input styling for forms and search */
.task-form input,
.task-form select,
```

```
.search-box input {
  width: 100%;
  padding: 12px;
  border-radius: 10px;
  border: 1px solid #334155;
  background: #0b1220;
  color: #e2e8f0;
}

/* Primary action button */
.task-form button,
.delete-btn {
  padding: 10px 14px;
  border: none;
  border-radius: 10px;
  background: #3b82f6;
  color: white;
  cursor: pointer;
  font-weight: 600;
}

/* Destructive action button */
.delete-btn {
  background: #ef4444;
}

/* Header area for the tasks list with controls */
.tasks-header {
  display: flex;
  justify-content: space-between;
  gap: 16px;
  align-items: end;
  margin-bottom: 16px;
}

.small-text {
  margin: 6px 0 0;
  color: #94a3b8;
  font-size: 14px;
}

/* List container for tasks */
.task-list {
  list-style: none;
  padding: 0;
  margin: 0;
  display: flex;
  flex-direction: column;
  gap: 12px;
}

/* Individual task item layout */
.task-item {
  display: flex;
  justify-content: space-between;
```

```
  align-items: center;
  gap: 12px;
  padding: 14px;
  border: 1px solid #273449;
  border-radius: 12px;
  background: #0b1220;
}

.task-left {
  display: flex;
  align-items: flex-start;
  gap: 12px;
}

.task-title {
  margin: 0;
  font-weight: 700;
}

/* Style for a completed task title */
.task-title.completed {
  text-decoration: line-through;
  color: #94a3b8;
}

.task-meta {
  margin: 6px 0 0;
  color: #94a3b8;
  font-size: 14px;
}

/* Utility class to hide content visually but keep it accessible */
.visually-hidden {
  position: absolute;
  left: -9999px;
}

/* Responsive adjustments for smaller screens */
@media (max-width: 800px) {
  .main-grid {
    grid-template-columns: 1fr;
  }

  .tasks-header {
    flex-direction: column;
    align-items: stretch;
  }
}
```

app.js

Found in folder: **/github-mcp-demo-repo/src**

This code manages a task list application. It loads tasks from local storage or initial data, renders them as interactive list items, and handles adding, toggling completion, deleting, and searching tasks. User interactions update the task list and persist changes to local storage.

```
import { initialTasks } from "./data.js";

// DOM element references
const taskForm = document.getElementById("task-form");
const taskTitleInput = document.getElementById("task-title");
const taskCategorySelect = document.getElementById("task-category");
const taskList = document.getElementById("task-list");
const searchInput = document.getElementById("search-input");

const STORAGE_KEY = "tasks";

// Load tasks from localStorage or use initial data
function loadTasks() {
  const saved = localStorage.getItem(STORAGE_KEY);
  return saved ? JSON.parse(saved) : [...initialTasks];
}

// Save current tasks array to localStorage
function saveTasks() {
  localStorage.setItem(STORAGE_KEY, JSON.stringify(tasks));
}

let tasks = loadTasks();
let searchTerm = "";

// Render the task list based on current tasks and search filter
function renderTasks() {
  // Filter tasks by search term matching title
  const filteredTasks = tasks.filter((task) =>
    task.title.toLowerCase().includes(searchTerm.toLowerCase())
  );

  taskList.innerHTML = "";

  // Create a list item for each filtered task
  filteredTasks.forEach((task) => {
    const li = document.createElement("li");
    li.className = "task-item";

    const leftSide = document.createElement("div");
    leftSide.className = "task-left";

    const checkbox = document.createElement("input");
    checkbox.type = "checkbox";
    checkbox.checked = task.completed;
    checkbox.addEventListener("change", () => toggleTask(task.id));

    const content = document.createElement("div");

    const title = document.createElement("p");
    title.className = task.completed ? "task-title completed" : "task-title";
```



```
    title.textContent = task.title;

    const meta = document.createElement("p");
    meta.className = "task-meta";
    meta.textContent = `Category: ${task.category}`;

    content.appendChild(title);
    content.appendChild(meta);

    leftSide.appendChild(checkbox);
    leftSide.appendChild(content);

    const deleteButton = document.createElement("button");
    deleteButton.className = "delete-btn";
    deleteButton.textContent = "Delete";
    deleteButton.addEventListener("click", () => deleteTask(task.id));

    li.appendChild(leftSide);
    li.appendChild(deleteButton);

    taskList.appendChild(li);
  });
}

// Add a new task to the beginning of the tasks array
function addTask(title, category) {
  const newTask = {
    id: Date.now(), // Use timestamp for unique ID
    title,
    category,
    completed: false
  };

  tasks.unshift(newTask);
  saveTasks();
  renderTasks();
}

// Toggle the completed status of a task by ID
function toggleTask(taskId) {
  tasks = tasks.map((task) =>
    task.id === taskId
      ? { ...task, completed: !task.completed }
      : task
  );

  saveTasks();
  renderTasks();
}

// Remove a task by ID
function deleteTask(taskId) {
  tasks = tasks.filter((task) => task.id !== taskId);
  saveTasks();
  renderTasks();
```

```
}

// Form submission handler for adding new tasks
taskForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const title = taskTitleInput.value.trim();
  const category = taskCategorySelect.value;

  if (!title) {
    alert("Please enter a task title.");
    return;
  }

  addTask(title, category);
  taskForm.reset();
});

// Search input handler to filter tasks
searchInput.addEventListener("input", (event) => {
  searchTerm = event.target.value;
  renderTasks();
});

// Initial render
renderTasks();
```

data.js

Found in folder: **/github-mcp-demo-repo/src**

This code exports an array of three task objects with properties for id, title, category, and completion status. It provides sample data that can be imported and used throughout the application. The tasks represent different categories and completion states for demonstration purposes.

```
// Export an array of initial task objects for use throughout the application
export const initialTasks = [
  {
    id: 1,
    title: "Prepare GitHub MCP demo",
    category: "Work",
    completed: false
  },
  {
    id: 2,
    title: "Review Claude Code prompts",
    category: "Study",
    completed: true
  },
  {
    id: 3,
    title: "Plan next video outline",
    category: "Personal",
```



```
    completed: false
  }
];
```

README.md

Found in folder: **/github-mcp-demo-repo**

This Markdown file provides an overview of the TaskBoard Lite project, describing its purpose as a demo for GitHub MCP workflows and listing its features, tech stack, and local setup instructions.

```
# TaskBoard Lite

TaskBoard Lite is a very small vanilla JavaScript web app for managing simple tasks.

This repository is designed as a demo project for showcasing real GitHub MCP workflow
s with Claude Code.

## Features

- Add tasks
- Search tasks
- Mark tasks as complete
- Delete tasks
- Small and beginner friendly codebase

## Tech Stack

- HTML
- CSS
- Vanilla JavaScript

## Run Locally

You can use any static file server. One simple option is:

```bash
npm install
npm run start
```
```

